CALIFORNIA STATE UNIVERSITY, NORTHRIDGE

Hyperspectral Image Classification using Convolutional Neural Networks

A graduate project submitted in partial fulfillment of the requirements

For the degree of Master of Science

In Computer Engineering

By

Troy Kerim Israel

May 2025

The graduate project of Troy Kerim Israel is approved:

_____	_____
Dr. Bingbing Li	Date
_____	_____
Dr. Shahnam Mirzaei	Date
_____	_____
Dr. Myung Cho, Chair	Date

Acknowledgments

First, I would like to express my deepest gratitude to Dr. BingBing Li for allowing me to work on his Astro Cultivators project and be part of the Autonomous Research Center for STEAHM. I am incredibly fortunate to have had such an exceptional mentor whose teachings and expertise have profoundly influenced both my academic and professional growth. His leadership, guidance, and support have been truly invaluable, and I will always be grateful for the inspiration he has provided me.

Another great mentor who had a significant role in my journey is Dr. Shahnam Mirzaei. I am extremely appreciative of all the experiences and knowledge he has shared with me throughout the coursework that I have done with him. The lessons that I have learned under his guidance will continue to be of immense value, accelerating my growth and helping me navigate my academic journey.

I would also like to extend my heartfelt thanks to Dr. Myung Cho for his support and role as chair in my project and for his wonderful support. I am thankful for his insightful feedback and encouragement which has helped me refine my work. I deeply appreciate his willingness to guide me through this process.

Finally, I am thankful to my parents for their unwavering support, hard work, and encouragement throughout my academic journey. Above all, I would like to thank God for providing me with opportunities in my life.

Table of Contents

List of Figures

Abstract

Hyperspectral Image Classification using Convolutional Neural Networks

By

Troy Kerim Israel

Master of Science Computer Engineering

Hyperspectral imaging is a powerful technique for analyzing pixel-level information across various electromagnetic wavelengths. Unlike traditional RGB cameras, which only capture images in three color channels (red, green, blue), hyperspectral cameras can capture hundreds or thousands of spectral bands. These spectral images form three-dimensional data cubes. These data cubes include the spatial dimension, which is the same for RGB images but includes an additional third dimension that contains the spectral dimension. This extra data provides a rich source of information that can be used to improve the image classification accuracy of machine learning and deep learning models.

This project leverages hyperspectral imaging to implement a three-dimensional Convolutional Neural Network (3D-CNN) for classifying the health of sweet potato leaves based on plant necrosis. Plant necrosis is the natural degree of decay. By using hyperspectral images captured by a hyperspectral camera, the 3D-CNN model effectively incorporates both spatial and spectral information to enhance health diagnostic accuracy. The model was developed using the Python programming language and the PyTorch library. To further optimize the model's performance, machine learning techniques like early stopping and cross-validation helped the model achieve an overall accuracy of 83% to 85% in classifying sweet potato leaves into one of six health categories.

The use of hyperspectral data in this project demonstrates the significant potential for improving plant health diagnostics by capturing unique spectral signatures of leaves in varying states of health. By integrating deep learning with hyperspectral imaging, this project provides a unique opportunity and framework for addressing challenges in agriculture health monitoring and offers a scalable approach to early detection and classification of plant health conditions.

Chapter 1

Introduction

The NASA Deep Space Food Challenge is an initiative designed for students, researchers, and professionals to develop innovative food production technologies that require minimal resources, generate little waste, and provide safe, nutritious food for long-duration space missions. Beyond space exploration, a key secondary goal of this challenge is to create food systems that can benefit Earth, particularly in extreme environments and resource-scarce regions. Many of the solutions proposed in this challenge have applications for improving food production in harsh climates, urban areas, and locations where traditional agriculture is impractical. One such approach is smart farming. Smart farming is a modern agricultural method that leverages technology to optimize food production while minimizing resource use and energy efficiency.

Smart farming often incorporates self-contained climate-controlled agricultural systems that maximize space, water, and energy. These systems are particularly beneficial in food deserts, which are in urban or rural areas where access to fresh, healthy food is limited due to economic and geographical constraints. Food deserts pose a serious challenge to public health, as residents often rely on processed or unhealthy food options due to a lack of nearby grocery stores or arable land. One promising smart farming solution for addressing food scarcity in these areas is freight farming.

A freight farm is a repurposed shipping container transformed into a climate-controlled, vertical hydroponic farm. These mobile farms are highly adaptable and can operate in diverse environments, from deserts and snowy regions to dense urban areas. By utilizing hydroponics and controlled climate systems, freight farms enable year-round crop production with minimal water and land usage. Their modular design allows for easy transportation and setup, making them a

scalable and effective solution for food production in remote areas. Freight farms contribute to sustainability by reducing land use, minimizing food transportation across large distances and conserving energy resources. Freight farms align with global efforts to create more resilient and efficient food systems.

The Astro Cultivators project was one solution I contributed to as part of the NASA Deep Space Food Challenge. This project is an autonomous farming system designed for both space and Earth applications. My role focused on plant health diagnostics and is the focus of this report. The plant health diagnostics I worked on utilized hyperspectral imaging to train machine learning models for monitoring sweet potato plants. The overall system integrates environmental sensors to track temperature, humidity, and pressure while incorporating a robotic arm to assist with plant care, growth monitoring, and automated harvesting. This autonomous approach reduces the need for human intervention and is ideal for deep-space missions where crew resources are limited. For usage on Earth, this project makes freight farming even more desirable as our system can be built into an existing freight farm. The crops selected for this system were Snap peas, Red Robin tomatoes, and Snow peas, which were all chosen due to their high nutrition and short harvest cycles. Leafy greens are another food source that we experimented with. Additionally, sweet potato plants were grown in a traditional outdoor farm specifically for the hyperspectral image collection.

Through the application of hyperspectral imaging, machine learning and automation, smart farming technologies such as freight farms, vertical farming and autonomous cultivation systems can revolutionize food production both on Earth and in space. These innovations play a crucial role in addressing food growth challenges, reducing environmental impacts and ensuring sustainable food production in extreme environments.

Chapter 2

Problem Statement

In today's digital world, where visual data is continuously generated, image classification is a crucial subject in computer vision and machine learning. Image classification enables machines to interpret, evaluate, and categorize visual information, making it essential in medical diagnostics, automation, autonomous vehicles, fraud detection, facial recognition, and many other technological fields. The primary goal of image classification is to analyze an image and categorize it into a predefined set of labels. These labels are assigned based on the extracted features of the image. The captured features include edges, textures, and colors, and are learned through complex machine-learning algorithms. The processing of image data typically includes resizing the images to a standard dimension, normalizing pixel values, and applying data augmentation techniques to help the model learn accurately [1].

Typically, most image classification models rely on datasets composed of RGB images. RGB stands for: red, green, and blue. Colored images are made up of different intensity values of those three primary colors. RGB images closely resemble how human vision perceives the world. This makes RGB cameras widely used for object detection, face recognition, and general image classification tasks [5]. Figure 2.1 below shows the primary color scale and how varying the three primary colors can create other colors.

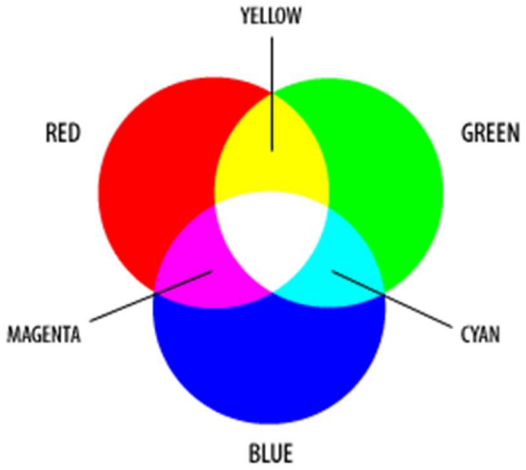

Figure 2.1: RGB Image [2]

Figure 2.2: Individual RGB channels being combined [2]

Figure 2.2 above shows how we can combine or split the individual color channels of an image. RGB images are the primary choice for many object detection and image classification models because they are relatively easy to implement [1]. RGB-based classification is highly effective when objects have distinct color patterns and sharp visual features. Comparing different objects with each other, detecting human faces, or recognizing handwritten text, are the main strengths of RGB-based image classification [7].

However, while RGB data works well for many applications, it has several shortcomings when applied to plant health diagnostics. Plant bacteria, fungi and diseases are a major concern for agriculture [8]. When comparing RGB images to multispectral and hyperspectral images, RGB images are limited to only three spectral bands. Multispectral and hyperspectral imaging can support several hundred spectral bands. RGB sensors can only capture data that is within the visible light spectrum. This limitation prevents RGB sensors from detecting subtle variations in plant stress, disease, or nutrient deficiencies, which often manifest in wavelengths beyond the visible light spectrum [3]. For example, early signs of plant necrosis, chlorophyll degradation, or water stress may not be visibly apparent but can be detected in the near infrared (NIR), shortwave infrared (SWIR), or other non-visible wavelengths [4]. Since RGB cameras cannot capture these critical spectral variations, they are ineffective in providing detailed plant health assessments.

To overcome these limitations, hyperspectral imaging offers a superior alternative for plant health diagnostics. Hyperspectral cameras capture data across hundreds of spectral bands, providing a much more comprehensive view of how plants interact with light at different wavelengths. This allows for the identification of subtle biochemical and structural changes in plant tissues that RGB cameras cannot detect [5]. By leveraging hyperspectral data, classification models can differentiate between healthy and unhealthy plants with far greater accuracy, even at the early stages of plant

development. Unlike RGB-based models that rely primarily on color differences, hyperspectral classification models can analyze spectral reflectance patterns, making them more effective for detecting plant diseases, nutrient deficiencies, and environmental stress factors [6]. Figure 2.3 below from Yee-Rendon et al. [9] demonstrates how alternative algorithms and non-RGB sensors can reveal significant structural changes within plant leaves. The use of NRBVI and NGBVI effectively highlights chlorotic regions, making it easier to distinguish between healthy and affected areas of a leaf. Additionally, healthy plants and infected plants exhibit unique patterns that require classification methods to identify infected plants accurately.

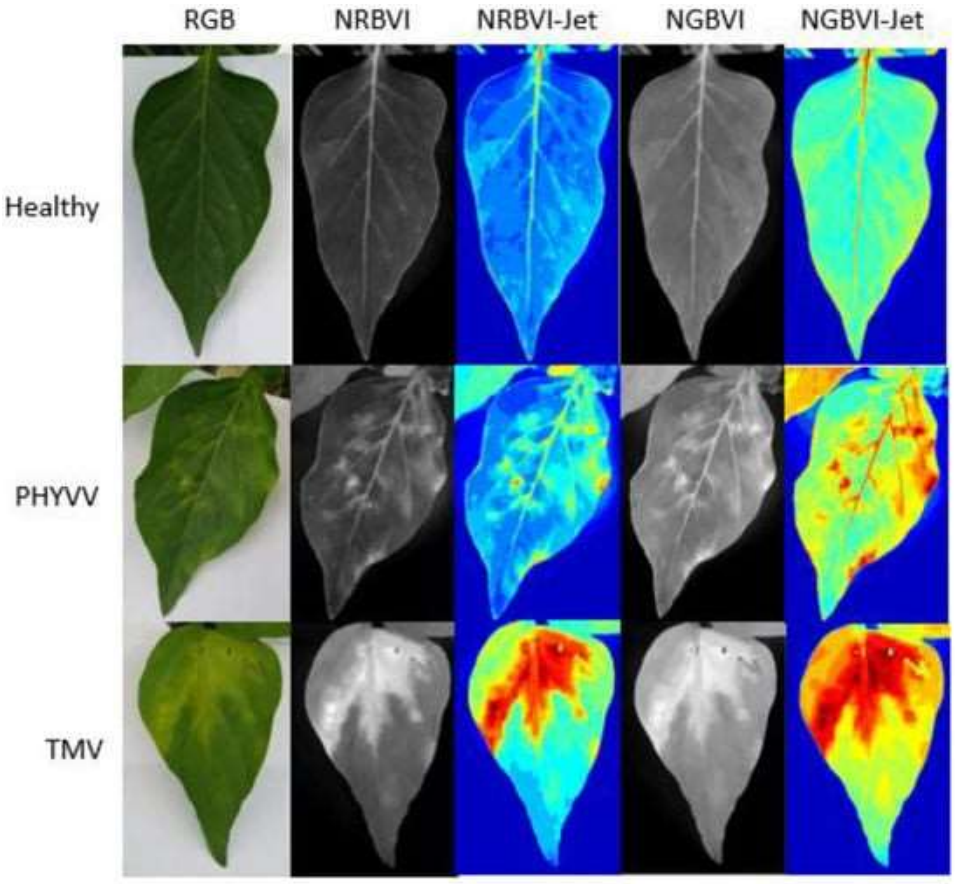

Figure 2.3: Leaf Image Processing Observations [9]

Chapter 3

Literature Review

In smart farming, deep learning and remote sensing are increasingly used to improve crop health, plant monitoring, and classification. One such study, *WeedNet: Semantic Classification Using Multispectral Images and MAV for Smart Farming,* by Inkyu et al., tackles the problem of accurate and efficient weed detection in smart farming [10]. Uncontrolled weed growth can reduce crop yield by competition. Plants are forced to compete with the weeds for nutrients, water, and sunlight, making weed management an essential task for high crop yield. Traditional methods for weed management include manual weed removal or chemical herbicides. These methods are inefficient and pose a risk to both the crops and the environment. Additionally, existing weed classification approaches often struggle with unclear crop-weed boundaries, making it difficult to develop automated weed removal solutions. To address these challenges, the authors developed WeedNet, a deep learning-based system that uses multispectral imaging captured by a micro aerial vehicle (MAV) for real-time pixel-wise classification [10].

The authors faced several challenges, including difficulties with manual data labeling, a lack of public multispectral datasets, computational constraints, and limitations of traditional image classification models [10]. To solve these problems, they collected multispectral images from a controlled test field and applied the Normalized Difference Vegetation Index (NDVI) to distinguish between crops and weeds. They utilized the crops and weeds' unique spectral signatures to help with the identification. A Convolutional Neural Network (CNN) model, based on SegNet architecture, was trained using multiple input channels, allowing it to process spectral data more effectively. The best-performing model achieved an F1-score of 0.8 for weed detection and was deployed on an embedded GPU system, the Jetson TX2, for real-time classification at 1.8 Hz [10].

By integrating deep learning with multispectral imaging, *WeedNet* provided a scalable and efficient approach to automated weed management in smart farming.

Building on the application of smart farming, Khan, et al. introduced the DeepLens Classification and Detection Model (DCDM) to tackle the critical issue of plant disease detection [11]. Traditional disease detection relies on the farmers' observations and expert opinions, which can be subjective and lead to inconsistent diagnoses. The DCDM model automates disease classification for various crops (apples, grapes, peaches, strawberries) and vegetables (potatoes and tomatoes) using a method called transfer learning. The transfer learning is deployed on AWS SageMaker and AWS DeepLens. This allows for both real-time and cloud computing detection. By using cloud computing, the model has scalability and easier accessibility, making it a practical tool for disease monitoring in agriculture [11].

The DCDM model development involved data collection, preprocessing, and model training from AWS SageMaker Studio. AWS DeepLens was used for the system inference. The DCDM model also incorporates a Deep Convolutional Neural Network (DCNN) and integrates with an IoT-powered AWS DeepLens camera to detect and classify leaf diseases. This system produced an average accuracy of 98.78% [11]. Data augmentation techniques were applied to address class imbalances within the dataset. The model was trained using the PlantVillage dataset as well as additional real-world images from Tarnab Farm in Pakistan. The primary challenge was that the PlantVillage dataset contains images with uniform backgrounds, which do not reflect real-world conditions. To mitigate this issue, field images from natural environments were incorporated into training data to increase classification accuracy [11].

Yee-Rendon et al. developed a predictive methodology aimed at detecting visual indicators of viral infections on Jalapeño pepper leaves [12]. The approach specifically targeted the identification of

Tobacco Mosaic Virus (TMV) and Pepper Huasteco Yellow Vein Virus (PHYVV) through the use of deep learning classification models. They incorporated the Red-Blue and Green-Blue Normalized Vegetation Indices (NRBVI and NGBVI) as part of their RGB-based vegetation analysis. These indices, along with their jet-colored visualizations, were utilized as part of the image preprocessing pipeline. The study employed four well-established deep learning models—VGG-16, Xception, Inception V3, and MobileNet v2—to carry out the classification task. To enhance the performance and accuracy of these models, Yee-Rendon et al. applied both transfer learning techniques and various data augmentation strategies [12].

The goal was to identify the most effective pairing of preprocessing method and classification architecture. The models' effectiveness was measured using evaluation metrics such as Top-1 accuracy, precision, recall, and F1-score. The most successful outcome was delivered by a model built on the Xception architecture when trained with the NGBVI dataset, achieving an average Top-1 accuracy of 98.3%. One challenge the study encountered was the limited number of training samples, which was partially resolved through data augmentation. Initial tests also revealed signs of overfitting, which the authors addressed by leveraging transfer learning techniques and expanding the dataset through augmentation [12].

Luo, Y. et al. introduced a novel Convolutional Neural Network architecture known as HSI-CNN to enhance hyperspectral image (HSI) classification, specifically by tackling the issue of having insufficient training and testing data and improving the use of both spectral and spatial information. HSI classification presents a unique challenge due to the data's high dimensionality, which complicates the process of extracting essential features without discarding useful information. Traditional approaches often depend on domain-specific expertise and manual feature engineering, which can hinder model performance. Although deep learning models such as CNNs have proven

effective for general image classification, they struggle to capture spatial hierarchies and are not inherently robust to rotation, limiting their success with hyperspectral inputs [13].

In addition to CNNs, Luo, Y. et al. explored the use of Capsule Networks (CapsNet) as a potential alternative, but they did not yield the desired improvements. To overcome the outlined limitations, the HSI-CNN model converts one-dimensional spectral-spatial data into two-dimensional, image-like structures, enabling more efficient feature learning through standard convolutional processing. Within this framework, the model derives spectral and spatial features from the target pixel and its neighboring pixels, then applies convolutional operations to extract meaningful patterns. These one-dimensional outputs are then reshaped into a two-dimensional matrix, which is passed through a conventional CNN. This conversion is a crucial step, as it boosts the model's capability to learn deep, layered features from hyperspectral inputs while also addressing the challenge of limited data. To further counteract overfitting, the authors proposed a variation called HSI-CNN + XGBoost, replacing the softmax output layer with XGBoost to improve generalization. When benchmarked against alternative methods like HSI-CapsNet and CCS, HSI-CNN consistently outperformed them across a variety of datasets [13].

Eshkabilov, S. et al. utilized hyperspectral imaging along with waveband-based indexing techniques to assess nutrient concentrations in varieties of lettuce. Traditional approaches for determining nitrogen content in lettuce leaves are often expensive, invasive, and time-consuming. To overcome these limitations, the researchers employed hyperspectral imaging data combined with predictive models derived from techniques such as Principal Component Analysis (PCA), Partial Least Squares Regression (PLSR), multivariate regression, and Variable Importance in Projection (VIP) to evaluate nutrient content in plant tissue. The study successfully pinpointed the most informative wavebands in six spectral regions for soil-grown lettuce and four regions for

those cultivated using a nutrient film technique (NFT). The findings revealed a strong positive correlation between the hyperspectral data of lettuce leaves and nitrogen application levels, with high accuracy scores ($R^2 = 0.82–0.99$) especially within the blue and green spectral range between 400 and 575 nm. Based on these results, the study concluded that hyperspectral imaging is a more precise and dependable method for evaluating nutrient content in lettuce plants [14].

Danfeng Hong et al. investigated the application of Convolutional Neural Networks and Graph Convolutional Networks (GCNs) for hyperspectral image classification. While CNNs are effective at capturing both spatial and spectral features, they struggle to model relationships between samples. In contrast, traditional GCNs excel at modeling structural dependencies between data points but come with high computational costs, especially in large-scale remote sensing tasks. Furthermore, GCNs are limited to full-batch learning which restricts their scalability for large hyperspectral datasets. To address these challenges, the authors introduced miniGCN, a mini-batch training approach that allows GCNs to scale more efficiently and classify new inputs without requiring retraining [15]. Additionally, they explored three different fusion strategies to combine CNNs with miniGCNs. Those fusions were: additive fusion, element-wise multiplicative fusion, and concatenation fusion. The proposed methods were evaluated using three widely-used public hyperspectral datasets: Indian Pines, Pavia University, and Houston2013, all of which are satellite images [16]. The study aimed to overcome the computational burden of GCNs, their dependence on full-batch learning, and the inability to classify new data without retraining [15].

To further enhance classification accuracy, Danfeng Hong et al. also proposed FuNet, an end-to-end fusion network that integrates CNNs with miniGCNs for further improvement of hyperspectral image classification. FuNet consists of a feature extraction module, which extracts features from

both CNNs and miniGCNs, and a fusion module, which merges these features using additive, element-wise multiplicative, and concatenation strategies. The experimental results demonstrated that FuNet consistently outperformed single models due to its ability to fuse different spectral representations, with the concatenation strategy (FuNet-C) yielding the best performance. The final results of the study confirmed the superiority of miniGCN over traditional GCNs and the advantages of fusion networks like FuNet over standalone models such as CNNs or miniGCNs, proving the effectiveness of combining multiple spectral learning techniques for hyperspectral image classification [15].

Md Abir Hossen, et al. proposed an AI-based solution that utilizes an unmanned aerial vehicle (UAV) with multispectral imaging to predict total nitrogen (TN) levels in soil for agricultural applications. Monitoring TN is critical for optimizing crop production by enabling precise decisions regarding planting schedules and fertilization strategies. Conventional techniques for analyzing soil health are often costly, time-intensive, and require extensive manual labor, making them impractical for large-scale farming. To address this, the authors combined laser-induced breakdown spectroscopy (LIBS) for reference data, UAV-acquired multispectral imagery, and machine learning algorithms to achieve accurate nitrogen estimations [17]. The imagery was captured using a DJI Mavic 2 Pro drone equipped with a multispectral sensor, while LIBS was employed to determine TN content by analyzing spectral intensity from soil samples. Machine learning approaches such as multi-layer perceptron regression (MLP-R) and support vector regression (SVR) were then trained using a combination of multispectral features, NDVI metrics, environmental factors, and LIBS-based nitrogen readings. One notable challenge involved noise introduced by atmospheric nitrogen during LIBS analysis, which the researchers reduced by applying lower laser pulse energy settings to enhance measurement precision. Ultimately, the study

demonstrated that the predictive models were able to estimate soil nitrogen levels with a root mean square percent error of 10.8%, highlighting the value of integrating UAV sensing and machine learning in precision agriculture and smart farming practices [17].

Tianqi Wei et al. introduced PlantSeg, a large-scale image dataset for plant disease segmentation, designed to overcome the limitations of existing plant disease datasets that often lack detailed labeling, real-world images, and size. The dataset consisted of 11,400 images with segmentation masks across 115 disease categories and 34 plant types. It also consisted of 8,000 images of healthy plants. Unlike traditional datasets that rely on controlled laboratory settings, PlantSeg primarily contains in-the-wild images, ensuring greater diversity and real-world complexity. The study also evaluates state-of-the-art segmentation models, such as Side Adapter Network (SAN), DeepLabv3, DeepLabv3+, and SegNeXt, to establish benchmark performance metrics for future research. The authors found that RGB data was easier to implement rather than using hyperspectral and multispectral imaging, which could enhance plant disease detection and quantification. The authors noted that hyperspectral and multispectral sensors are expensive and impractical in their case [18].

Zhang et al. addressed the problem of leaf angle affecting plant reflectance spectra in hyperspectral imaging, which impacts the accuracy of plant trait assessment. The authors developed an improved image calibration model by integrating hyperspectral images with 3D point clouds to mitigate these angle-related effects. Experiments on corn and soybean leaves revealed that leaf angle significantly alters the NDVI, with species-specific trends. A support vector regression model was created using leaf angle and leaf orientation data from 3D point clouds to normalize NDVI values, effectively simulating a flat leaf. The results show that the new method shows potential for

improving the quality of phenotyping in plant imaging systems. This approach offers a method to remove leaf angle biases, improving the accuracy of plant trait analysis [19].

Previously, I worked on this project and paper with Morales-Badajoz, Anthony, et al., focusing on the development of Astro Cultivators, an efficient autonomous farming system for long-duration space missions. This project began under the NASA Deep Space Food Challenge and its goal was to address the urgent need for autonomous plant growth systems capable of operating in controlled space environments. The proposed growth system was designed to be housed within an enclosure equipped with an advanced automated plant health monitoring system that utilized various cameras and sensors. A key component of the monitoring system was the hyperspectral camera, which was used in plant health diagnostics. The initial goal was for the hyperspectral camera to collect romaine lettuce leaf images and classify them as healthy or unhealthy. The hyperspectral camera was to be integrated into a larger enclosure system, which featured a UR3e robotic arm, an Nvidia Jetson AGX Orin, and additional sensors like temperature and humidity sensors. All devices would be connected to the Jetson, enabling real-time data collection and transmission of data to auxiliary computers for continuous monitoring [20].

One of the key challenges in this project was the lack of publicly available datasets containing hyperspectral images of romaine lettuce, requiring us to create our own dataset. Additionally, we discovered that standard LED lighting was incompatible with our hyperspectral camera and had to use halogen lighting instead. Another challenge was that romaine lettuce proved to be a difficult crop to cultivate in our vertical farming environment, making it less than ideal for our experimental needs. These findings highlighted the complexities of integrating hyperspectral imaging into autonomous farming systems and the importance of optimizing plant growth conditions for space-based agriculture [20].

Building on this approach, a three-dimensional Convolutional Neural Network (3D-CNN) was later developed to classify hyperspectral images of sweet potato leaves, expanding on the use of hyperspectral imaging for plant health diagnostics. Unlike the previous system, which focused on binary classification of romaine lettuce health, this project aimed to categorize sweet potato leaves into six distinct health labels: Healthy, Moderately Healthy, Early Necrosis, Moderate Necrosis, Severe Necrosis, and Dead or Inanimate Object. The classification process involved collecting hyperspectral images of sweet potato leaves, applying the NDVI formula to quantify plant health, and training the CNN model to differentiate between various necrotic stages. By integrating deep learning with spectral data analysis, this approach enhanced plant health diagnostics beyond traditional RGB-based classification, providing a more detailed and data-driven method for monitoring plant conditions in controlled environments.

Chapter 4

Proposed Project

In this chapter, I will outline the development and implementation of my project. The first section provides an overview of how this project is set up and how the dataset was created. The second section dives into the background and design considerations that influenced the project's development. Lastly, the third section covers the source code of the project's implementation, the architecture of the 3D CNN model, the selection of the hyperspectral camera, and the methods and techniques used to train and test the model.

4.1 Project Overview

In this chapter, I will outline the development and implementation of my project. The first section provides an overview of how this project is set up and how the dataset was created. The second section dives into the background and design considerations that influenced the project's development. Lastly, the third section covers the source code of the project's implementation, the architecture of the 3D CNN model, the selection of the hyperspectral camera, and the methods and techniques used to train and test the model.

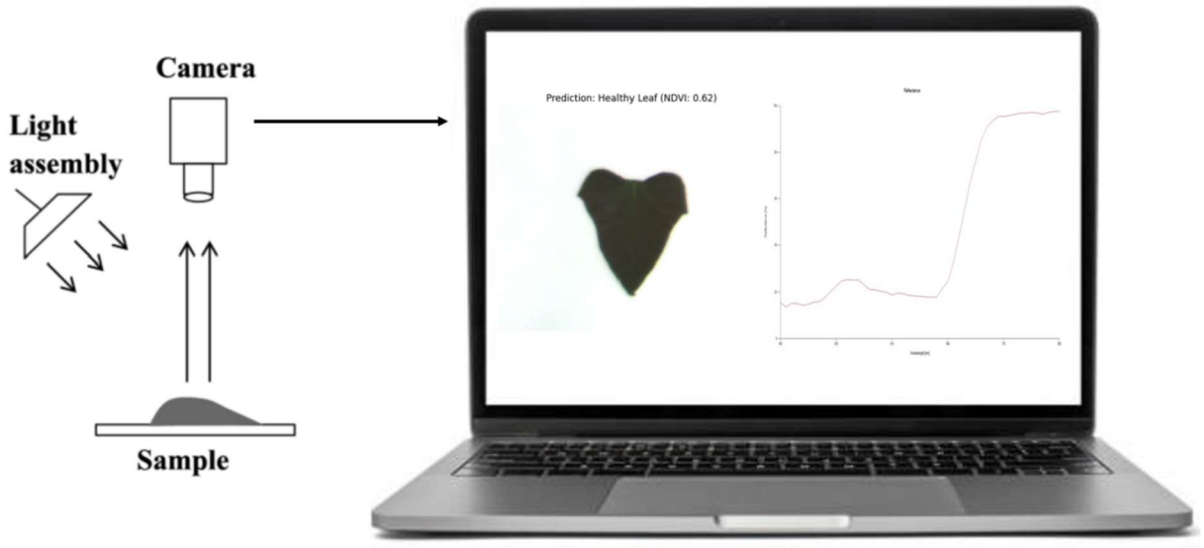

Figure 4.1: Diagram of project [21, 22]

Starting from the left-hand side of Figure 4.1, the diagram illustrates a basic imaging setup, consisting of a hyperspectral camera, light source, and plant sample. This portion of the block diagram was taken from Lodhi et al. [21]. The laptop on the right-hand side is a stock image sourced from an image database [22]. The diagram conceptually demonstrates the camera setup and image classification process. The setup above showcases a sweet potato leaf positioned within the hyperspectral camera's field of view. The light source will be angled off the side, next to the hyperspectral camera, illuminating the viewing region. The camera will be connected to a laptop so the image collection process can proceed using the camera's software. The camera's software enables the user to calibrate the system and capture the hyperspectral images. Once the calibration process is completed, individual sweet potato leaves are sequentially placed in front of the camera, illuminated by the angled light source, and captured as hyperspectral images. These images are then examined for image quality and the generated spectral graph is checked to verify that there

were no errors with the lighting or image. The pictures will then be transferred from the camera directly to the computer's storage and into a dataset folder.

After all images have been collected in the dataset folder, they are loaded into the model and undergo preprocessing. During the preprocessing phase, each image is resized, and image masks are created to help the model learn. The dataset is then divided into batches of 16 images, which are sequentially fed into the model and then shuffled. The model's training loop iterates through the entire dataset, applying cross-validation to split the dataset into smaller folds. Each fold contains a subset of the entire dataset and splits the images into both a training set and a testing set. By using cross-validation, each image will be used for both training and testing at least once. Once the model has processed an image, it assigns it to one of six classification labels: "Healthy", "Moderately Healthy", "Early Necrosis", "Moderate Necrosis", "Severe Necrosis" and "Dead or Inanimate Object".

4.2 Project Design

4.2.1 Hyperspectral Imaging Background

Hyperspectral imaging is a sophisticated form of remote sensing that captures and analyzes light across an extensive spectrum of wavelengths within the electromagnetic range. In contrast to conventional digital cameras, which are limited to capturing images in just three color channels (red, green, and blue), hyperspectral sensors can record data in hundreds or even thousands of narrow, continuous spectral bands. This allows for much richer and more detailed imaging. As a result, each material or object in a hyperspectral image can be identified by its distinct spectral fingerprint—information that cannot be detected by standard RGB cameras or human vision [23, 24]. Figure 4.2 below illustrates the position of visible light within the broader electromagnetic

spectrum, while Figure 4.3 shows how hyperspectral imaging can produce a unique spectral curve that represents the light characteristics of the scene.

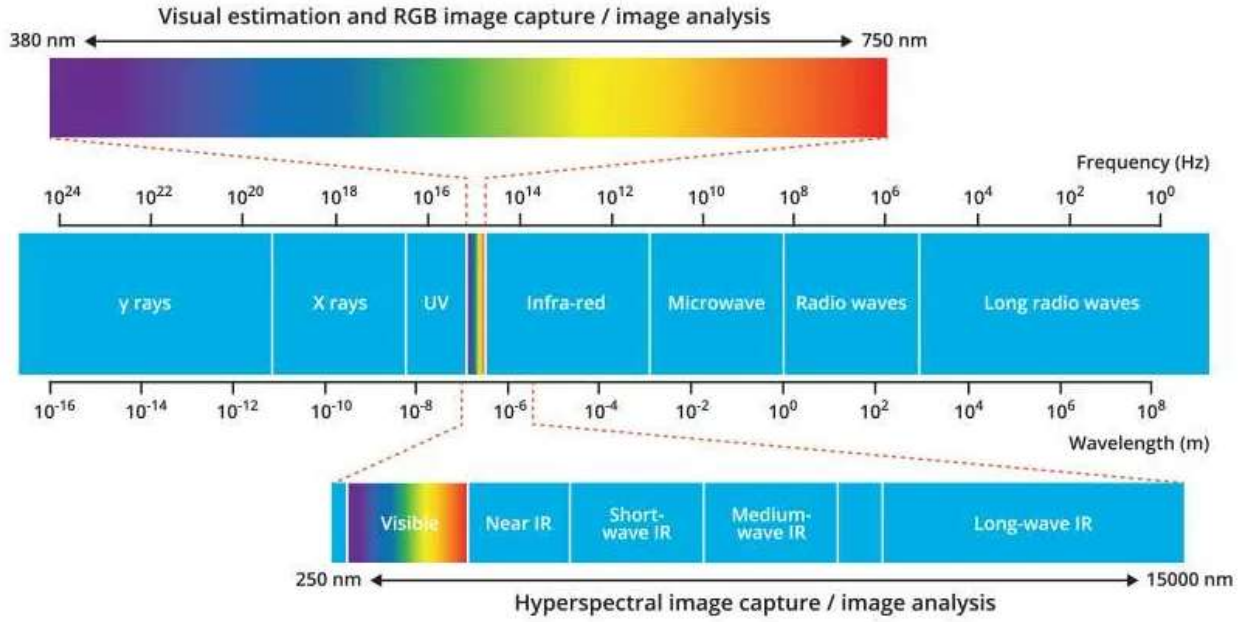

Figure 4.2: Electromagnetic Light Spectrum with visible light [23]

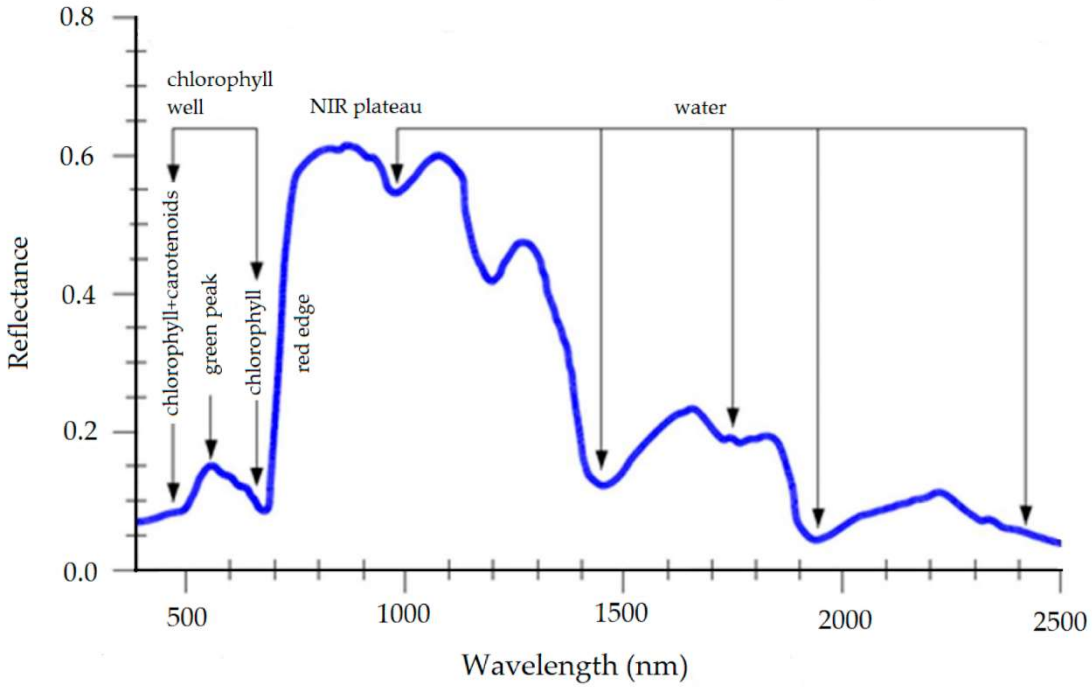

Figure 4.3: Spectral Signature of scene image [25]

Figures 4.2 and 4.3 both show the wavelength range and reflectance of various plant health information. For example, water content information can be found in the near-infrared range and the shortwave infrared spectrum. Also shown in the same figure, Chlorophyll can be found in the red region and when the reflectance percentage is low.

A hyperspectral image is organized in the form of a 3D data cube. Two of these dimensions correspond to spatial coordinates, while the third dimension contains the spectral data. This spectral axis reveals how light is absorbed or reflected by materials at different wavelengths. Many of these wavelengths extend into regions not visible to the human eye. Wavelength ranges such as ultraviolet (UV), near-infrared (NIR), and shortwave infrared (SWIR) offer data far beyond what RGB cameras can capture [23]. By combining spatial and spectral dimensions, hyperspectral imaging supports more precise material classification, enhanced image analysis, and better object recognition. Figures 4.4 and 4.5 illustrate the differences between traditional RGB imagery and hyperspectral data.

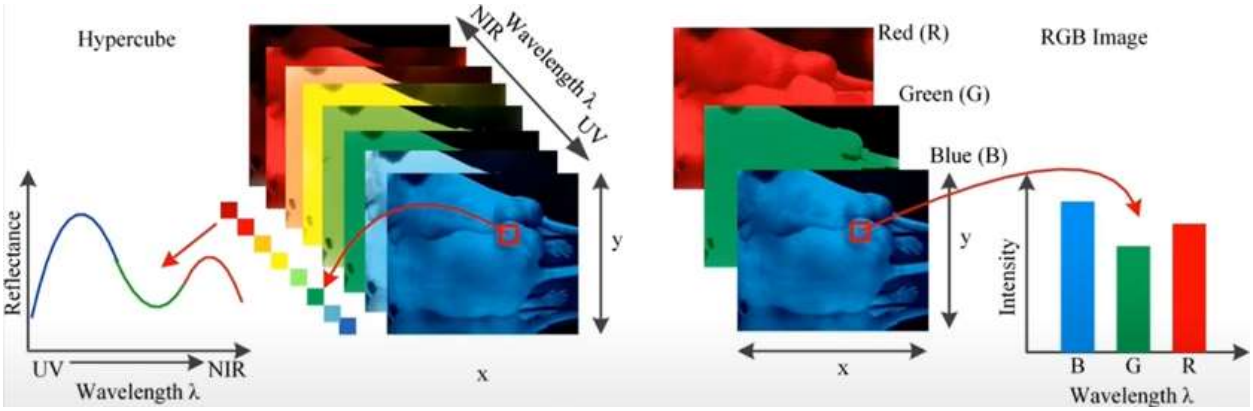

Figure 4.4: Hyperspectral and RGB image differences [23]

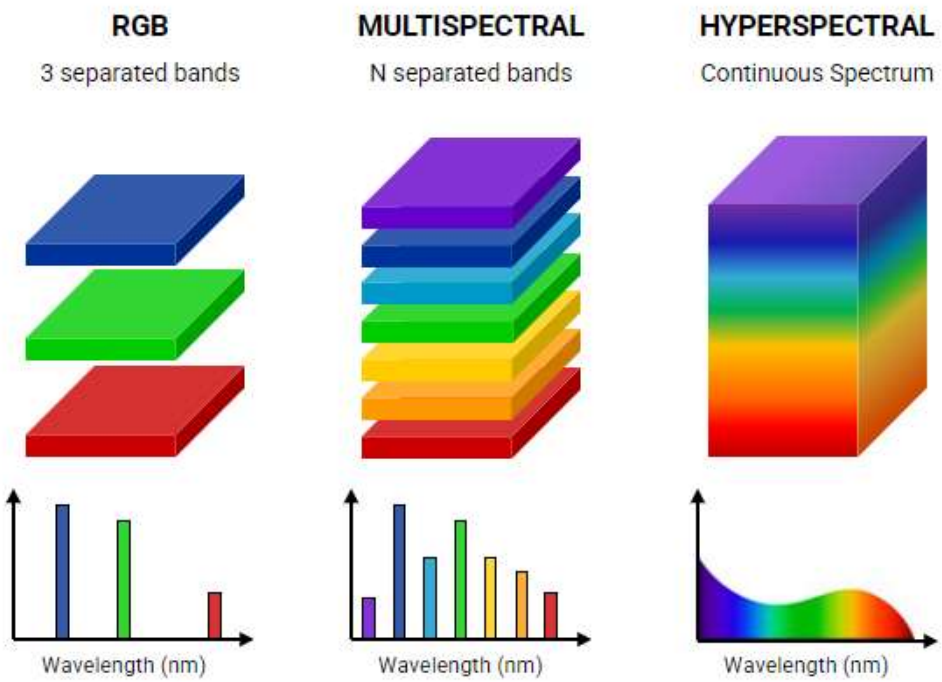

Figure 4.5: RGB, Multispectral, and Hyperspectral band comparison [26]

While RGB imaging is widely used for capturing and analyzing color-based information, it is inherently limited in its ability to differentiate materials and detect subtle variations in spectral properties [23, 24, 27]. RGB cameras can only capture images in three spectral bands and machine learning models based on RGB data primarily rely on color differences to distinguish between objects. In contrast, hyperspectral imaging captures a continuous spectrum of light, allowing for a much more detailed spectral signature for each object. Additionally, RGB images are two-dimensional and can only represent spatial information compared to hyperspectral additional spectral dimension [5, 26, 27]. A useful analogy to visualize the difference between an RGB image and a Hyperspectral image is shown below in Figure 4.6.

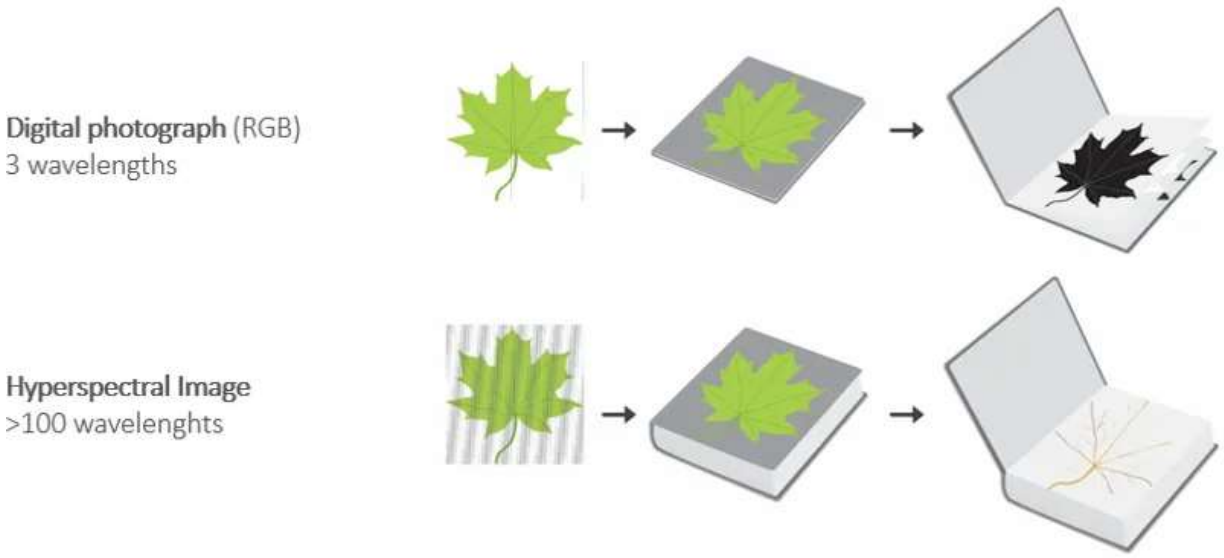

Figure 4.6: RGB data size vs hyperspectral data visualization [24]

Figure 4.6 above highlights the fact that we can visually represent an RGB image as a pamphlet because there are only three colors. Each page in this pamphlet represents a single color. Hyperspectral data can be visualized as a book, where each page will represent a specific spectral band. Since hyperspectral cameras often support dozens of spectral bands, we can see how the number of pages will increase to form a book. This addition of spectral information allows a hyperspectral camera to detect variations in an object's light absorption and reflection. It can uncover anomalies, textures, and patterns that RGB cameras cannot detect too easily. As a result, RGB-based classification models often struggle in scenarios where objects share similar color characteristics. Implementing hyperspectral data into a machine learning or deep learning model can greatly improve the feature selection and provide more precise image classification and object detection [23, 24, 27].

4.2.2 Lighting Background

Light is a form of electromagnetic radiation that travels in waves and is essential for vision, energy transfer, and various scientific applications. When light interacts with a surface, it can undergo different transformations like reflection, absorption, or transmission depending on the material it encounters. Light reflection occurs when light bounces off a surface and changes direction without being absorbed. Light absorption happens when a material takes light energy and converts it into heat or another form of energy. Light transmission refers to the passage of light through a material, such as glass or water, allowing it to continue traveling. Some materials, like clear glass, allow nearly all light to pass through, while others, like frosted glass or water vapor, scatter light, making objects behind them appear blurry. In many natural and technological applications, the combination of reflection, absorption, and transmission determines how objects appear and how they interact with different wavelengths of light [27]. Figure 4.7 shows a visualization of light properties.

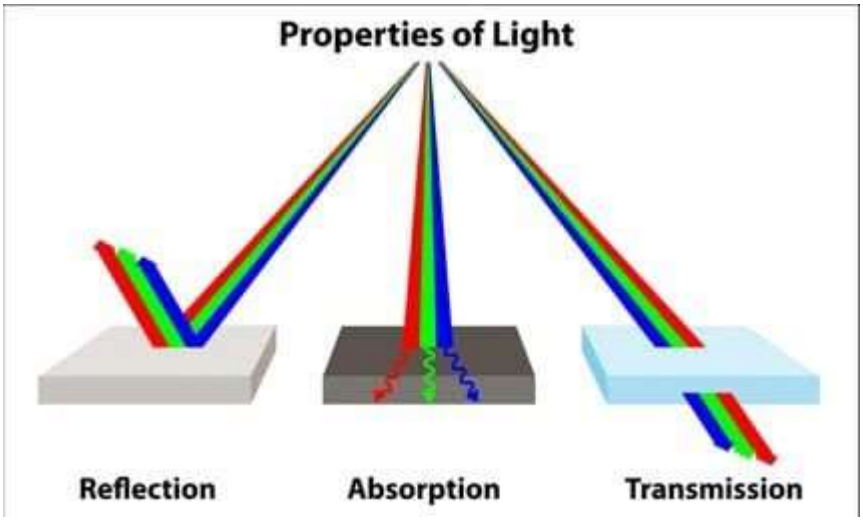

Figure 4.7: Properties of Light [27]

Having the proper light source for your project is paramount. Light sources vary in their properties and applications and are essential for different projects that involve object detection, image classification, or even more directly, image collection. The Sun is the most abundant source of light and provides the full electromagnetic spectrum from ultraviolet to infrared light. The Sun is ideal for outdoor imaging, satellite-based remote sensing, and agricultural monitoring. Light Emitting Diode (LED) is a highly efficient artificial light source commonly used in indoor imaging, controlled plant growth environments, and laboratory-based analysis due to its energy efficiency, adjustable wavelength outputs, and long lifespan. Halogen lighting is another light source that produces light by passing electricity through a tungsten filament in a gas-filled bulb. It Is capable of emitting a continuous light spectrum with strong infrared components and is useful for hyperspectral imaging, spectroscopy, and experiments that require the full light spectrum for illumination. Fluorescent lighting, often used in office and laboratory settings and provides a more energy-efficient alternative to incandescent bulbs [27, 28]. Figure 4.8 shows the electromagnetic spectrum of various light sources.

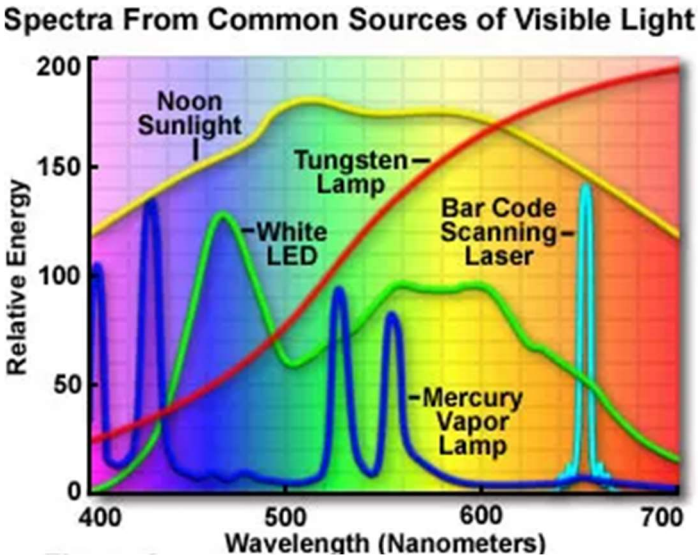

Figure 4.8: Spectra from common sources of visible light [28]

4.2.3 Normalized Difference Vegetation Index Background

The Normalized Difference Vegetation Index (NDVI) is a ratio that measures the amount of vegetation in an area. NDVI measures the amount of near-infrared light reflected by vegetation and the amount of red light absorbed by vegetation [29, 30]. NDVI is one of the most common metrics used in remote sensing because it can be used as an entryway for researchers and scientists to help diagnose plants or vegetation health conditions [31]. NDVI is always in the range from -1 to +1. NDVI is often applied with hyperspectral imaging and is typically calculated from satellites [29]. Figure 4.9 below shows the formula for NDVI.

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

Figure 4.9: NDVI formula [31]

When negative values are calculated, this can mean clouds, water or snow have been observed from the spectral sensor [31]. Values near or at 0 mean that rocks or bare soil were observed. Values that are above 0.3 are vegetation at various health levels [30, 31]. The closer the NDVI value is to 1 the healthier the plant or vegetation is [29, 30, 31]. To get accurate NDVI values from a vegetation image, the sensor that collected that image light source must be either direct sunlight or from a halogen light source [28]. Figure 4.10 shows a visual scale for various NDVI values.

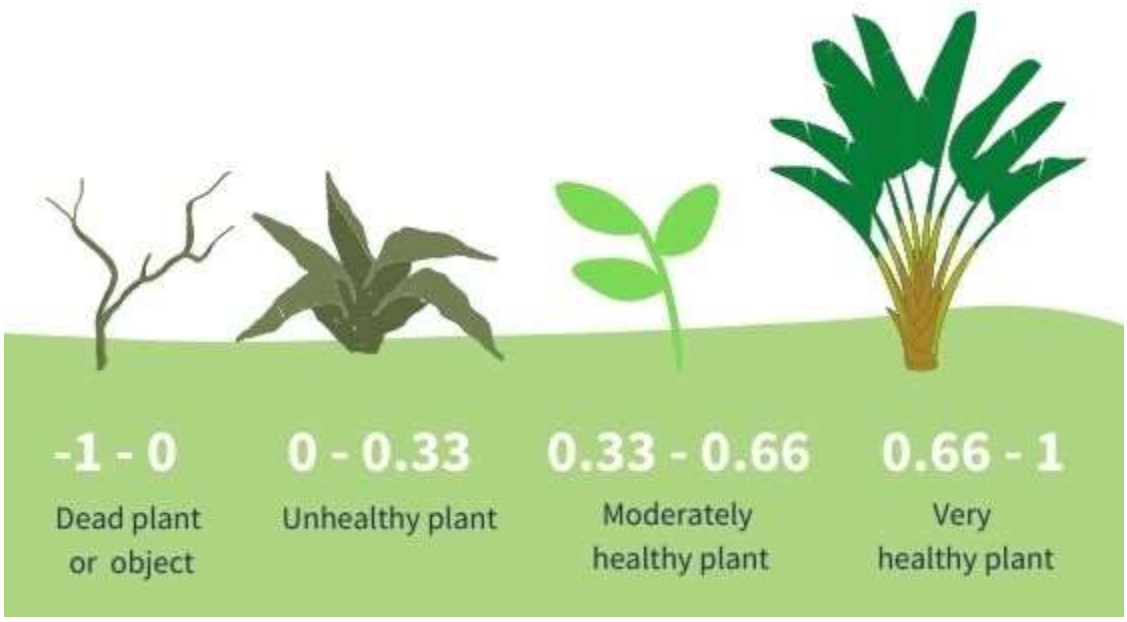

Figure 4.10: Various NDVI threshold levels

How much Chlorophyll content inside a plant will determine how strongly or weak light is absorbed. When a plant becomes stressed, dehydrated, or affected by a disease, the plant tends to absorb more near-infrared light than reflecting it. This is caused by the decay in the plant's internal organic structures. Observing the changes in near-infrared and red light regions can be an indicating factor for the changes going on inside the plant [29]. Figure 4.11 shows how the changes in NIR and red-light absorption and reflection affect a tree. Figure 4.12 illustrates how a healthy, stressed, and dead leaf reflects and absorbs different light waves.

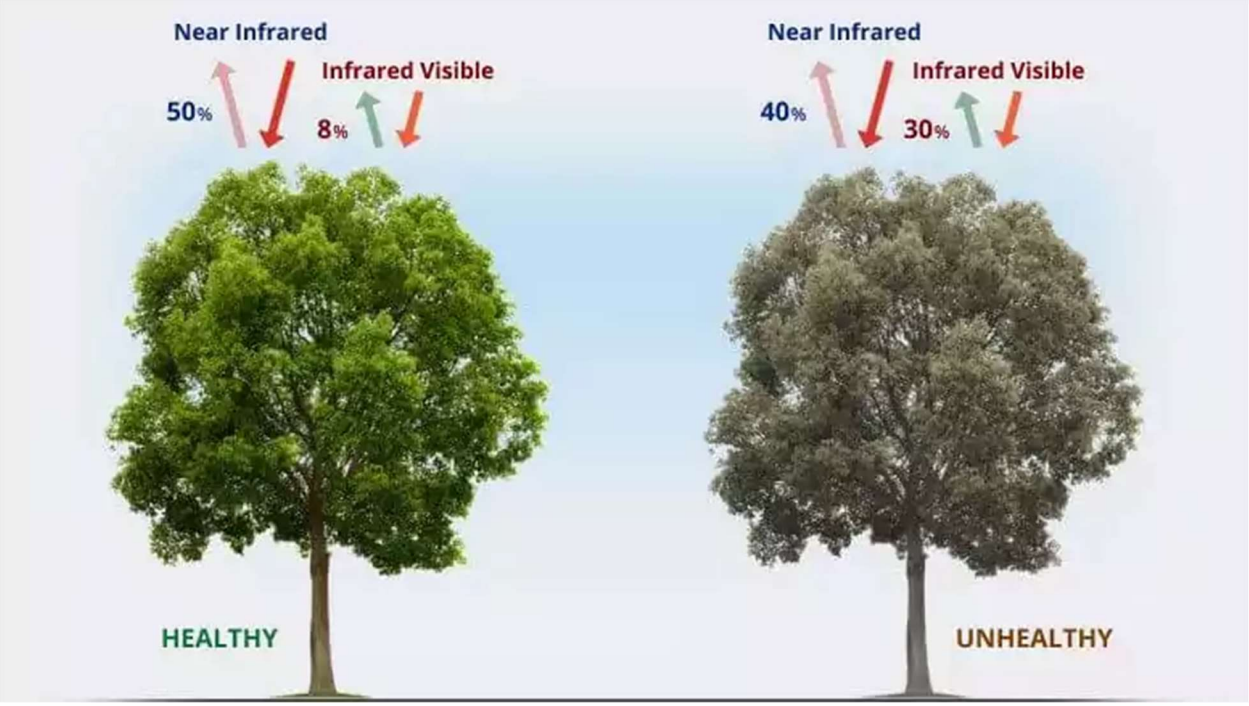

Figure 4.11: Healthy vs Unhealthy light absorption and reflection [30]

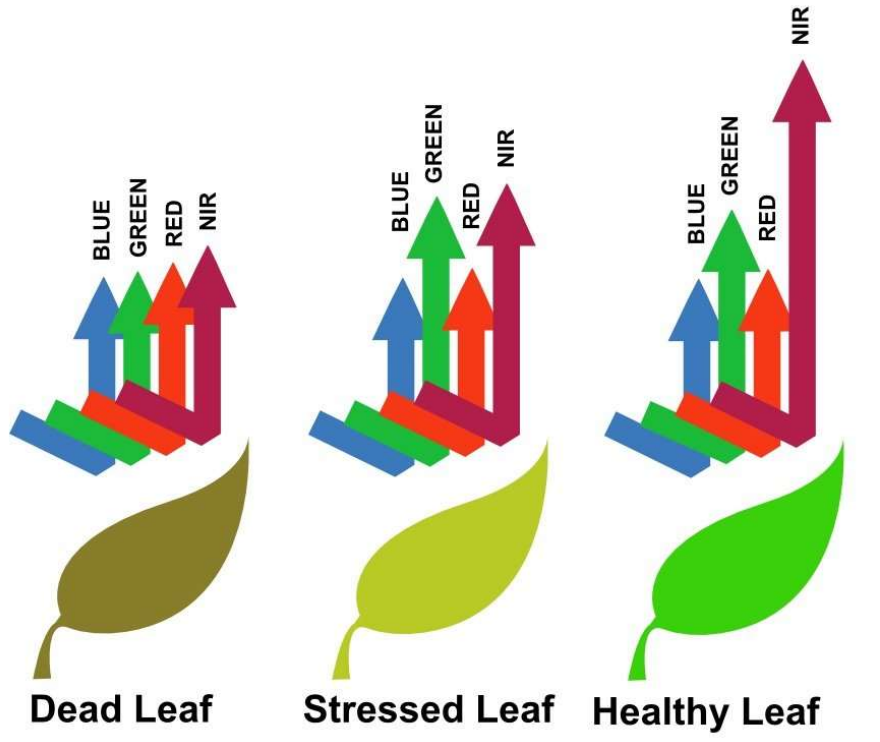

Figure 4.12: Leaf absorption and reflection based on overall health [33]

NDVI can be applied to hyperspectral data with spectral signatures. A hyperspectral camera can be used to take an image of a plant or forest. That image is then saved as a 3D hypercube. Depending on the spectral camera, which can support large spectra ranging from visible light to short wave infrared (SWIR), we can slice the hypercube at specific bands and process them in the NDVI formula. We would slice a specific band at a wave close to NIR and a wavelength close to red light. The typical frequency used for red light is between 620 nm and 710 nm and 750 nm to 850 nm for NIR [32]. This depends on the spectral camera. By using a spectral camera, we can then gain access to that spectral signature that can be generated after an image is collected. Figure 4.13 shows the typical range for NDVI concerning healthy and unhealthy vegetation.

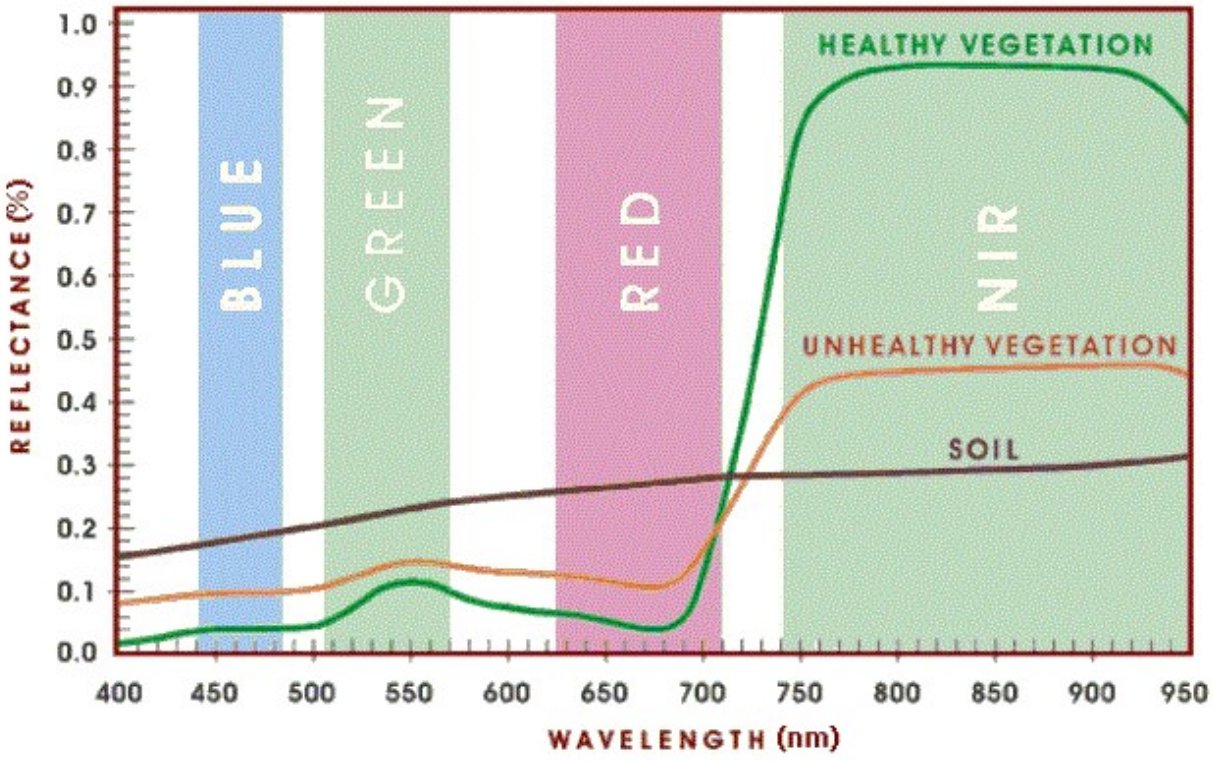

Figure 4.13: NDVI spectral chart for vegetation and soil [33]

In Figure 4.13 we can see where exactly the red and NIR wavelengths are. Depending on our hyperspectral camera, we would want to choose a spectral band that lies within those two regions. We can see that healthier vegetation in the NIR region will have a much higher reflectance while unhealthier vegetation will be much lower. Similarly, healthier vegetation will have a lower reflectance in the red region while unhealthier vegetation will have an elevated reflectance.

4.2.4 Sweet Potato Dataset

The dataset for this project consists of hyperspectral images of sweet potato leaves. It was created as a custom dataset due to the lack of publicly available spectral datasets for any plant or leaves. The sweet potato leaves were selected due to their large surface area, year-round availability, and abundant supply. Sweet potato leaves were a far more suitable choice than the previously used tomato or lettuce leaves used in Astro Cultivators. All leaves were provided by CSUN's Marilyn Magaram Center (MMC) and Figure 4.14 below shows the exact outdoor farm.

Figure 4.14: Outdoor sweet potato farm at Marilyn Magaram Center

The data collection process took place in July and September and involved the manual cutting of leaves from the plant stems. A total of 10 visits to the MMC garden occurred and, in each visit, approximately 70-100 leaves were collected at a time. Garden visits were weekly, and pictures were taken daily. Figure 4.15 shows the leaves being collected. Once the leaves were removed from their stems, they were then transported to the camera setup, where individual hyperspectral images were captured. After the images were taken, the leaves would then be placed into plastic bags and then into a refrigerator to preserve them for as long as possible. This preservation process in shown in figure 4.16. In total, 1,123 images were captured with the hyperspectral camera. Later

in September, an additional 600 images were artificially created using data augmentation techniques, which will be discussed later.

Figure 4.15: Sweet potato leaves being collected

Figure 4.16: Leaves stored in plastic bags for refrigeration

Initially, half of the 1,123 images in the dataset were healthy leaves and entirely green. This caused the dataset to be imbalanced initially. Since the farm had no reported plant health issues, there was no natural way to collect unhealthy sweet potato leaf images. To ensure a balanced dataset, a controlled method was used to induce leaf deterioration. Beginning in September, leaves would no longer have an image taken until they have been detached from their host plant for more than a week or so. The control method involved leaving the stems attached to the leaves and then placing the stems into a water tray. Figure 4.17 below shows the water trays in use. It took approximately 7-10 days for the leaves to gradually transition from green to yellow, then brown-yellow and eventually brown. Also, Figure 4.18 below shows how the leaves transition over time. This control method allowed the dataset to have more leaf samples of various health conditions. It should also be noted that these leaves during the manual degradation were not stored in the refrigerator initially. They would have their image taken with the hyperspectral camera and then placed back into the water trays. Eventually, after a month, I started to place them into plastic bags and store them inside a refrigerator to preserve the yellow and brown leaves.

Figure 4.17: Sweet Potato leaves in a water tray

Figure 4.18: Sweet Potato leaves deteriorating over time

In Figure 4.18, not all the leaves would turn yellow or brown at the same rate. Some leaves like in the upper section of the figure remained relatively green for more than 2 weeks. This posed a slight challenge as I did not require fully green leaves. Even though some of the leaves shown in that figure are yellow, the leaves that had slight yellowing would not be classified as "Moderately Healthy" or "Early Necrosis" but would be classified as "Healthy". This issue will be explained later in Chapter 5, but to resolve it, those leaf images would not be collected until they turned yellow. Figures 4.19 and 4.20 show another example of the leaves degrading over time.

Figure 4.19: Healthy Sweet Potato leaves left outside

Figure 4.20: The same leaves were left outside without water for three days

Figures 4.19 and 4.20 are related to each other. They both show another experiment done on the leaves to see how long it would take the leaves to degrade if left completely without water. These leaves were left outside for approximately three days. When left without water, the leaves rapidly degrade to brown and shrink in overall size. This happened much faster than other leaves that remained in their water trays during off hours.

4.2.5 Data Augmentation

Data augmentation was necessary to expand the dataset beyond the 1,123 original images, particularly to address performance issues related to the six classification labels. An additional 600 images were manually created to help the model's ability to learn from the dataset. I created the data augmentation by rotating each image 90 degrees to the left and right. The goal was to see if the dataset's variability would increase, and the model's performance would improve. Chapter 5 will discuss in more detail why data augmentation was necessary. Figures 4.21 to 4.23 illustrate examples of these transformations. Additionally, masked images underwent the same augmentation process to ensure compatibility with preprocessing methods.

Figure 4.21: Original masked image

Figure 4.22: Masked image rotated 90 degrees to the right

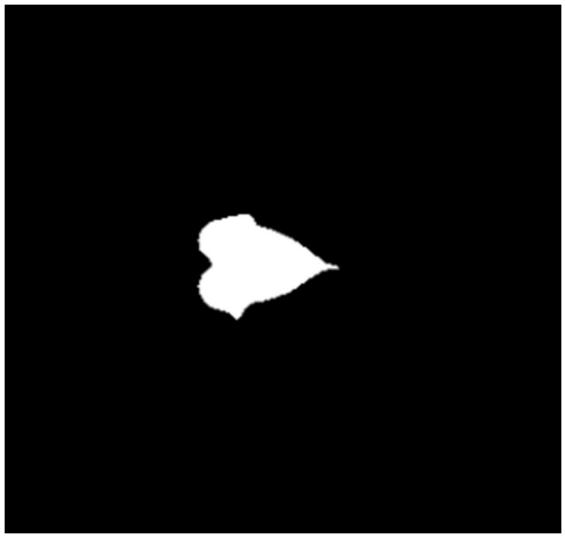

Figure 4.23: Masked image rotated 90 degrees to the left

Figures 4.21 to 4.23 above show an original masked image of a sweet potato leaf and its two augmented images. The original spectral files were also augmented as well. The purpose of these augmentations was to address performance issues that will be explained later in Chapter 5.

Of the 600 augmented images, approximately 100 were black-and-white images, representing non-leaf scenarios such as camera malfunctions or an obstructed view. These were designed to train the model to properly classify "Dead or Inanimate Object" cases. To create these images, the camera's lens cap was left on while capturing an image, simulating a completely dark scene. Then, an image was taken with the camera facing a wall with a light on to represent an empty background. To ensure sufficient representation of this class in the dataset, 50 copies of each scenario were generated. Figures 4.24 and 4.25 below show the black-and-white augmented images.

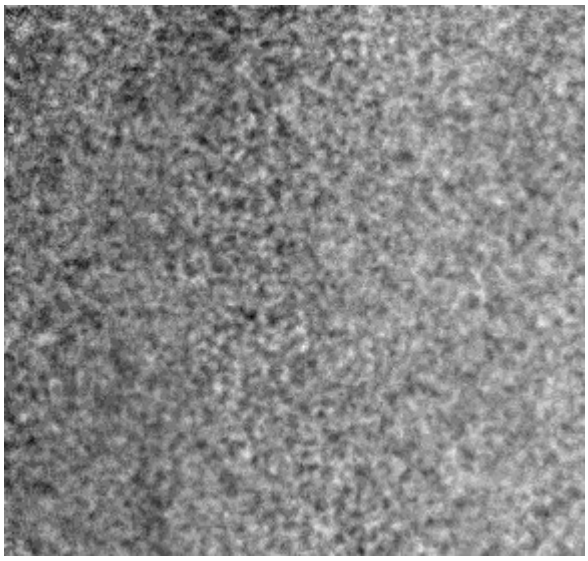

Figure 4.24: White image augmentation

Figure 4.25: Black image augmentation

The two figures above have a distinct look. The white image in Figure 4.24 appears to be a bit fuzzy and the reason for that is not because the camera is out of focus or is not calibrated correctly. The reason for this is due to the camera filter. The camera's filter was set to "NDVI" in the view mode when the image was captured. Additionally, this is to simulate the camera being attached to a moving system where the camera would be set up with a robotic arm and where the camera

would be moving to different locations. In doing so, the camera could take a picture of a non-uniform surface. Figure 4.25 is the black image to simulate a completely dark environment. This simulates a scenario where the camera could accidentally take a picture when the light source is nonfunctioning or if the robotic arm scenario is where it positions the camera in a very dim-lit area. Like Figure 4.24, the white spots shown in Figure 4.25 are due to the same camera filter. The performance of these two image types will be discussed in Chapter 5.

4.2.6 Basics of Hyperspectral Cameras

A hyperspectral camera is used to capture spectral photographs of objects. It has a different imaging process than that of an ordinary digital camera. A digital camera will focus light reflected or transmitted by visible light through an optical lens onto a light-sensitive element. This will format the image to be either colored or black and white. Hyperspectral cameras use a multi-channel spectral sensor that can simultaneously capture the spectra data in hundreds or thousands of narrow bands [34]. As mentioned earlier, a large portion of spectra are typically beyond visible light. The type of hyperspectral camera will determine what electromagnetic wavelengths you will be able to work with and the number of spectral bands. Digital cameras can only support three color channels: red, green, and blue.

For data acquisition, digital cameras will typically use a single exposure to capture an image. Hyperspectral cameras will use a continuous scanning sensor to capture the spectral data at a fast rate. Digital cameras are more suitable for scene shooting while hyperspectral cameras are more suitable for real-time monitoring, continuous data streaming, and subtle chemical and visual changes. Lastly, hyperspectral cameras are more expensive, and complex compared to digital cameras [34]. One of these complexities occurs with lighting which will be discussed in section 4.3.

4.2.7 Overview of Convolutional Neural Networks

Modern deep learning and machine learning models are developed to address a broad variety of complex tasks, including speech recognition, image analysis, data enhancement, object detection, and image classification. These intelligent systems allow for effective data interpretation, pattern discovery, and automation across multiple domains. Different algorithms are tailored to excel in specific applications. One widely used model in computer vision is the Convolutional Neural Network (CNN), which has proven highly effective in image classification tasks. Other deep learning and machine learning architectures, such as Recurrent Neural Networks (RNNs), are well-suited for processing sequences and are commonly used in language-related applications. Long Short-Term Memory (LSTM) networks, a specialized form of RNNs, are frequently applied to time-series prediction, text generation, and financial forecasting. Generative Adversarial Networks (GANs) are commonly implemented for tasks involving synthetic image generation and deepfake creation [35, 36]. In this project, CNNs were selected as the model of choice due to their proven effectiveness in image classification and their adaptability to handle complex input formats such as hyperspectral data.

At their core, neural networks—including CNNs—are structured with three essential components: an input layer, one or more hidden layers, and an output layer. The input layer functions as the entry point, receiving and feeding raw data into the network. Hidden layers act as the internal processors, where data undergoes a series of transformations to identify patterns and extract features. These transformations allow the model to learn from the data and improve prediction accuracy. A neural network may include several hidden layers, each deepening the network's ability to model complex relationships. Finally, the output layer produces the model’s final prediction or

decision, based on the information processed by the preceding layers [35, 36]. Figure 4.26 illustrates this foundational network architecture.

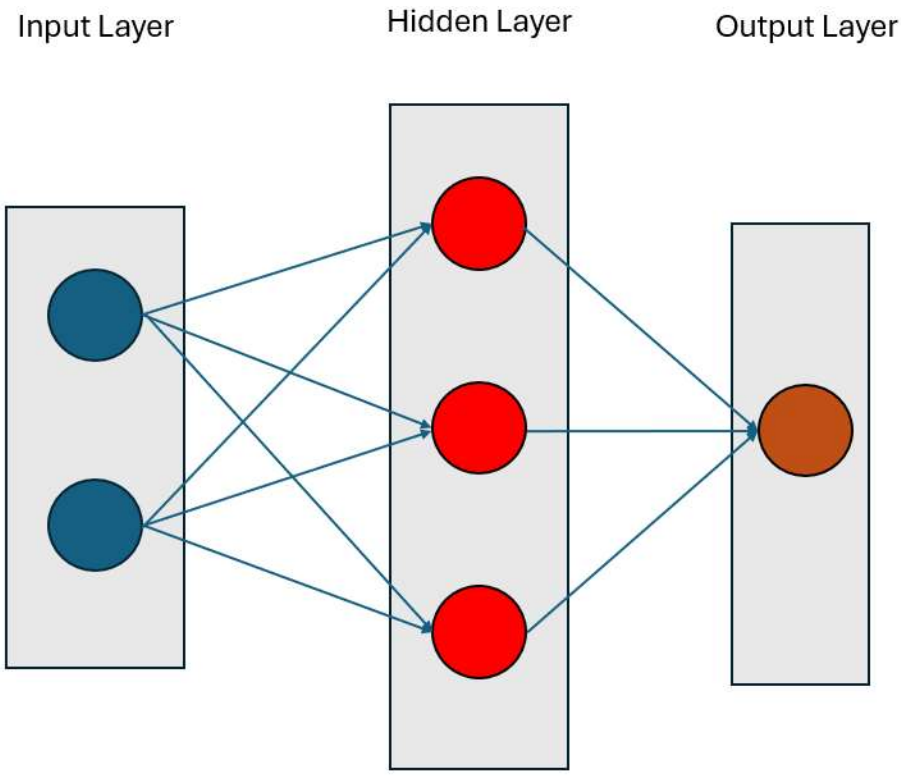

Figure 4.26: Basic neural network structure

Figure 4.26 illustrates the basic architecture of a neural network. A neural network will typically consist of an input layer, multiple interconnected layers with neurons, and an output layer. Neural networks often contain hundreds or thousands of neurons per layer. Some deep networks even contain hundreds to thousands of hidden layers. Before exploring Convolutional Neural Networks in more detail, it is important to first discuss the key components present in all neural networks, including CNNs. One of the most critical elements of any deep learning model is the neuron. This basic processing unit is found in both the hidden and output layers [35, 36]. Figures 4.27 and 4.28 show the basic structure of a neuron and how it acts as a computational unit that receives an input, processes it, and then passes that information to the next layer.

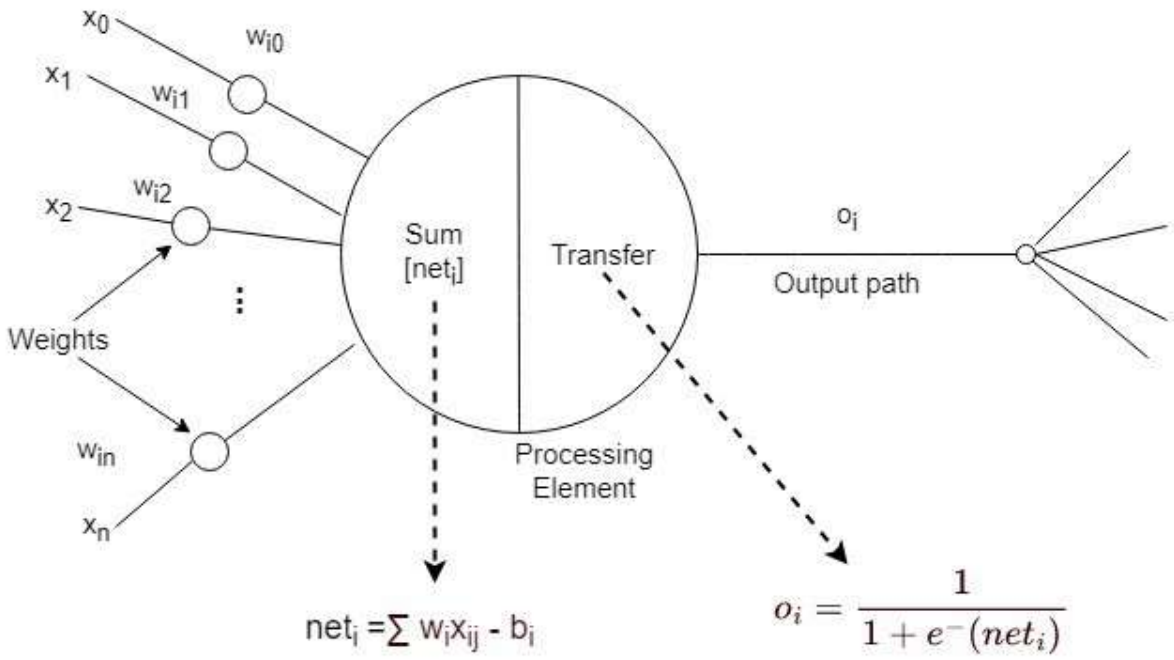

Figure 4.27: Inside a typical artificial neuron [38]

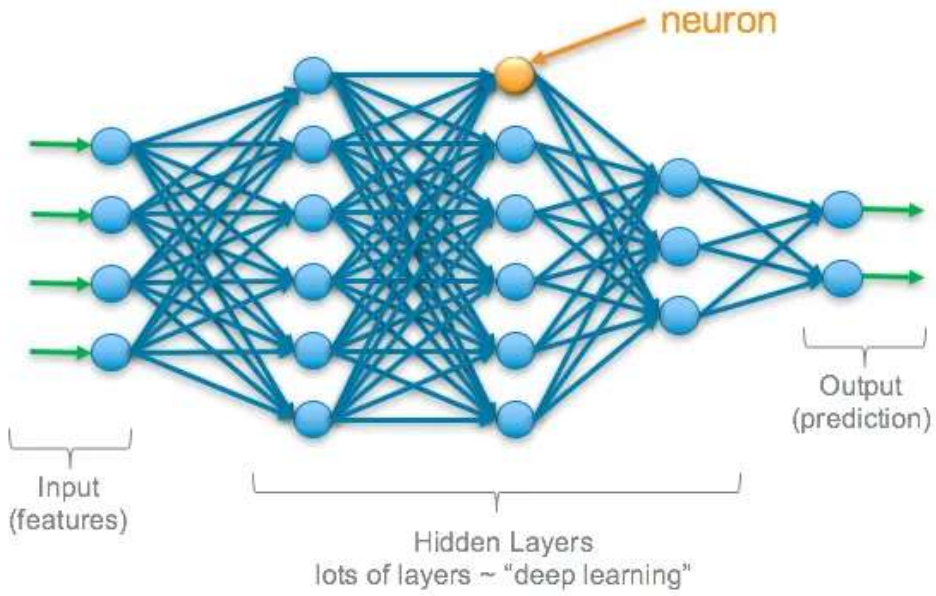

Figure 4.28: Multilayer neuron network [39]

Figure 4.27 above provides a detailed view of a single neuron and demonstrates how it processes its input channels through weights, biases, a summation function, and an activation function [39]. The inputs ($x_0, x_1, x_2, \ldots x_n$) represent the data received from the preceding layer, each associated with a weight ($w_i0, w_i1, w_i2, \ldots w_in$) that determines the strength of the connection between neurons. These weighted inputs are summed together with a bias term. The bias term is added before the result of the neuron is passed through the activation function. The bias allows the neuron to adjust its output even when all input values are zero. This allows the neuron more flexibility when learning complex patterns [37, 38, 40].

Figure 4.28 places the neuron within the broader structure of a neural network. This figure illustrates how multiple neurons are all interconnected with each other across the input, hidden, and output layers [37]. Each neuron receives information from preceding neurons. Before sending information to the next layer, it will first process its input information by using weights, bias and an activation function. Weights and biases play a critical role in how a model learns patterns. Weights are trainable parameters that are continuously updated during training. This enables the model to identify which features are the most important when making a prediction. The bias further refines the model's ability to adapt to different data distributions. Because weights and biases control the flow of information through the network, they significantly influence the model's accuracy and its ability to generalize new data [38].

An activation function is a computational mechanism used to transform the output of a neuron. Its primary role is to introduce non-linearity into the neural network, enabling the model to learn and represent complex data relationships. Without this non-linear behavior, the network would function similarly to a basic linear regression model, even if it contained multiple hidden layers [37]. This limitation would significantly hinder the network's ability to solve more advanced or

nonlinear problems. The activation function helps determine whether a neuron should be triggered based on a combination of the neuron's inputs, associated weights, and a bias value. Introducing non-linearity into the network allows it to better detect and learn intricate patterns in the data, as illustrated in Figure 4.29.

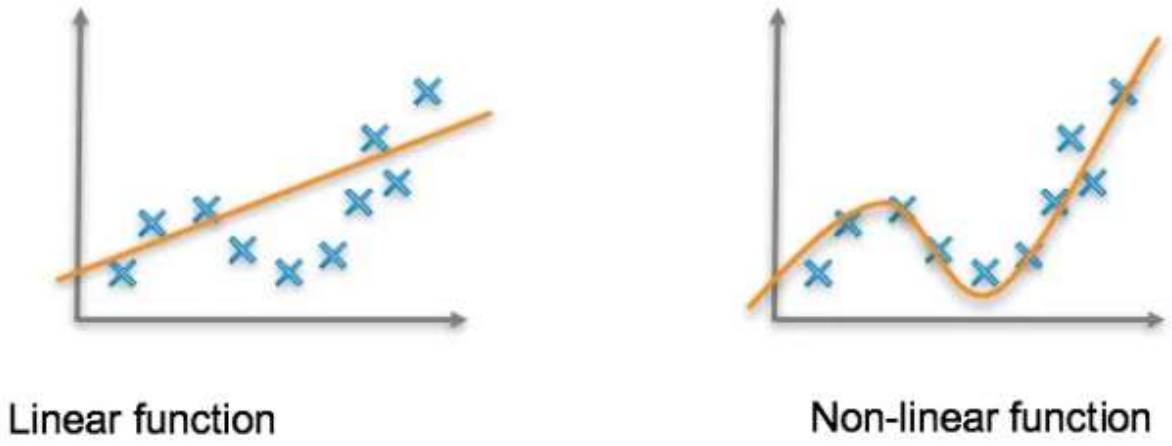

Figure 4.29: Linear vs non-linear function fitting [39]

Neural networks utilize several well-established activation functions to enhance learning performance. Among the most widely used are sigmoid, hyperbolic tangent (tanh), and rectified linear unit (ReLU) functions. Both sigmoid and tanh functions are particularly useful in binary classification tasks, where outputs need to be interpreted as one of two possible outcomes. The sigmoid function compresses input values into a range between 0 and 1. However, this function is susceptible to the vanishing gradient problem—where very high or very low input values result in gradients that approach zero—and its outputs are not centered around zero, which can hinder learning [37, 39]. The tanh function behaves similarly but outputs values between -1 and 1, making it zero-centered and more effective for balancing weight updates during training. Despite this improvement, it still suffers from vanishing gradients. ReLU, on the other hand, outputs the input value itself if it is positive and returns zero for negative inputs. Its range extends from zero to any

positive number, and zero serves as the output for any negative value. While ReLU tends to perform better in deep networks due to its simplicity and reduced likelihood of vanishing gradients, it can encounter issues when neurons receive only negative inputs, effectively halting learning in those nodes. Figures 4.30 to 4.32 illustrate the formulas, value ranges, and graphical representations for each of these three activation functions.

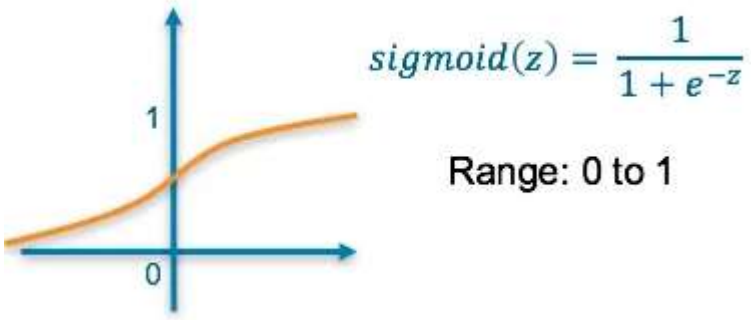

Figure 4.30: Sigmoid activation function [39]

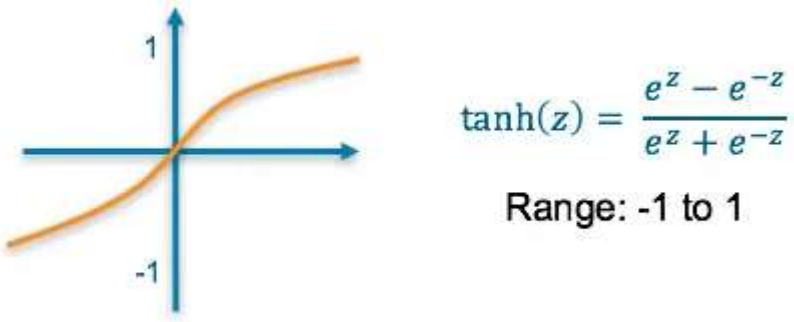

Figure 4.31: Hyperbolic tangent activation function [39]

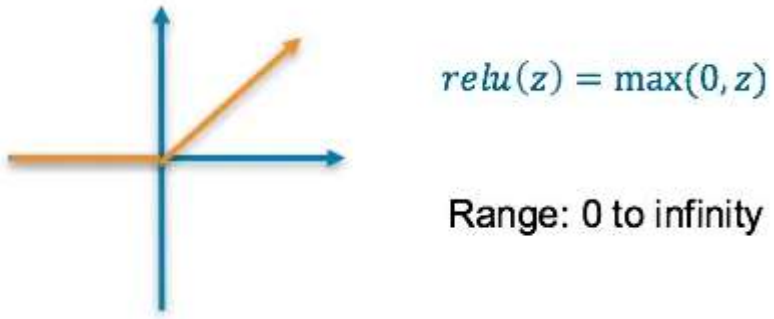

Figure 4.32: Rectified linear unit activation function [39]

A Convolutional Neural Network (CNN) is an advanced type of neural network specifically designed for image processing and pattern recognition. While CNNs share similarities with standard neural networks, they incorporate additional layers that enable them to extract spatial features from images more efficiently. Unlike traditional neural networks that treat all input features independently, CNNs process images using convolutional layers with filters (kernels) that scan the image in a process known as a convolution operation [40]. Kernels allow CNNs to detect edges, textures, and patterns at different levels within the model's architecture. Initially, a CNN applies a simple filter to detect basic features, like edges for example, but as the image passes through subsequent convolutional layers, more complex patterns emerge. By stacking multiple filters together, CNNs can recognize extra features like shapes and even internal objects. In addition to feature extraction, CNNs also use image resizing techniques, such as pooling layers, which help reduce dimensionality and improve computational efficiency. Figure 4.33 shows a basic diagram of a CNN model.

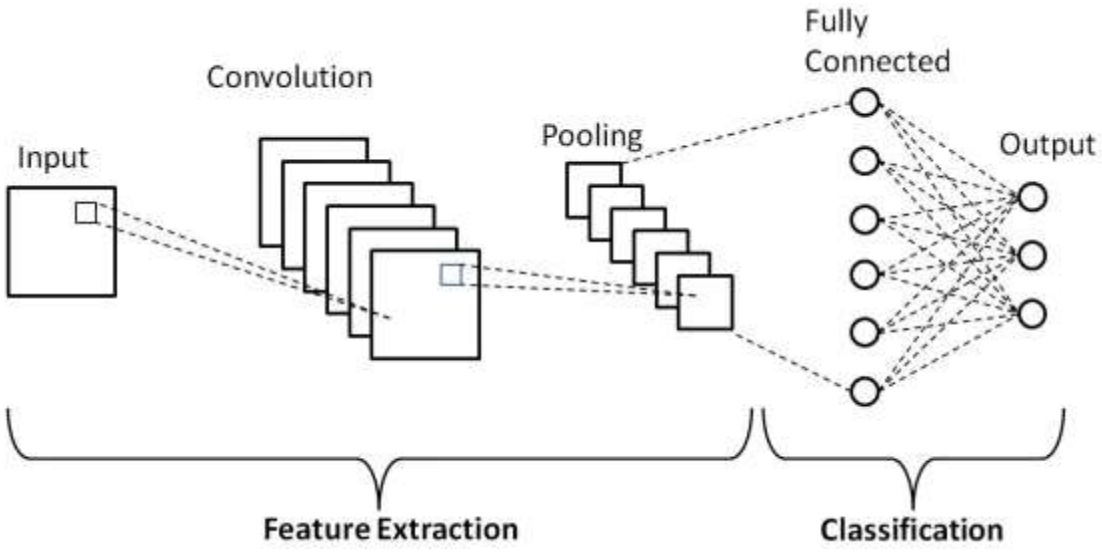

Figure 4.33: Basic CNN model structure

As illustrated in Figure 4.33, what distinguishes CNNs from traditional neural networks is the inclusion of three specialized components: the convolutional layer, pooling layer, and fully connected layer. The convolutional layer serves as the primary engine of feature extraction within the CNN. In this layer, small filters—often referred to as kernels—systematically move across the input image, calculating dot products between filter weights and pixel values. This scanning process produces feature maps that capture meaningful visual patterns, such as edges, textures, and basic shapes [41, 42, 43]. Figures 4.34 and 4.35 illustrate examples of these filtering techniques. As the data passes through successive convolutional layers, the model becomes

capable of identifying increasingly complex features and, ultimately, recognizing whole objects [42].

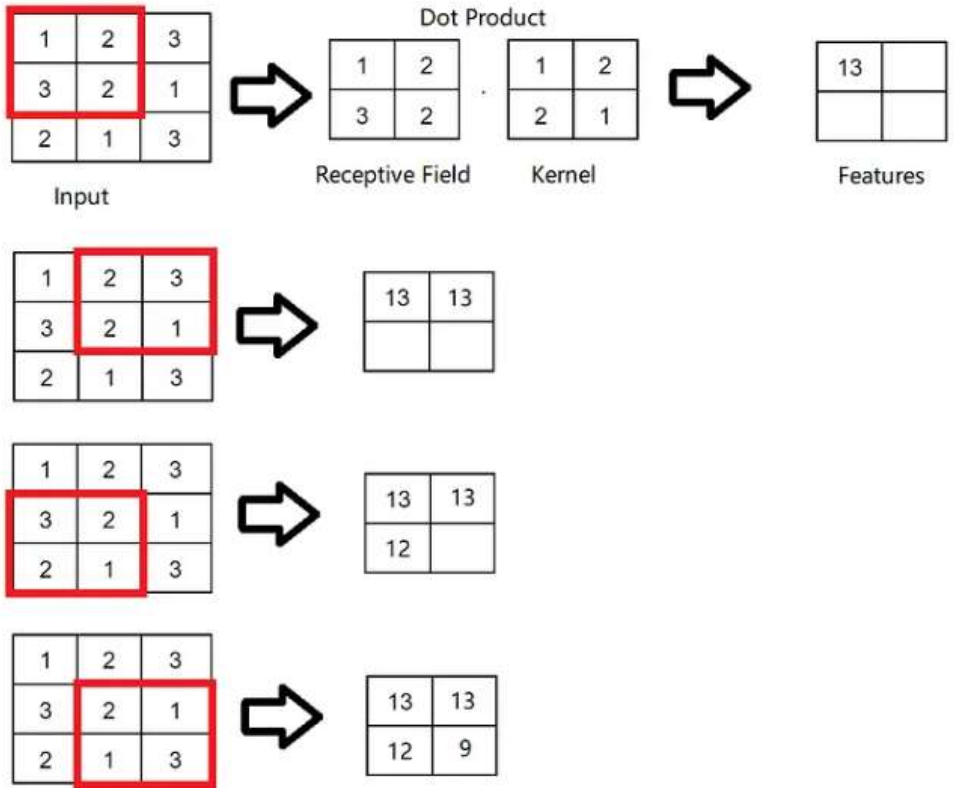

Figure 4.34: Basic kernel mapping [46]

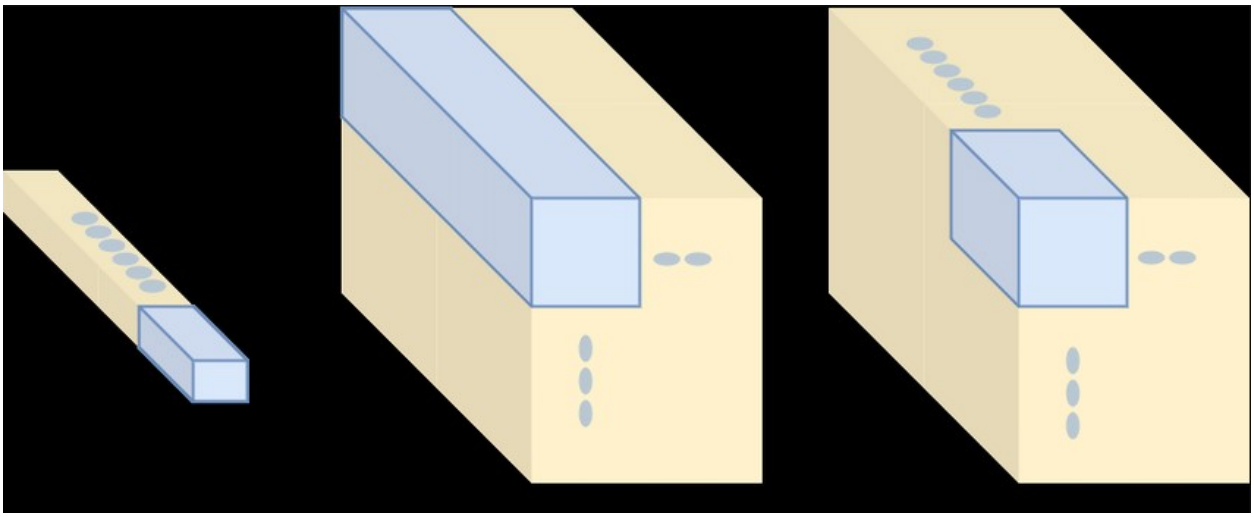

Figure 4.35: 1D, 2D, and 3D CNN kernel structure comparison [48]

As the image is processed through multiple convolutional layers, the model will detect more complex features and, in some cases, recognize entire objects [42]. Following the convolutional layer, the pooling layer is applied to reduce the spatial dimensions of the feature maps while retaining the most essential information. This process is also known as down sampling and the goal is to improve computational efficiency and reduce overfitting by forcing the model to focus on very specific and prominent features [42].

One of the most widely used methods for pooling is known as max pooling, which identifies and retains the largest value within a specified region of the feature map. Another frequently applied approach is average pooling, which calculates the mean of values within the same region [41, 42, 43, 49]. Figure 4.36 provides a side-by-side illustration of how these two pooling methods operate. Once the network completes feature extraction and reduces dimensionality through pooling, it moves to the final stage—the fully connected layer—where the data is flattened and prepared for classification. In this layer, every neuron is connected to all neurons in the previous layer, allowing the network to evaluate all extracted features simultaneously when generating predictions. While fully connected layers are essential for final decision-making, they also pose a risk of overfitting,

especially when the model contains a high number of parameters, making computations dense and complex [36, 42].

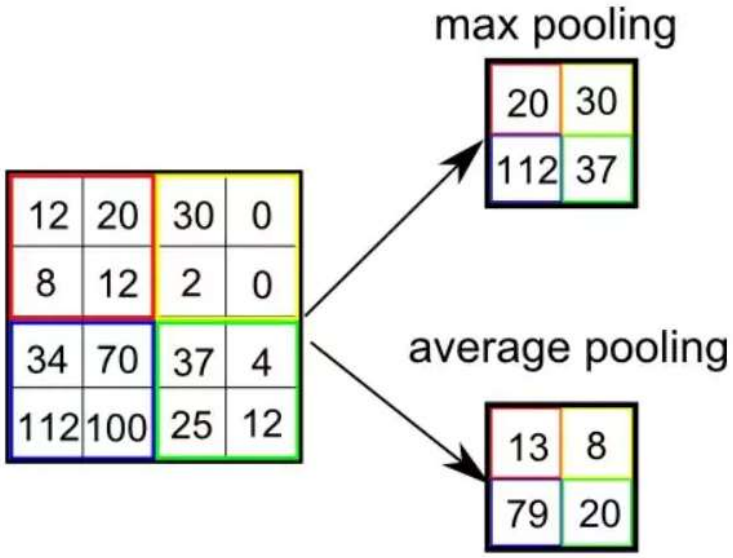

Figure 4.36: Max pooling vs. average pooling [49]

CNNs come in three different configurations, which are one-dimensional (1D), two-dimensional (2D), and three-dimensional (3D) models. These configurations are based on the types of input data. 1D CNNs are typically used for sequential data like speech recognition and natural language processing, where the input data is structured as a one-dimensional array. 2D CNNs are found in image processing tasks like object and facial detection as well as image classification. Lastly, there are 3D CNNs that further extend 2D CNNs and allow these particular models to capture depth. 3D CNNs can be found in medical imaging and hyperspectral imaging [35, 47]. Figure 4.37 visually compares a 1D, 2D, and 3D CNN.

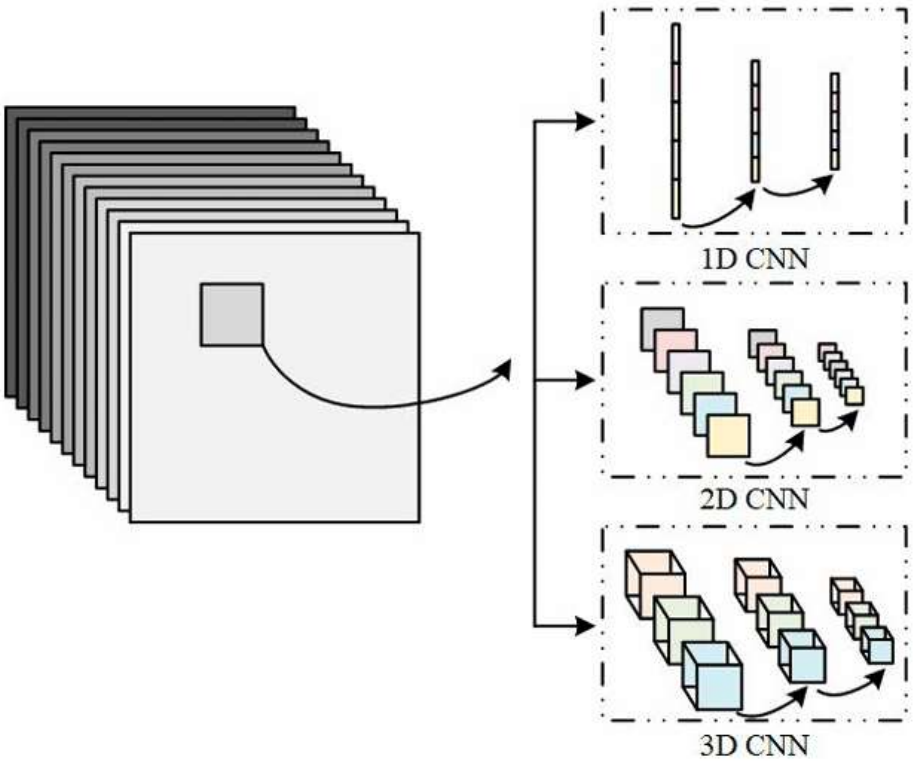

Figure 4.37: 1D, 2D, and 3D CNN structure comparison [47]

4.3 Project Implementation

4.3.1 Cubert Ultris S5 Hyperspectral Camera

The Cubert Ultris S5 hyperspectral camera was selected for this project due to its affordability, compact size, and high-performance capabilities. Its small form factor made it an ideal fit for the Astro Cultivators project, where space efficiency was a critical design factor [20]. The S5 camera offers real-time hyperspectral imaging, capturing wavelengths from 450 nm to 850 nm with 51 spectral bands, allowing for detailed spectral analysis of the sweet potato leaves. Figure 4.38 shows the physical model of the camera.

Figure 4.38: Cubert Ultris S5 hyperspectral camera

The S5 camera comes with its own graphical user interface (GUI) which allows for quick and efficient image acquisition and visualization, streamlining the data collection process for this project. The camera's ability to capture full-frame spectral images without requiring scanning made it well-suited for high-throughput data collection, further enhancing its applicability to the project. Figures 4.39 and 4.40 provide an overview of the camera's specifications.

Data Quality	
wavelength range	450 nm – 850 nm
typ. spectral resolution (FWHM)	8 nm
spectral sampling (read-out)[a]	8 nm
spectral channels (read-out)[a]	51
physical channels	42
spectral smile (wavelength error)	≤ 8 nm
spatial resolution (sampling)	290 px · 275 px
physical pixel size	3.4 µm
typ. spatial resolution (modulation width @ 10% MTF)	5 µm
spatial channel overlay error	≤ 2px
spectral data (sampled)	79 750 spectra / cube
total data points	4 067 250 data points / cube
Sensor Data	
detector	CMOS (Sony IMX264)
readout	global shutter
cooling	passive air-cooled
digitalization	12 bit
integration time	100 µs – 60 s
measurement frequency, max [b]	15 Hz
Data size (single measurement, unprocessed)	≈ 8 MiB
Data size (single measurement, processed)	≈ 15 MiB
Power-over-Ethernet[c]	
PoE Voltage	36 V – 57 V
PoE TDP	≈ 3.1 W @ 48 V

Figure 4.39: Cubert Ultris S5 datasheet part 1

Operating Data	
operating distance (default distance)	6 m – ∞
operating distance (custom distance)[a]	50 cm – ∞
typ. viewing angle (image width)	15°
Dimensions	
width	29 mm
height	29 mm
length	65 mm
weight	126 g

Figure 4.40: Cubert Ultris S5 datasheet part 2

The two figures above highlight quick facts about the S5 camera. Most notably the camera's frequencies for an image are between 450 nm and 850 nm. As mentioned earlier in this paper, that

wavelength range means that this camera will primarily capture images in the visible light spectrum and a portion of near-infrared light. Each image size will be 290 x 275 pixels and will be collected with 51 spectral bands. All images were collected using a light source and a laptop. Figures 4.41 and 4.42 show the setup for this project. Two computers were used interchangeably for this project, an Asus ROG Strix gaming laptop and a Dell Alienware desktop. Neither computer impacted the performance of the camera.

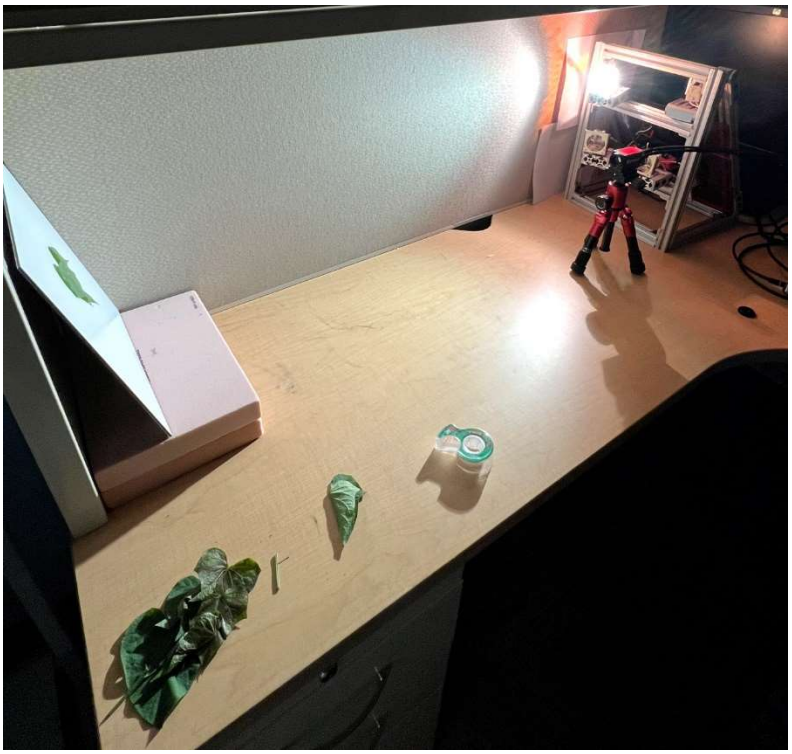

Figure 4.41: Camera setup part 1

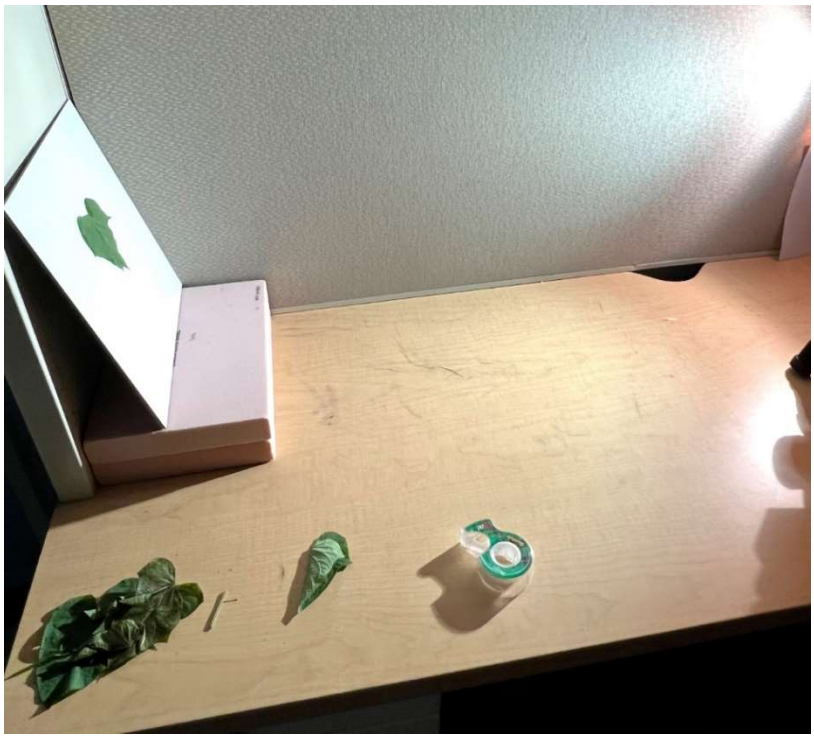

Figure 4.42: Camera setup part 2

In Figures 4.41 and 4.42 we can see the setup for the camera and how the images were captured. A single halogen light bulb was placed near the hyperspectral camera, shown in Figure 4.41. The camera itself needed to be about 900 mm from the target object. If the camera was placed closer than that, the camera software would throw an error message, and the image quality would suffer as well. This will be explained in more detail in chapter 5. Figure 4.42 also shows the placement of sweet potato leaves. Individual leaves were taped in place on a white background. The white background was angled slightly because the light source would bleed into the image and overshine the target object.

Before capturing images, the S5 camera required calibration to ensure accurate spectral data. The calibration process involved connecting the camera to a computer, powering it on, and launching the CubertTouch GUI software. Once the lens cap was removed, the halogen light source was turned on, and the camera, white backdrop, and light source were positioned correctly. Inside the

software, a white reference was collected using highly reflective material to ensure uniform reflectance across all wavelengths, allowing the camera to normalize illumination variations. Next, the black reference was taken with the lens cap on, correcting for sensor noise, thermal noise, and unwanted light variations. The final step in calibration was manually inputting the distance between the camera and the object to around 900 mm to achieve proper focus. With calibration complete, the camera was ready for image acquisition and dataset collection.

4.3.2 Python Code Implementation Introduction

The image classification model for this project was implemented using Python and PyTorch library and structured into three main sections: preprocessing, model architecture, and training. Preprocessing involved loading the dataset, extracting NDVI values from individual images, and preparing the data for training. The model architecture section defined the 3D Convolutional Neural Network (3D CNN), including its convolutional layers, hidden layers, pooling layers and output layer. Finally, the training section implemented the training loop, hyperparameters, and performance metrics, ensuring the model could learn effectively and be fine-tuned for optimal classification accuracy. The following sections will provide a detailed breakdown of each component, outlining the step-by-step implementation of hyperspectral image classification.

This entire code was run inside the Technology Infrastructure for Data Exploration (TIDE) from San Diego State University. This allowed me to run this model on higher-end hardware such as a Nvidia A100 GPU and set a custom amount of RAM space and CPU cores. Inside TIDE was its own Juypter notebook system that allowed me to create my own Python environment and import all the necessary libraries.

4.3.3 Model Preprocessing

```
import torch
import spectral
import numpy as np
import os
import matplotlib.pyplot as plt
from torch.utils.data import Dataset, DataLoader, random_split
import torch.nn as nn
import torch.optim as optim
import warnings
import logging
import torch.nn.functional as F
import random
import seaborn as sns
from sklearn.metrics import confusion_matrix
import torchvision.transforms as transforms
import time
from collections import Counter
import torchvision.transforms as transforms
import torch
from torch.utils.data import Dataset
from sklearn.model_selection import KFold
from torch.optim.lr_scheduler import ReduceLROnPlateau
```

Figure 4.43: Python library imports

Figure 4.43 above shows all the libraries that the program needs for the model to work properly. Most of the libraries come directly from PyTorch however a few additional models from scikit-learn library and others packages like “os”, “random” and “time” come directly from the Python standard library. The “spectral” library is used to handle the spectral data, specifically the ENVI file format for all spectral data.

```
print(torch.cuda.is_available())
print(torch.cuda.device_count())
print(torch.cuda.get_device_name(0))
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

dirpath = '/home/troy/hsi_dataset'

print(dirpath)
```

Figure 4.44: Checking for a Cuda core inside GPU

The code snippet in Figure 4.44, checks whether there is a “Cuda” core(s) available inside the GPU’s architecture of the host computer. Verifying if there are available Cuda cores, the device variable will move the model’s program to run directly on the GPU and have the entire program run faster. Otherwise having the model’s program run on the CPU will cause the program to run significantly slower. This does not change the model’s results but rather changes the computer’s performance. And lastly, I defined a variable that will hold the path location for the entire dataset. That path variable will be used later in the program.

```
def split_files(dirpath):
    # Get a list of all files in the folder
    files = os.listdir(dirpath)

    # Sort files numerically begining at 1
    files = sorted(files, key=lambda x: int(''.join(filter(str.isdigit, x)) or -1))

    # Filter out .hdr files, data files, and .png files into their own list
    hdr_files = [f for f in files if f.endswith('.hdr')]
    data_files = [f for f in files if not f.endswith('.hdr') and not f.endswith('.png')]
    mask_files = [f for f in files if f.endswith('.png')]

    # Return Each file type as a list
    return hdr_files, data_files, mask_files
hdr_files, data_files, mask_files = split_files(dirpath)
```

Figure 4.45: Splitting files by extension type

In the figure above, the dataset folder path is passed into the split_files() function, marking the beginning of the preprocessing stage for my model. This function’s purpose is to scan the dataset folder and categorize files based on their file extensions. The model works with three specific file types: .hdr files (header files for the .ENVI file), data files (which have no extension and second portion of the .ENVI file), and masked images (.png files). The dataset folder has been pre-cleaned to ensure that only these three file types are present in the folder. The dataset folder contains 1717 images sequentially and each image has three files. The function sorts the files numerically and

then filters them into three separate lists: hdr_files, data_files, and mask_files. Finally, it returns three lists that each contain a specific file type.

```
def group_files(hdr_files, data_files, mask_files):
    #hdr_files, data_files, mask_files = split_files(dirpath)
    paired_files = []
    for hdr_file in hdr_files:
        base_name = hdr_file[:-4]
        data_file = next((f for f in data_files if f.startswith(base_name)), None)
        mask_file = next((f for f in mask_files if f.startswith(base_name)), None)
        if data_file and mask_file:
            paired_files.append((os.path.join(dirpath, hdr_file),
                                 os.path.join(dirpath, data_file),
                                 os.path.join(dirpath, mask_file)))
    return paired_files
#Display each spectral image with its 3 files
paired_files = group_files(hdr_files, data_files, mask_files)
```

Figure 4.46: Regrouping images based on file type

Figure 4.46 above shows a function for regrouping all three file types into a single data point. The group_files() function ensures that each image is correctly paired with its corresponding files. For example, an image named sweet_potato_300.hdr, has two associated files which are sweet_potato_300 (the data file) and sweet_potato_300_gt.png (the masked image). The "_gt" suffix in the masked image filename stands for 'Ground Truth,' which is the common naming convention used throughout the dataset for masked images. When this function is executed, it takes the three lists generated earlier—hdr_files, data_files, and mask_files—and iterates through them, matching files based on their shared base names. It then combines the corresponding header file (.hdr), data, and masked image (.png) files into tuples and stores them in the list called "paired_files". This grouping ensures that the three files for each image are treated as a single data point in the later stages of the program.

```python
def load_and_resize_data(hdr_file, data_file, mask_file):
    # Load hyperspectral data
    img = spectral.open_image(hdr_file)
    data = img.load()

    # band indices start from 0 when view the .HDR file
    # Band 23 = 634 nm
    # Band 25 = 650 nm
    # Band 45 = 810 nm
    # Band 50 = 850 nm
    selected_bands = data[:, :, [23, 45]]

    # Resize the selected bands
    resized_data = torch.nn.functional.interpolate(
        torch.tensor(selected_bands).permute(2, 0, 1).unsqueeze(0),  # Convert to tensor and prepare for resizing
        size=(224, 224),
        mode='bilinear',
        align_corners=False
    ).squeeze(0).permute(1, 2, 0)  # Convert back to original shape

    # Load and resize the mask
    mask = plt.imread(mask_file)  # Assuming mask is a .png image
    mask_tensor = torch.tensor(mask, dtype=torch.float32).unsqueeze(0)
    resized_mask = torch.nn.functional.interpolate(
        mask_tensor.unsqueeze(0),
        size=(224, 224),
        mode='bilinear',
        align_corners=False
    ).squeeze(0)
    return resized_data, resized_mask
resized_data, resized_mask = load_and_resize_data(paired_files[0][0], paired_files[0][1], paired_files[0][2])
```

Figure 4.47: Slicing and resizing data and masked files

Figure 4.47 is a function that takes in the three image lists previously created in Figure 4.45 to resize all images from 290 x 275 pixels down to 224 x 224 pixels. It also selects the two specific bands needed for the NDVI formula. Originally, all spectral images captured with the S5 hyperspectral camera contained 51 bands. Using all 51 bands causes each image to take 15.5 MB (megabytes) of data, and with over 1,700 images, this results in the dataset using approximately 13 GB (gigabytes) of storage. This also poses performance issues, which will be discussed in the next chapter. Since the NDVI formula only requires two specific bands, this function extracts bands 23 and 45, corresponding to 634 nm (red region) and 810 nm (near-infrared region).

After selecting the two relevant bands, the function in Figure 4.47 resizes the data images and the masked images to the new resolution. The resizing process is handled by PyTorch's "interpolate"

function with bilinear interpolation. First, the selected bands are converted into a PyTorch tensor and rearranged to match the specific format for interpolation. The tensor is reshaped to [C, H, W] where C represents the channels (bands), H represents height and W represents width. An extra dimension is added using unsqueeze(0) to help with the computation. The interpolate function performs bilinear interpolation to resize the image, and the tensor is then reshaped back to its original [H, W, C] format for consistency.

Similarly, the binary mask image (.png file) is loaded and converted into a PyTorch tensor with a single channel. It is then resized using the same bilinear interpolation method, ensuring that it matches the dimensions of the resized spectral image. The function returns both the resized hyperspectral image and its corresponding resized mask, maintaining the spatial alignment between the two image formats. Figure 4.48 below shows the wavelength ranges for NDVI and Figure 4.49 displays the .hdr file that lists the available bands and their associated wavelengths.

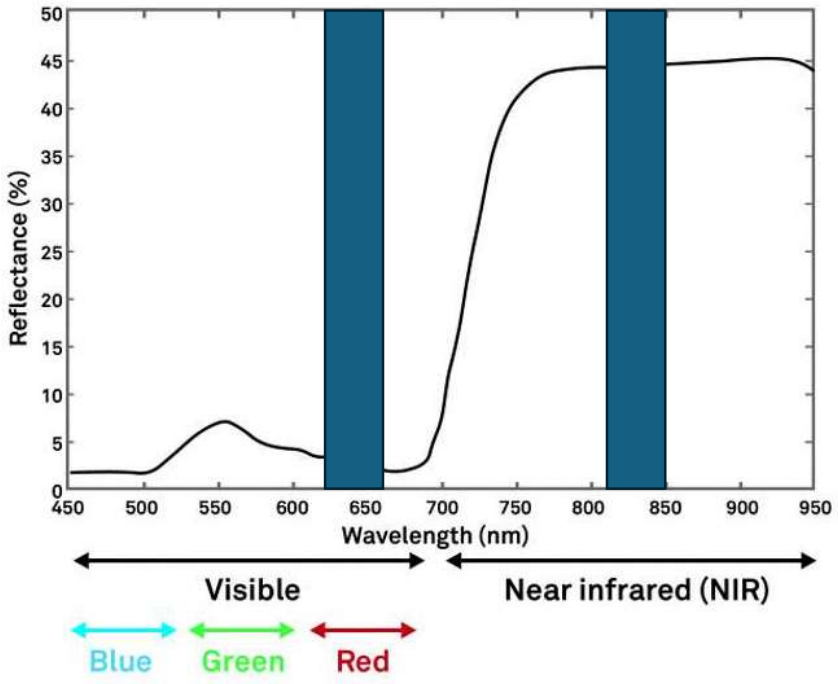

Figure 4.48: NDVI wavelength ranges [50]

Typically, the NDVI values for red are between 620 and 700 nm, and for near-infrared (NIR) are around 800 to 850 nm. The most common spectral values chosen for NDVI are shown in blue bars in the same figure. Figure 4.49 below shows the .hdr file that is generated for each individual hyperspectral sweet potato image.

```
ENVI
description = {
 Date: 2024-07-02,
 Time: 18:49:20.695UTC,
 Serial: BS5224601,
 Comment: exp. 100 ms (main) }

samples =290
lines   =275
bands   =51
header offset = 0
file type = ENVI Standard
data type = 12
interleave = BIP
sensor type = Cubert Ultris S5
z plot average = {1,1}
default stretch  = square root
z plot titles = {wavelength [nm]!N, reflectance [%*100]}
reflectance scale factor = 10000
byte order = 0
bbl= {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,}
Wavelength = {450,458,466,474,482,490,498,506,514,522,530,538,546,554,562,570,578,586,594,602,610,618,626,634,642,650,658,666,674,682,690,698,706,714,722,730,738,746,754,762,770,778,786,794,802,810,818,826,834,842,850}
band names =
{450nm,458nm,466nm,474nm,482nm,490nm,498nm,506nm,514nm,522nm,530nm,538nm,546nm,554nm,562nm,570nm,578nm,586nm,594nm,602nm,610nm,618nm,626nm,634nm,642nm,650nm,658nm,666nm,674nm,682nm,690nm,698nm,706nm,714nm,722nm,730nm,738nm,746nm,75
4nm,762nm,770nm,778nm,786nm,794nm,802nm,810nm,818nm,826nm,834nm,842nm,850nm}
wavelength units = nm
```

Figure 4.49: A typical .ENVI header file

Figure 4.49 shows a standard header file (.hdr) that is created with every data file of the spectral image. It is very similar to a JSON file or XML file where it shows basic information about a piece of data. In this case, it shows information like the date the image was taken, the number of samples and lines corresponding to the size of the image. It also shows how many bands the image has originally. At the very end, it shows the band names and which specific frequency they correspond to. The bands are saved as an array, and the array starts at index 0. Band 0 corresponds to the frequency of 450 nm. Band 50 (the 51st band) corresponds to the frequency of 850 nm.

```python
def find_roi_pixels(mask):
    # Ensure mask is a PyTorch tensor
    if not isinstance(mask, torch.Tensor):
        mask = torch.tensor(mask)

    # Remove any singleton dimensions (e.g., [1, H, W] -> [H, W])
    mask = mask.squeeze()

    # Convert mask to binary (white pixels as 1, everything else as 0)
    binary_mask = (mask > 0.5).float()

    # Find coordinates of white pixels
    roi_coords = torch.nonzero(binary_mask, as_tuple=False)

    # Ensure roi_coords has exactly 2 columns (row, col) by checking the mask's shape
    if roi_coords.shape[1] > 2:
        roi_coords = roi_coords[:, :2]  # Take only the row and col indices

    return roi_coords.numpy()  # Convert to numpy if needed


def extract_roi_spectral_data(hyperspectral_data, roi_coords):
    roi_spectral_data = hyperspectral_data[roi_coords[:, 0], roi_coords[:, 1], :]
    return roi_spectral_data
# Find ROI coordinates
roi_coords = find_roi_pixels(resized_mask.numpy())
roi_spectral_data = extract_roi_spectral_data(resized_data, roi_coords)
```

Figure 4.50: Region of interest functions

To help the model focus on the essential portion of each image, Figure 4.50 presents two functions that utilize the mask images to identify the relevant pixel coordinates within each data file. The first function, find_roi_pixels(), processes the binary mask image to locate the pixels corresponding to the leaf region, filtering out the background. Since the mask images are black and white, with the leaf represented in white and the background in black, extracting only the white pixel coordinates ensures that subsequent NDVI calculations focus exclusively on the plant and not on irrelevant areas. The second function, extract_roi_spectral_data(), uses these identified pixel coordinates to extract the corresponding spectral information from the hyperspectral image, ensuring that only meaningful data is used for further processing.

The find_roi_pixels() function first ensures that the mask is a PyTorch tensor. If the mask is not already in tensor format, it converts it using torch.tensor(). It then removes any singleton dimensions using squeeze(), ensuring that the mask is in a two-dimensional format [H, W]. The function then normalizes the mask by setting pixel values above 0.5 to 1 (white) and everything else to 0 (black), creating a binary representation of the mask. The white pixels are then identified using torch.nonzero(), which returns the row and column coordinates of all nonzero (white) pixels. Since torch.nonzero() can sometimes return additional dimensions, the function ensures that only the first two columns (row and column indices) are used. Finally, the coordinates are converted to a NumPy (Number Py) array for easier processing. The second function extract_roi_spectral_data(), takes these ROI coordinates and uses them to extract the spectral data from the corresponding locations in the hyperspectral image. By indexing the hyperspectral data using roi_coords[:, 0] for row indices and roi_coords[:, 1] for column indices, it retrieves all available spectral bands for those specific pixels.

```
def calculate_ndvi(roi_spectral_data):
    # Assume band 0 is Red and band 1 is NIR in the resized_data
    red_band = roi_spectral_data[:, 0]
    nir_band = roi_spectral_data[:, 1]

    # NDVI Forumla
    ndvi = (nir_band - red_band) / (nir_band + red_band + 1e-8)  # Adding epsilon to avoid division by zero
    return ndvi
```

Figure 4.51: NDVI formula calculation

Figure 4.51 illustrates how the NDVI formula is implemented. The function calculate_ndvi()'s input is the spectral data extracted from the region of interest functions and isolates the red and near-infrared bands. The bands are indexed from the region of interest function, where the first column represents the red band, and the second column represents the NIR band. The NDVI formula is then applied and calculates the difference in reflectance between the NIR and red

wavelengths. To prevent division by zero, a small epsilon value (1e-8) is added to the denominator. The function returns the NDVI values, which will be used later in the model.

```
def get_label_name(label):
    labels = [
        "Healthy",
        "Moderately Healthy",
        "Early Necrosis",
        "Moderate Necrosis",
        "Severe Necrosis",
        "Dead or Inanimate Object"
    ]
    return labels[label]


def assign_label_from_ndvi(ndvi_value):
    if ndvi_value > 0.6:
        return 0  # Healthy
    elif 0.5 <= ndvi_value < 0.59:
        return 1  # Moderately Healthy
    elif 0.4 <= ndvi_value < 0.49:
        return 2  # Early Necrosis
    elif 0.3 <= ndvi_value < 0.39:
        return 3  # Moderate Necrosis
    elif 0.05 <= ndvi_value < 0.29:
        return 4  # Severe Necrosis
    else:
        return 5  # Dead or Inanimate Object
```

Figure 4.52: Output labels

Figure 4.52 above categorizes plant health based on NDVI values and assigns corresponding labels. The function assign_label_from_ndvi() takes an NDVI value as input and then assigns a label based on predefined the ranges shown. Higher NDVI values correspond to healthier plants. Lower NDVI values indicate increasing levels of plant necrosis. The labels range from 0 (Healthy) to 5 (Dead or Inanimate Object). The get_label_name() function then maps these labels to a predefined list.

4.3.4 Model Architecture

This project utilizes a 3D Convolutional Neural Network (3D CNN) because all hyperspectral images in the dataset are three-dimensional. The three-dimensional data includes both spatial information and spectral information. As previously mentioned, a 2D CNN is commonly used for image classification and object detection when processing 2D images like RGB data. I also mentioned in the literature review that other authors have used 2D CNNs for hyperspectral data, but some of them were either using strict 2D CNNs or 1D and 2D CNN hybrid approaches. A 2D CNN will be able to analyze spatial information like edges, shapes, and textures but it will not be that effective at capturing spectral information. Using a 3D CNN extends the convolution into the third dimension, allowing it to capture the spectral information. The following figures below in this section will break down the core components of the 3D CNN. I will illustrate how convolutional layers, pooling layers, and fully connected layers are structured and implemented in PyTorch. Figures 4.53 to 4.55 show a custom class designed for processing the dataset.

```
class SweetPotatoDataset(Dataset):
    def __init__(self, data_list, augment_minority=False, augment_factor=3):
        self.data_list = data_list
        self.augment_minority = augment_minority
        self.augment_factor = augment_factor

        # augmentation transformations
        self.augmentations = transforms.Compose([
            transforms.RandomHorizontalFlip(),
            transforms.RandomRotation(20),
            transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1)
        ])

        # Augment the minority classes if specified
        if self.augment_minority:
            self.data_list = self._augment_minority_classes(self.data_list)

    def __len__(self):
        return len(self.data_list)
```

Figure 4.53: Sweet Potato dataset Class part 1

In Figure 4.53, is the first snippet of the SweetPotatoDataset class. It is structured to handle the image loading, preprocessing, basic data augmentation, and label assignments. The variable data_list contains the tuples of each image. As mentioned earlier, each tuple contains a header file, a data file, and a .png file to represent a single image. There is some data augmentation applied for underrepresented class labels. This data augmentation applies a set of basic transformations like color jitter, rotations, and flips. Some performance issues within the dataset were encountered during testing and this will further be discussed in the next chapter. The small function at the bottom of the figure simply returns the total number of samples in the dataset.

```
def __getitem__(self, idx):
    data, mask = self.data_list[idx]

    # Process hyperspectral data
    data = data.permute(2, 0, 1).unsqueeze(1)  # [2, 1, 224, 224] (depth=1)

    # Process mask
    mask = mask.squeeze(0).unsqueeze(0).unsqueeze(1)  # [1, 1, 224, 224] (depth=1)

    # Concatenate data and mask along the channel dimension
    combined_tensor = torch.cat((data, mask), dim=0)  # [3, 1, 224, 224] (channels=3, depth=1)

    # Calculate NDVI-based label
    roi_coords = find_roi_pixels(mask.squeeze().numpy())
    roi_spectral_data = extract_roi_spectral_data(data.squeeze().permute(1, 2, 0).numpy(), roi_coords)
    ndvi_values = calculate_ndvi(roi_spectral_data)

    # Assign label based on NDVI
    mean_ndvi = ndvi_values.mean().item()
    label = assign_label_from_ndvi(mean_ndvi)  # Assign one of the 6 labels

    if label is None:  # Handle cases where the label is invalid
        raise ValueError(f"Invalid NDVI value {mean_ndvi} for index {idx}")

    label = torch.tensor(label, dtype=torch.long)

    # Apply augmentations if augment_minority is True and the label is a minority class
    if self.augment_minority and label in [3, 5]:
        combined_tensor = self.augmentations(combined_tensor)

    return combined_tensor, label
```

Figure 4.54: Sweet Potato dataset class part 2

Figure 4.54 is a continuation of the SweetPotatoDataset class. Focusing on the __getitem__() function that retrieves both the data files and the masked files from the dataset. It calls all the

functions that were previously defined in the preprocessing section. It first converts both the data and masked files into a PyTorch tensor. There are two formats for the tensor since the masked images and data files are slightly different. For the data files, the format is [2, 1, 224, 224] and [3, 1, 224, 224] for the masked images. The “2’ in the beginning represents the number of spectral bands used by the NDVI calculation in the preprocessing section and those bands are the red and NIR values. The “3” for the masked images is for the red band, nir band, and the binary mask. The two “224” are the dimensions o” the’Image. The “1” is the depth dimension in the tensor format. This follows the 3D CNN input format of [Channels, Depth, Height, Width].

```
def augment_minority_classes(self, data_list):
    # Count the occurrences of each label to determine minority classes
    label_counts = Counter()
    augmented_data_list = []

    # Calculate labels for all data in the list to count class occurrences
    for data, mask in data_list:
        combined_tensor, label = self._get_combined_tensor_and_label(data, mask)
        label_counts[label.item()] += 1

    # Identify minority classes based on label counts
    minority_classes = [label for label, count in label_counts.items() if count < max(label_counts.values())]

    # Augment minority classes
    for data, mask in data_list:
        combined_tensor, label = self._get_combined_tensor_and_label(data, mask)
        if label.item() in minority_classes:
            # Append the original and multiple augmented versions
            augmented_data_list.append((data, mask))
            for _ in range(self.augment_factor):
                augmented_data = self.augmentations(combined_tensor)
                augmented_data_list.append((augmented_data[:2], augmented_data[-1].unsqueeze(0)))
        else:
            # Append the original sample for majority classes
            augmented_data_list.append((data, mask))

    return augmented_data_list

def get_combined_tensor_and_label(self, data, mask):
    # Process data and mask to calculate label (similar to __getitem__)
    data = data.permute(2, 0, 1).unsqueeze(1)
    mask = mask.squeeze(0).unsqueeze(0).unsqueeze(1)
    combined_tensor = torch.cat((data, mask), dim=0)
    roi_coords = find_roi_pixels(mask.squeeze().numpy())
    roi_spectral_data = extract_roi_spectral_data(data.squeeze().permute(1, 2, 0).numpy(), roi_coords)
    ndvi_values = calculate_ndvi(roi_spectral_data)
    mean_ndvi = ndvi_values.mean().item()
    label = assign_label_from_ndvi(mean_ndvi)
    return combined_tensor, torch.tensor(label, dtype=torch.long)
```

Figure 4.55: Sweet Potato dataset class part 3

At the end of the SweetPotatoDataset class from Figure 4.55, there are two functions for the class augmentation. The augment_minority_classes() function will count the number of occurrences of each label in the dataset and identify which class has the lowest sample count. It will then generate additional augmentations using those transformations defined in Figure 4.55. The last function is get_combined_tensor_and_label() which processes the data and mask images and combines both into a single tensor.

```python
class model_3DCNN(nn.Module):
    def __init__(self):
        super(model_3DCNN, self).__init__()

        self.conv1 = nn.Conv3d(in_channels=3, out_channels=64, kernel_size=(1, 3, 3), padding=(0, 1, 1))
        self.pool = nn.MaxPool3d(kernel_size=(1, 2, 2), stride=(1, 2, 2))
        self.conv2 = nn.Conv3d(in_channels=64, out_channels=128, kernel_size=(1, 3, 3), padding=(0, 1, 1))

        # Dropout layer with a probability of 0.5 is default
        self.dropout = nn.Dropout(p=0.4)

        #  Hidden Layers
        self.fc1 = nn.Linear(128 * 1 * 56 * 56, 256)
        self.fc2 = nn.Linear(256, 256) # Hidden Layer 1
        self.fc3 = nn.Linear(256, 256)
        self.fc4 = nn.Linear(256, 256)
        self.fc5 = nn.Linear(256, 256)
        self.fc6 = nn.Linear(256, 256)
        self.fc7 = nn.Linear(256, 256)
        self.fc8 = nn.Linear(256, 6)  # Output layer (6 classes)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(x)
        x = F.relu(self.conv2(x))
        x = self.pool(x)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = F.relu(self.fc2(x))
        x = self.dropout(x)
        x = F.relu(self.fc3(x))
        x = self.dropout(x)
        x = F.relu(self.fc4(x))
        x = self.dropout(x)
        x = F.relu(self.fc5(x))
        x = self.dropout(x)
        x = F.relu(self.fc6(x))
        x = self.dropout(x)
        x = F.relu(self.fc7(x))
        x = self.dropout(x)
        x = self.fc8(x)
        return x
```

Figure 4.56: 3D CNN model structure

Figure 4.56 above showcases the model_3DCNN class and defines the 3D Convolutional Neural Network (3D CNN). This model processes input tensors with three channels (two spectral bands and a binary mask) using multiple convolutional, pooling, and fully connected layers. The convolutional layers extract spatial and spectral features, while max-pooling layers reduce the feature map size to enhance computational efficiency. The final output layer produces six class scores, corresponding to different plant health conditions based on NDVI values.

Additionally, the 3D CNN has two convolutional layers. They are used to extract features from the hyperspectral data. They apply a convolution operation across three dimensions: depth, width,

and height. In conv1, the input has the three dimensions and then outputs 64 feature maps. Conv2 then takes these 64 feature maps as input and produces 128 feature maps. Each convolutional layer extracts features from the input, such as edges, textures, and spectral patterns. The kernel size (1, 3, 3) means the convolution filter is 1 pixel deep (since our depth dimension is 1) and slides over 3 × 3 spatial regions in height and width. The padding (0, 1, 1) ensures that the output dimensions are preserved in height and width.

The pooling layer Is a 3D max-pooling that reduces the spatial dimensions of the feature maps to make them more computationally efficient. The kernel size of (1,2,2) means the pooling widening is 1 pixel deep and covers 2 x 2 regions in height and width. The stride controls how the pooling window moves. The 1 means the depth dimension remains unchanged and the two 2s mean it moves at 2 pixels at a time.

The dropout layer is a technique used to prevent overfitting. The variable p=0.4 is the probability that a particular neuron would be dropped (set to zero). In this case, 40% of the neurons in a particular hidden layer could be randomly dropped. This forces the network to not rely on specific neurons. By default, p is set to 0.5. Other p values were tested and will be further discussed in Chapter 5.

Underneath the dropout layer in Figure 4.56 is where the fully connected layers are defined. These fully connected layers in particular are also called the hidden layers. The fc1 is the input layer and takes in feature maps from the two convolution layers. Fc2 to fc7 are the six hidden layers that find the extracted features. Each layer has 256 neurons and the number of neurons per layer was tested extensively. The conclusion for setting the neuron count to 256 will be further discussed in Chapter 5. The last layer, fc8, is the output layer which maps the 256 neurons from fc7 to the 6 output classes.

The last section of Figure 4.56 is the forwarding function. The forward() method, also known as a forwarding function, defines how input data flows through the network, applying convolutions, activations, pooling, and fully connected layers to produce a final classification output. The forwarding function first applies the convolution layer 1 (conv1) followed by the ReLU activation function. Then the pool layer downsamples the feature maps and conv2 is applied with the same activation function followed by another pool layer. The view() function applies a feature map to reshape them into a 1D vector that will be used by subsequent fully connected layers. The flattened vector passes through each fully connected layer with a ReLU activation function and the dropout being applied. At the"end,'fc8 will output one of the 6 classes for a particular Image.

```
resized_data_list = []
resized_mask_list = []

# Process each file pair to create resized data and mask tensors
for hdr_file, data_file, mask_file in paired_files:
    resized_data, resized_mask = load_and_resize_data(hdr_file, data_file, mask_file)
    resized_data_list.append(resized_data)
    resized_mask_list.append(resized_mask)
paired_tensors = []
for i in range(len(resized_data_list)):
    paired_tensors.append((resized_data_list[i], resized_mask_list[i]))
```

Figure 4.57: Data loading helper loops

Figure 4.57 shows a standalone section of code in between the 3D CNN model class and the training loop that helps the data loading process. Two empty lists are defined to store the resized image data files and the masked files respectively. These lists will later be used to structure the dataset in the resized format shown in the previous figures. The first for loop will iterate through all the image tuples, resize the mask and data files as well as extracting the two spectral bands. This information is then passed into another empty list called paired_tensors that stores the resized

images. After this section of the code is run, the resized images are loaded into the model for training.

4.3.5 Model Training

The upcoming figures below will highlight the implementation of the model's training loop, detailing how the model learns from the dataset. This includes the initialization of training weights, the selection of hyperparameters, and the performance optimization techniques used to enhance the model's accuracy. Key components such as loss calculations, backpropagation, and evaluation metrics will be covered. The final results of this project, the model's performance and testing outcomes, will be covered in Chapter 5.

```
# Number of folds for Cross Validation
k = 5
kf = KFold(n_splits=k, shuffle=True, random_state=42) # Initialize KFold

# Lists to store accuracies and confusion matrices for each fold
fold_accuracies = []
confusion_matrices = []

# Variables to track hyperparameters
epoch_counts = []          # Number of epochs completed for each fold (including early stopping)
initial_weights_list = []  # Initial weights for each fold
final_weights_list = []    # Final weights for each fold
fold = 1                   # Starting fold
```

Figure 4.58: Training loop initializations

Figure 4.58 shows a portion of the training loop parameters. One of the primary performance algorithms used in this project is cross-validation. Cross-validation is a technique used in machine learning that divides the training data into multiple subsets (folds). In this case, 5 folds have been defined. One of the folds will be used as a validation set and the remaining four folds will be used for training the model. The main purpose of cross-validation is to prevent overfitting. This project

uses k-fold cross-validation. The cross-validation is repeated multiple times and allows each fold to be used for validation and training [51].

Additionally, in Figure 4.59, there are empty lists that have been defined. These lists are used to track the final results of the various hyperparameters used. For example, the epoch_counts will track the number of times the training loop runs in each fold. That number will be displayed after the training loop for that fold has finished. The data that is saved in each of these list variables will be shown in Chapter 5.

```
for train_index, test_index in kf.split(paired_tensors):
    print(f"Fold {fold}/{k}")

    # Hyperparemeters
    batch_size = 16
    patience = 5
    lr = 0.0001
    weight_decay = 5e-4
    num_epochs = 80    # 50

    weights = torch.tensor([0.2, 0.2, 0.2, 0.2, 0.05, 0.3]).to(device)


    # Split the data into training and testing sets for the current fold
    train_data = [paired_tensors[i] for i in train_index]
    test_data = [paired_tensors[i] for i in test_index]

    # Create datasets for the current fold
    train_dataset = SweetPotatoDataset(train_data)
    test_dataset = SweetPotatoDataset(test_data)

    # Create DataLoaders for training and testing
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

    # Initialize model, criterion, optimizer for this fold
    model = model_3DCNN.to(device)
    criterion = nn.CrossEntropyLoss(weight=weights)
    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=patience, verbose=True)

    best_loss = float('inf') # Early Stopping parameters
    epochs_no_improve = 0
    initial_weights_list.append(weights.cpu().numpy().tolist()) # Save initial weights for this fold
```

Figure 4.59: Training loop part 1

Figure 4.59 highlights the top part of the training loop. The training loop contains a nested for loop that will be shown in Figure 4.60. Focusing on the above figure, the outer loop runs on the k-fold cross-validation parameters. It splits the dataset into five train-test folds. For every fold, the dataset is divided into training (train_data) and testing (test_data) sets based on the indices provided by the kf.split() function. The subsets are then wrapped into SweetPotatoDataset objects, which structure the data appropriately before being loaded into the model by PyTorch's DataLoader function. The DataLoader ensures that the images are loaded into batches of 16 and then shuffled.

After preparing the training and testing datasets, the 3D CNN model is initialized and assigned to a designated computation device, such as a GPU or CPU. This project uses the CrossEntropyLoss function, which incorporates class weighting to address imbalances in the dataset—ensuring that underrepresented classes have a meaningful impact on the overall loss. The Adam optimization algorithm is employed, with its learning rate treated as a tunable hyperparameter. Through extensive experimentation, a learning rate of 0.0001 was selected as the most effective. Weight decay was also applied, set to a value of 5E-4, to help regularize the model. In addition, a learning rate scheduler (ReduceLROnPlateau) was integrated to adapt the learning rate when the validation loss stops improving, reducing it if no progress is observed for five consecutive epochs. A variable called best_loss is used to keep track of the lowest recorded validation loss during training, and an early stopping mechanism is implemented to halt training when the model begins to overfit. This strategy evaluates the validation loss at each epoch and stops training if no significant improvement is observed after five epochs, minimizing wasted computation and promoting better

generalization to new data [52]. Lastly, the model's initial weights for each cross-validation fold are stored in a list named initial_weights_list to monitor how weights evolve during training.

```
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0

    # Track time for each epoch
    epoch_start_time = time.time()

    # Training loop
    for i, (inputs, labels) in enumerate(train_loader):
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    # Calculate average training loss for the epoch
    avg_train_loss = running_loss / len(train_loader)

    # Validation step
    model.eval()
    val_loss = 0.0
```

Figure 4.60: Training loop part 2

Figure 4.60 shows a portion of the nested for loop mentioned earlier. This for loop shows how the model is updated at each step. The loop iterates over the set number of epochs and at the start of each epoch. The model is set to training mode (model.train()) enabling features like dropout and batch normalization. The variable running_loss is initialized to track the cumulative loss across batches. Additionally, the epoch start time is recorded using time.time() to measure the duration of each epoch.

An additional for loop is also nested and iterates over the training dataset processing one batch of images at a time. The image inputs and the output labels are moved into the appropriate computing device (GPU or CPU). Before the forward propagation runs, the gradients are reset to zero to prevent any runover data from previous iterations. The model then makes predictions by passing

the inputs through the model, producing class scores. The loss function (criterion) calculates the difference between the predicted outputs and actual labels, and backpropagation (loss.backward()) computes the gradients of the loss concerning the model's parameters. The optimizer (optimizer.step()) updates the model's parameters using these gradients. The training loss for the batch (loss.item()) is added to running_loss, which will later be used to compute the average training loss for the epoch.

After training on all batches in the epoch, the average training loss is calculated by dividing running_loss by the total number of batches (len(train_loader)). At this point, the model is switched to evaluation mode (model.eval()) to disable dropout and batch normalization, ensuring that validation performance is assessed without randomness. The validation loss (val_loss) is initialized to zero, preparing for the next step where the model's performance on the validation set will be evaluated.

```
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

    avg_val_loss = val_loss / len(test_loader)
    scheduler.step(avg_val_loss)

    epoch_end_time = time.time()
    epoch_duration = epoch_end_time - epoch_start_time

    print(f'Epoch [{epoch + 1}/{num_epochs}], Fold [{fold}/{k}], Train Loss: {avg_train_loss:.4f}, Val Loss: {avg_val_loss:.4f}, Time: {epoch_duration:.2f}s')

    # Early Stopping Check
    if avg_val_loss < best_loss:
        best_loss = avg_val_loss
        epochs_no_improve = 0
        best_model_state = model.state_dict()  # Save the best model
    else:
        epochs_no_improve += 1

    if epochs_no_improve == patience:
        print(f'Early stopping at epoch {epoch + 1} for Fold {fold}/{k}')
        model.load_state_dict(best_model_state)  # Load the best model state
        epoch_counts.append(epoch + 1)  # Track the number of epochs run before stopping
        break
if epochs_no_improve < patience:
    epoch_counts.append(num_epochs)
```

Figure 4.61: Training loop part 3

Figure 4.61 shows the validation stage of the training loop. The torch.no_grad() disables gradient calculations to reduce memory usage during the model evaluation. Inside this code block, a loop iterates through the test_loader, processing each batch. The model then performs forward propagation (model(inputs)), generating predicted outputs for the validation data. The loss function (criterion) calculates how far the predicted outputs deviate from the actual labels, and the resulting loss value is added to val_loss, accumulating the total validation loss across batches. After all validation batches have been processed, the average validation loss (avg_val_loss) is computed by dividing the total loss by the number of batches in test_loader. This average loss provides a quantitative measure of the model's performance on unseen data. The learning rate scheduler (scheduler.step(avg_val_loss)) then adjusts the learning rate based on the validation loss, reducing it if no improvement is observed over multiple epochs, helping to refine the model's learning process.

Following the validation step, the training loop then prints key statistics about the model's performance. These statistics include the epoch number, fold number, training loss, validation loss, and the total time per epoch. Next, early stopping is applied to prevent unnecessary training if the model stops improving. If the average validation loss is lower than the best-recorded loss, then the model updates best_loss and resets the epochs_no_improve counter, while also saving the current model state (model.state_dict()) as the best-performing model so far. However, if no improvement is observed, the epochs_no_improve counter increases. If this counter reaches the specified patience level of 5, then the training for the current fold is stopped early, and the model reverts to the best-performing state by loading best_model_state. Finally, the total number of epochs completed before early stopping is recorded in epoch_counts, ensuring that the training process is well-documented for each fold.

```
# Post-training evaluation for this fold
with torch.no_grad():
    model.eval()
    correct = 0
    total = 0
    all_labels = []
    all_predictions = []

    # Evaluate model on test data
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        all_labels.extend(labels.cpu().numpy())
        all_predictions.extend(predicted.cpu().numpy())

    # Save final weights - Selecting only the first 6 final weights from fc6 for display
    final_weights = model.fc6.weight.detach().cpu().numpy().flatten()[:6]
    final_weights_list.append(final_weights.tolist())

    # Calculate accuracy for the fold
    accuracy = 100 * correct / total
    fold_accuracies.append(accuracy)
    print(f'Fold {fold}/{k} Test Accuracy: {accuracy:.2f}%')

    # Confusion matrix for the fold
    cm = confusion_matrix(all_labels, all_predictions)
    cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
    confusion_matrices.append(cm_normalized)

    plt.figure(figsize=(10, 8))
    sns.heatmap(cm_normalized, annot=True, fmt=".2%", cmap="Blues",
                xticklabels=["Healthy", "Moderately Healthy", "Early Necrosis", "Moderate Necrosis", "Severe Necrosis", "Dead or Inanimate Object"],
                yticklabels=["Healthy", "Moderately Healthy", "Early Necrosis", "Moderate Necrosis", "Severe Necrosis", "Dead or Inanimate Object"])
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.title(f'Confusion Matrix - Fold {fold}')
    plt.show()

# Move to the next fold
fold += 1
```

Figure 4.62: Training loop part 4

Figure 4.62 shows the last part of the training loop code and performs a post-training evaluation for the current fold using the trained model. The torch.no_grad() context ensures that no gradients are computed to allow for reduced memory usage. The model is then set to evaluation mode (model.eval()), disabling dropout and batch normalization to ensure consistent predictions. The evaluation loop iterates through the test_loader, where inputs and labels are moved to the appropriate device (CPU or GPU). The model makes predictions, and torch.max(outputs.data, 1) selects the highest probability class for each input. The total number of correct predictions is tracked, and classification accuracy is computed as (correct/total) * 100. The model's final learned weights from fc7 are extracted and stored for analysis. A confusion matrix is generated to visualize

classification performance across different categories, and a heatmap is plotted to show the normalized confusion matrix. The confusion matrix will be shown in Chapter 5.

```
# After cross-validation: Calculate and display average accuracy and consolidated confusion matrix
avg_accuracy = sum(fold_accuracies) / len(fold_accuracies)

# Count the number of fully connected layers starting from fc2
num_fc_layers = len([layer for layer in dir(Simple3DCNN()) if layer.startswith('fc')]) - 1

# Display Model Results:
print(f"\n=== Model Summary ===")
print(f"Batch Size: {batch_size}")
print(f"Learning Rate: {lr}")

# Display initial class weights with 2 decimal formatting
initial_weights_to_display = [weights[:6] for weights in initial_weights_list]
print("\nInitial Class Weights for CrossEntropyLoss (showing up to 6 values per fold):")
for i, weights in enumerate(initial_weights_to_display):
    formatted_weights = ', '.join(f"{w:.2f}" for w in weights)
    print(f"Fold {i + 1}: [{formatted_weights}]")

# Display final class weights and accuracy for each fold
final_weights_to_display = [weights[:6] for weights in final_weights_list]
print("\nFinal Class Weights at End of Training and Accuracy per Fold (showing up to 6 values per fold):")
for i, (weights, accuracy) in enumerate(zip(final_weights_to_display, fold_accuracies)):
    formatted_weights = ', '.join(f"{w:.3f}" for w in weights)
    print(f"Fold {i + 1}: Weights: [{formatted_weights}], Accuracy: {accuracy:.2f}%")

print(f"Patience (Early Stopping): {patience}")
print(f"Weight Decay: {weight_decay}")
print(f"Total Number of Epochs: {num_epochs}")
print(f"Number of Epochs completed with Early Stopping per fold: {epoch_counts}")
print(f"Number of Fully Connected Layers (excluding fc1): {num_fc_layers}")
print(f"Dropout Probability: {model.dropout.p}")
print(f"Average Accuracy across {k} folds: {avg_accuracy:.2f}%")

# Consolidate confusion matrices into a single matrix
consolidated_cm = np.sum(confusion_matrices, axis=0)
consolidated_cm_norm = consolidated_cm.astype('float') / consolidated_cm.sum(axis=1)[:, np.newaxis]

plt.figure(figsize=(10, 8))
sns.heatmap(consolidated_cm_norm, annot=True, fmt=".2%", cmap="Blues",
            xticklabels=["Healthy", "Moderately Healthy", "Early Necrosis", "Moderate Necrosis", "Severe Necrosis", "Dead or Inanimate Object"],
            yticklabels=["Healthy", "Moderately Healthy", "Early Necrosis", "Moderate Necrosis", "Severe Necrosis", "Dead or Inanimate Object"])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Consolidated Confusion Matrix - All Folds')
plt.show()
```

Figure 4.63: Code for displaying the model's results

Figure 4.63 represents the code snippet for displaying the final results of the model. Chapter 5 will provide a more detailed discussion of these results. The average accuracy across all folds is displayed, along with all key hyperparameters such as batch size, learning rate, weight decay, and dropout probability. The number of epochs completed before early stopping for each fold is also shown. The initial and final class weights for each fold are displayed, illustrating how they evolved throughout the five-fold cross-validation process along with their respective accuracies. Finally,

the confusion matrices from all folds are consolidated into a single confusion matrix, providing a comprehensive visualization of the model's classification accuracy across all six class labels.

Chapter 5

Numerical Experiments

5.1 Lighting Issues

Before any data collection and model building, one of the first issues encountered in this project was the lighting. As discussed in the previous chapter, hyperspectral cameras require special lighting. The lighting either needs to be sunlight or halogen light bulbs to provide the full electromagnetic light spectrum. Led bulbs or fluorescence office lighting do not generate the full spectrum and flicker [28, 29, 30]. Initially, halogen lighting was not known to me and I experienced a lot of data loss and software errors inside the camera's GUI. The other issue with the lighting was the over-illumination of the object. Figure 5.1 shows the error message when there is over-illumination in the image.

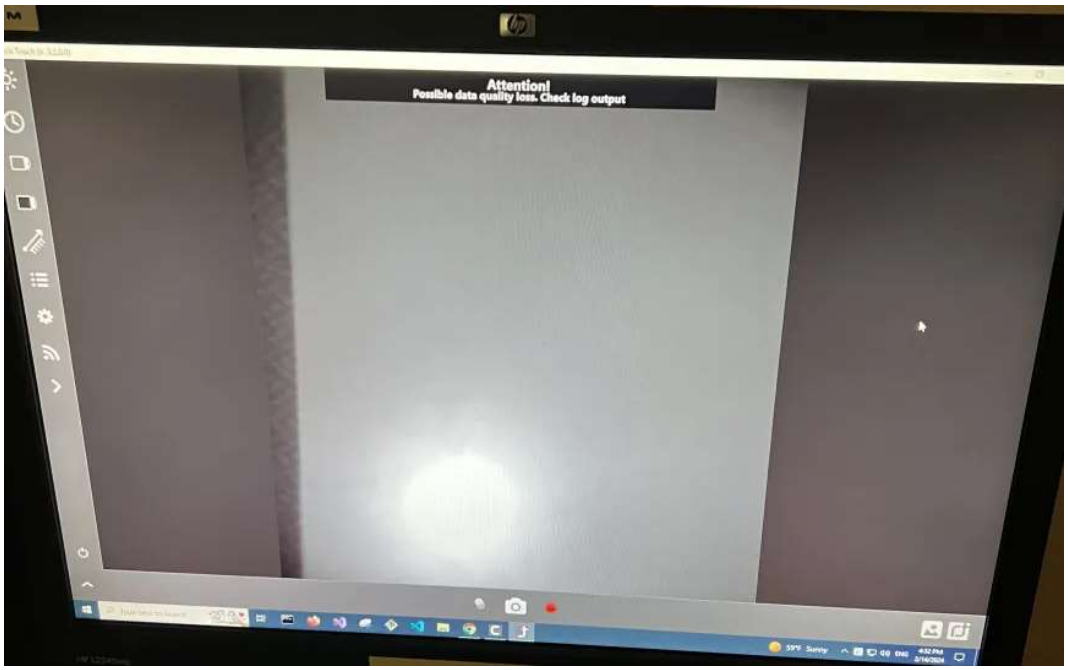

Figure 5.1: Over illumination from light source warning

The CubertTouch software that I used to collect the images, will provide a warning message if it detects issues with the lighting. In Figure 5.1, I was not using halogen lighting at this time. Also, I was not tilting the backdrop and we can see a white ball showing in the lower portion of the image. This showed that I would need to tilt the backdrop as this was a clear case of over-illumination of the image. Figures 5.2 and 5.3 below will illustrate what happens when I am not using halogen lighting.

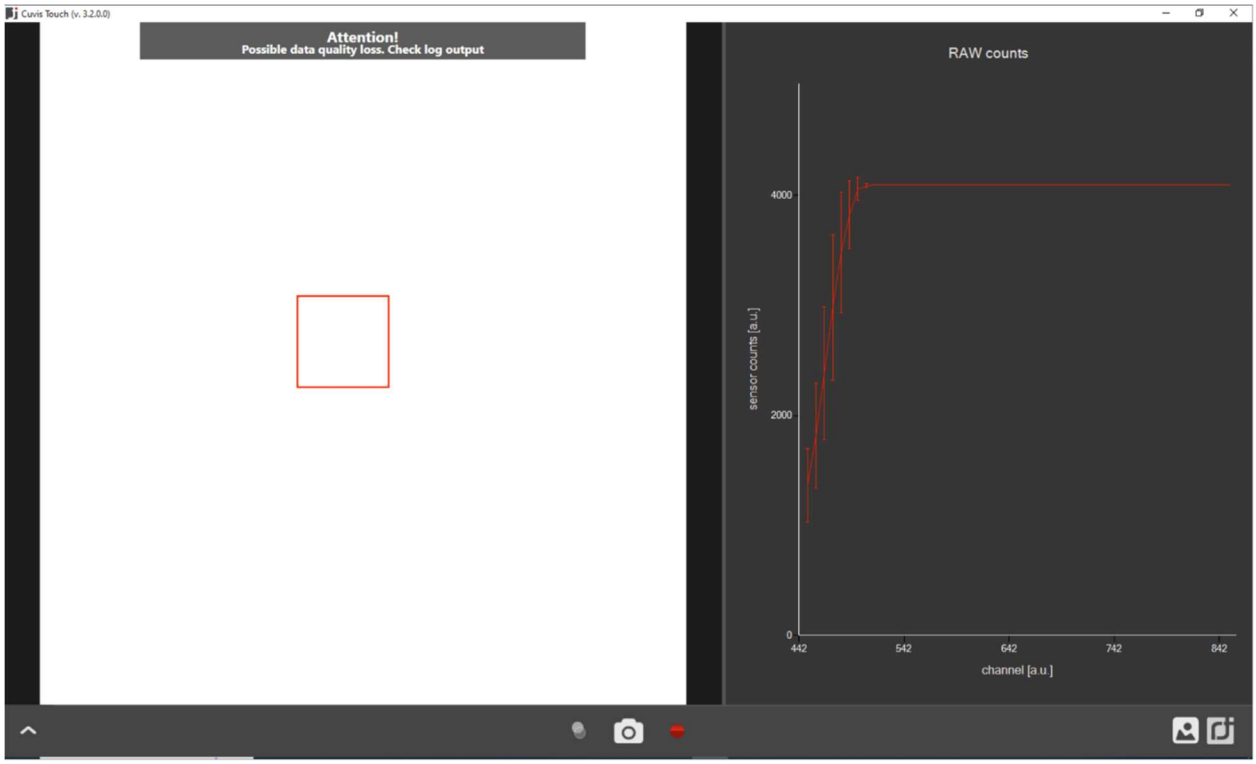

Figure 5.2: Over illumination from light source warning

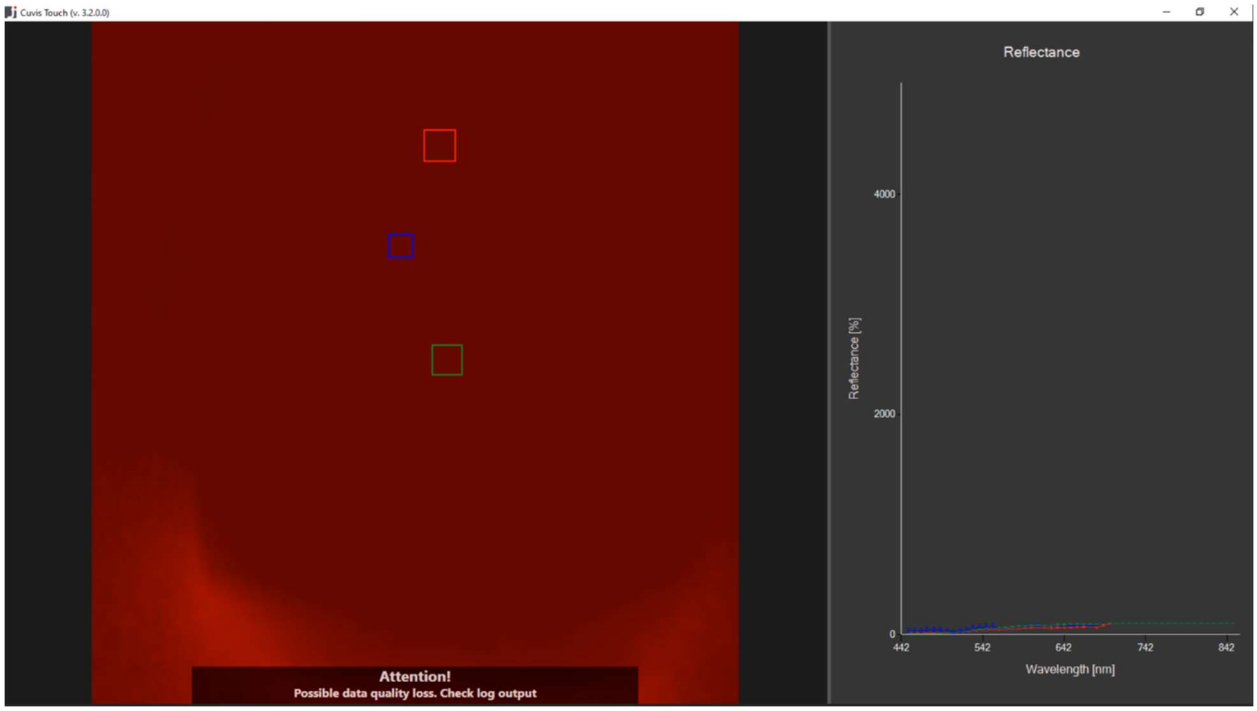

Figure 5.3: NDVI filter without halogen lighting

Figures 5.2 and 5.3 above showcase what happens when no halogen lighting is in use. In this case, office lighting was in use. Despite the white reference calibration picture in Figure 5.2 looking correct, the issue was that the lighting was inconsistent. The spectral graph on the right side shows that several light frequencies were not represented. Figure 5.3 illustrates after the calibration had taken place, the built-in NDVI filter was applied to the white background. We can see that some parts of the image on the left-hand side are darker and brighter. The spectral graph on the right side shows very low reflectance.

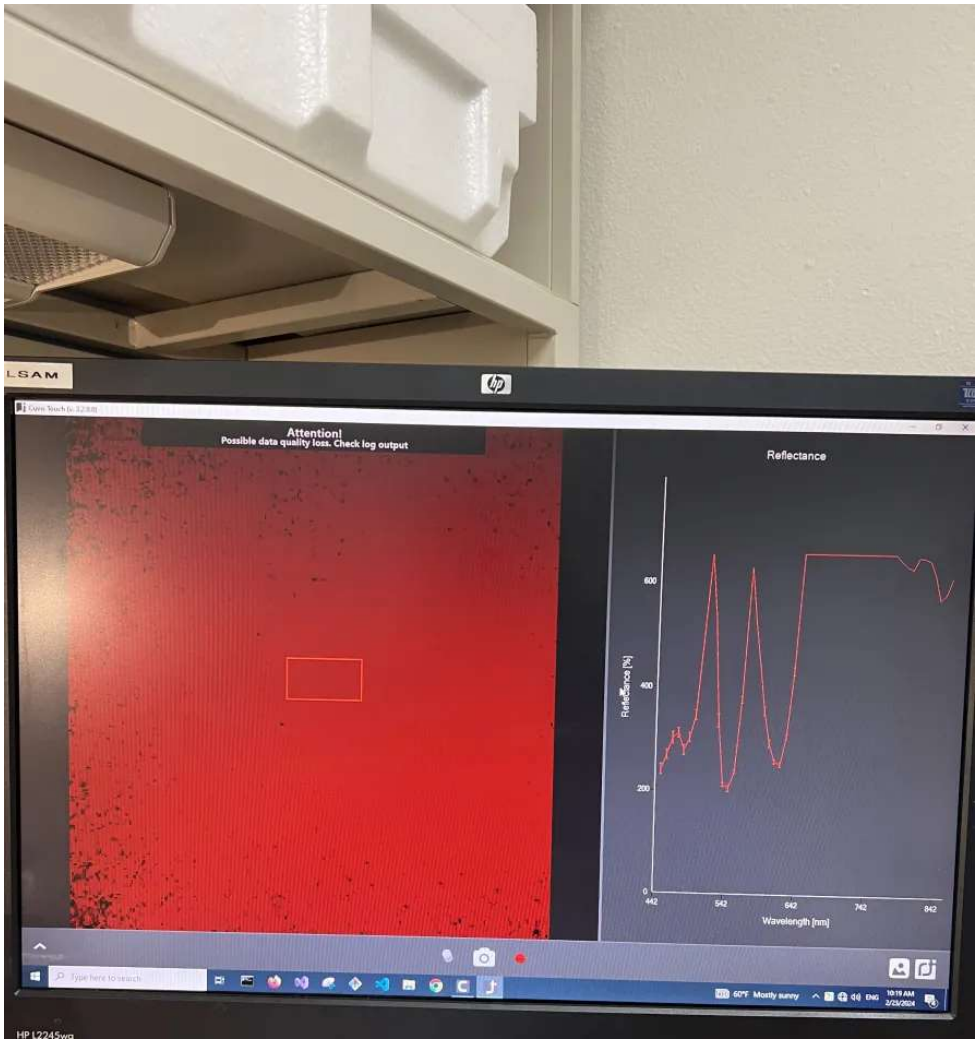

Figure 5.4: Halogen light source with office lighting

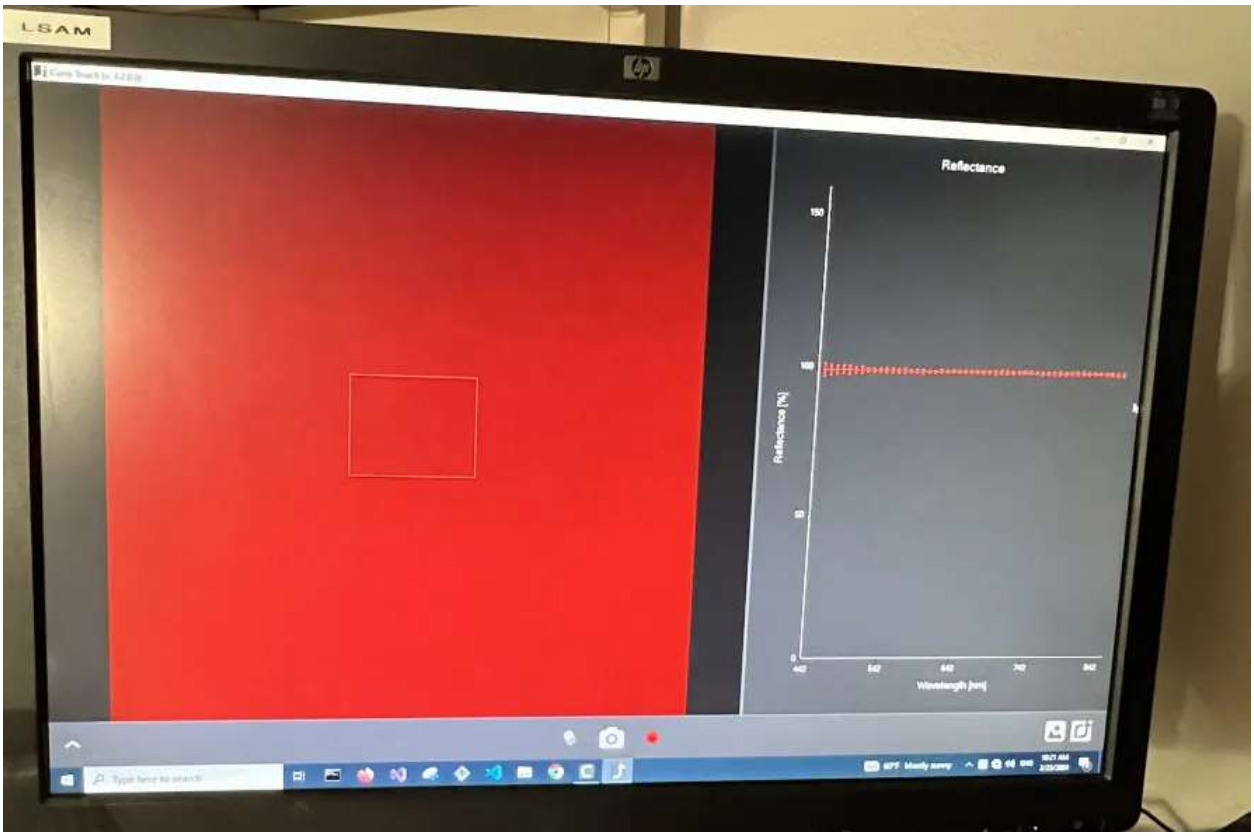

Figure 5.5: Halogen light source without office lighting

Figures 5.4 and 5.5 illustrate the impact of different lighting conditions on the camera's data collection. In Figure 5.4, both the halogen light source and the ambient office lighting illuminate the target, causing interference in the spectral data. This is evident from the warning message displayed by the camera's software and the irregular dips in the spectral graph, indicating anomalies in lighting conditions. In contrast, Figure 5.5 demonstrates how using only the halogen light source as the primary illumination resolves these issues. The spectral graph in this figure exhibits a nearly perfect horizontal line, signifying consistent and uniform lighting. In both figures, the NDVI filter was applied to verify the camera's image quality.

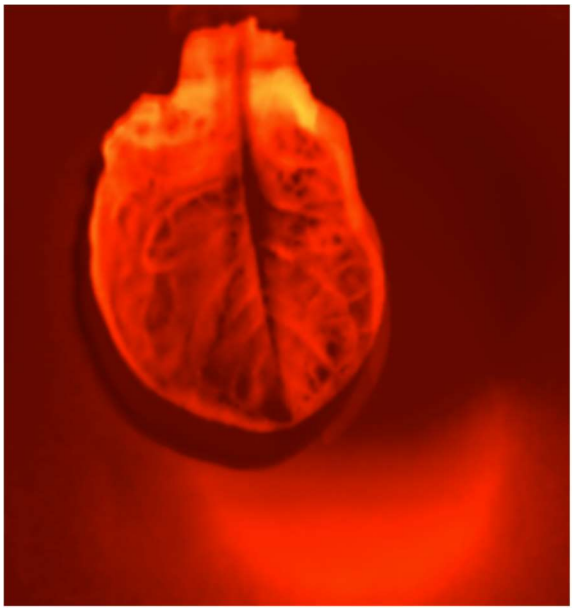

Figure 5.6: Lettuce leaf with improper lighting

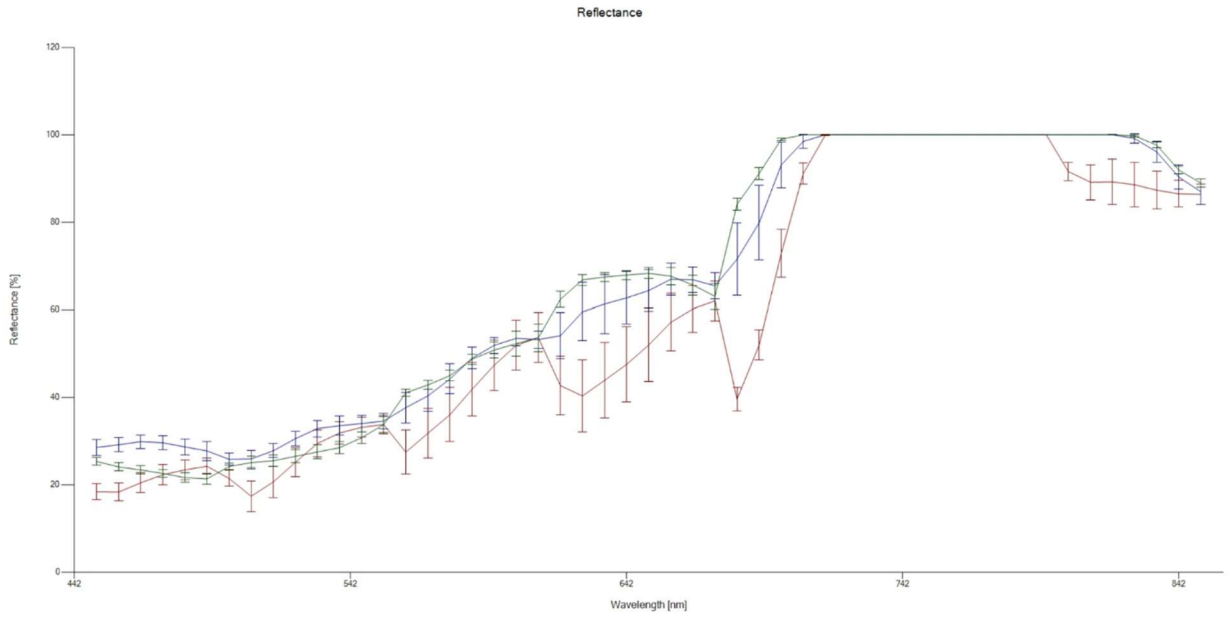

Figure 5.7: Lettuce leaf's spectral graph

Figures 5.6 and 5.7 illustrate how using improper lighting affects both the image quality and the spectral graph. The NDVI filter was applied for Figure 5.6. The spectral graph in Figure 5.7 provides a visual representation of the reflectance values of each pixel in Figure 5.6. If the pixel values are not captured with good lighting, the NDVI calculation that is applied in the model will

be erroneous. Figures 5.8 and 5.9 show the differences in performance when proper halogen lighting is used.

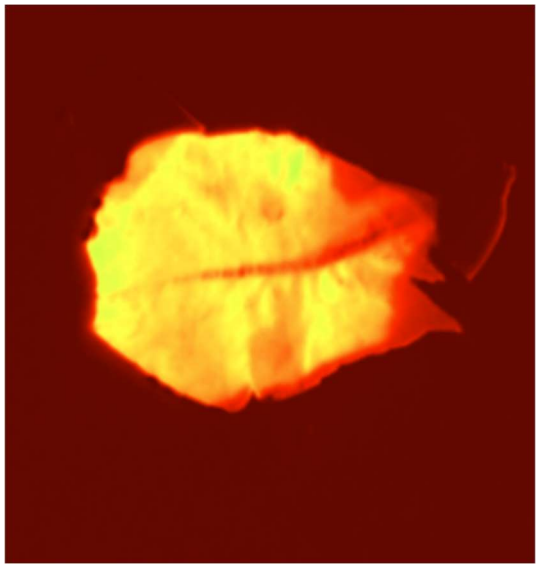

Figure 5.8: Lettuce leaf with proper lighting

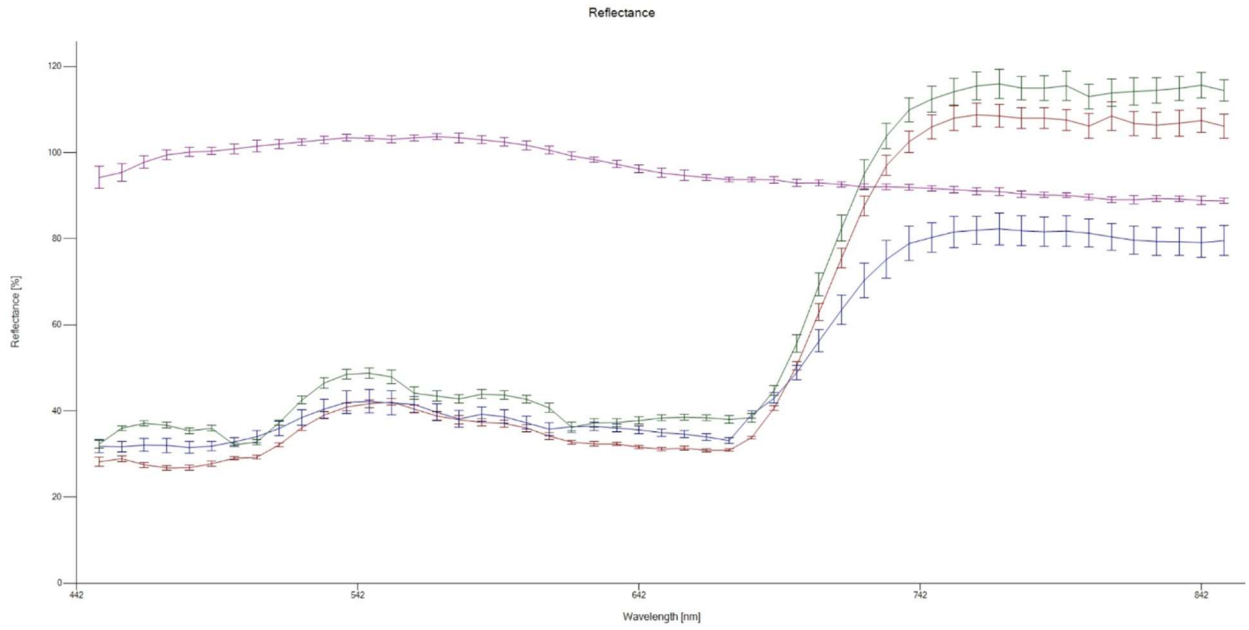

Figure 5.9: Associated spectral graph

Figures 5.8 and 5.9 demonstrate the massive performance differences when halogen lighting is used. Figure 5.8 looks much more clear than the leaf image in Figure 5.6. We can also see the

spectral graph in Figure 5.9 has fewer dips and random spikes than in Figure 5.7. The bars that are shown in Figures 5.7 and 5.9 are small error bars. Even though some of them look quite large, it was verified that these values were below 5% error. The results of Figures 5.1 to 5.9 highlighted the importance of proper lighting and how it can affect the overall performance of the model. Now that the lighting issues were fixed, I then moved on to the image collection and later testing the images with the model.

5.2 Preliminary Testing

The testing for this model first began with only healthy sweet potato leaves and two class labels. Approximately 500 leaves were collected in July and were later formatted into the dataset. Initially, this project started as a binary classification task where the only two class labels at the time were “Healthy” and “Unhealthy”. The initial goal was to implement a basic CNN model that could handle hyperspectral data and classify the plant data into those two class labels. During the preliminary testing, several key issues needed to be addressed related to the model. These issues were the NDVI calculation being accurate, using a 2D or 3D CNN, and the number of class labels. One of the first tests performed in this project was implementing the NDVI calculation. Before creating the entire program shown throughout Chapter 4, I needed to verify that specific operations functions worked correctly. The functions that needed to be properly tested were reading in files, evaluating spectral data files, and testing the NDVI formula on spectral images. A test program was created to load spectral files and perform the NDVI formula on the spectral images. Figure 5.1 shows how the spectral images are viewable in Python.

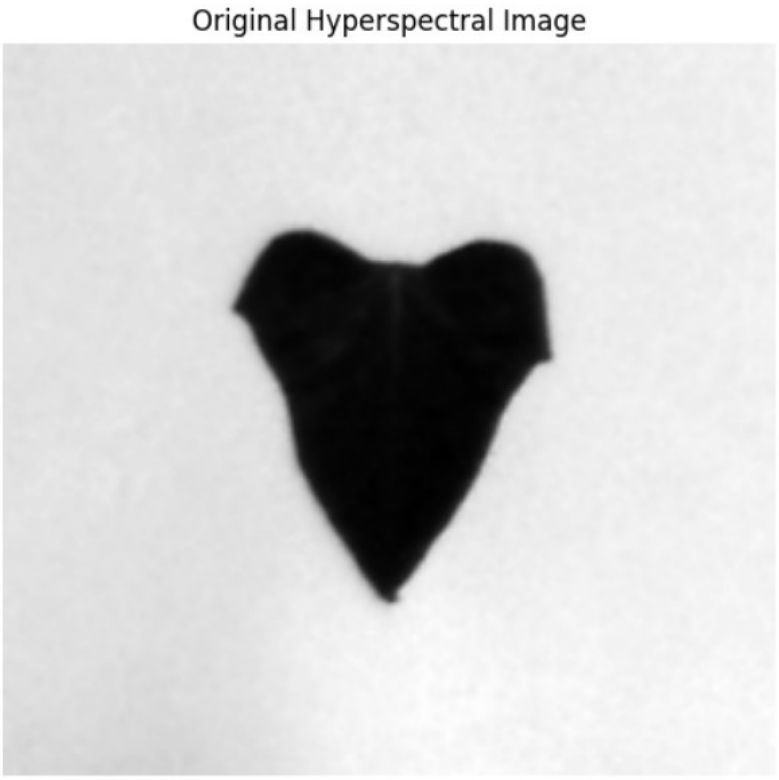

Figure 5.10: Original hyperspectral image

Figure 5.10 shows the output result of a test program created to test how the hyperspectral data could be loaded into a program and later into a model. All spectral images from the dataset by default look like the monochromatic image above. This image is also using all 51 spectral bands. It was only when I applied specific bands from the header files and sliced the spectral data file that I could recreate an artificial RGB image of a hyperspectral data file. Figure 5.11 shows an artificial RGB image of Figure 5.10.

Figure 5.11: RGB image of Figure 5.10

The figure above shows how I could choose specific spectral bands from a data file and process that data accordingly. In this case, I had chosen three spectral bands that lay within the blue (450 nm – 500 nm), green (500nm – 570 nm), and red (620 – 750 nm) wavelengths. This concept is array indexing. The 51 bands are loaded into an array that has 51 elements. The 51 elements are defined from the header file (.hdr) that was previously shown in Chapter 4. By indexing the band array at various locations, I could save specific bands and use them for specific tasks. The next step after figuring out how to load spectral images in Python and select specific bands, was to apply the NDVI formula. Figure 5.12 shows some preliminary results of band indexing for the NDVI formula.

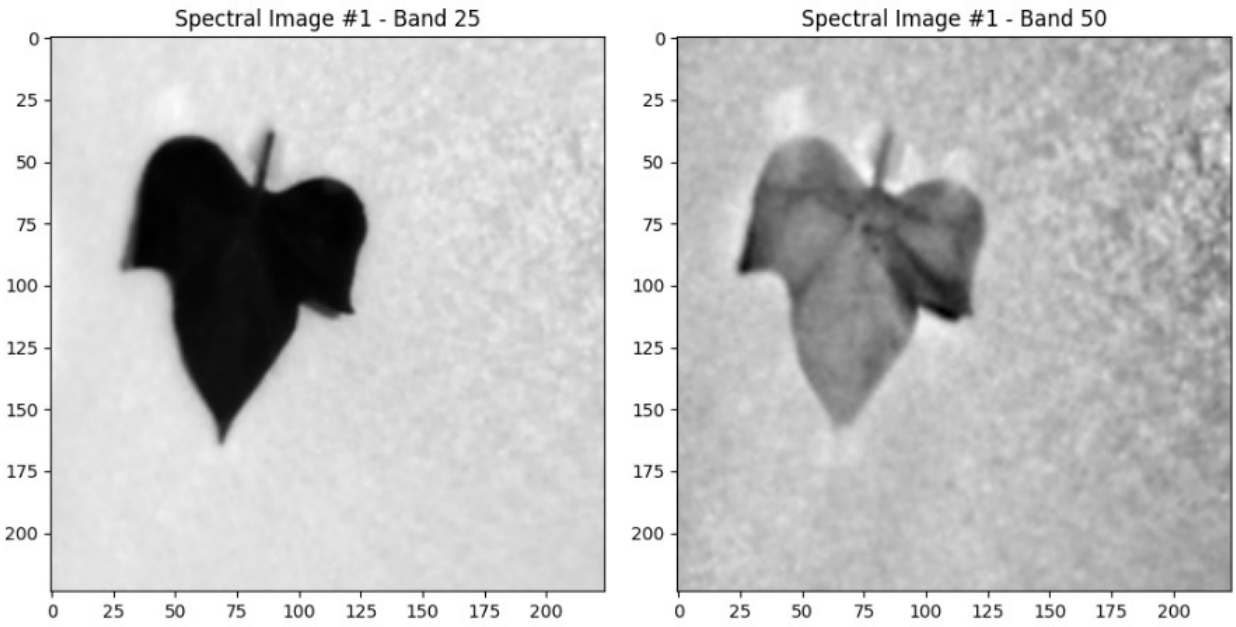

Figure 5.12: Indexing bands 25 and 50

In the figure above, we can see the same leaf image at different spectral bands. In this case, bands 25 and 50 were sampled. At different frequencies, a spectral image will have a different look based on the selected wavelength. Again, these images use a single spectral band so they will appear to be monochromatic and have no real color to them unless multiple spectral bands are combined to form a single image.

Now that I have found a way to reliably load in the spectral data and index specific bands, I then moved on to the NDVI formula. As mentioned earlier in this report, the NDVI formula takes red and near-infrared light and produces a number between -1 and 1. I experimented with different bands within the red and infrared region and ultimately settled for band 23 (634 nm) and band 45 (810 nm). However, I ran into an issue with my original implementation of the NDVI formula. Initially, the NDVI formula would be applied to the entire 290 x 275 pixeled data file. This proved to be erroneous as the computed NDVI values had significant numerical errors.

Pixel values that were not part of the leaf pixel region, like background pixels, were also calculated to the overall NDVI value. To fix this error, I needed to create a region of interest (ROI) for correctly calculating the appropriate leaf pixels. I ultimately settled on creating binary masked images for all sweet potato leaves in the dataset. But before creating masked images, I initially thought of creating a bounding box to target the pixels within the box for the NDVI calculation. However, I soon learned that this was very ineffective as it still caused errors in the NDVI calculation. Figure 5.13 shows a bounding box being applied to a hyperspectral image.

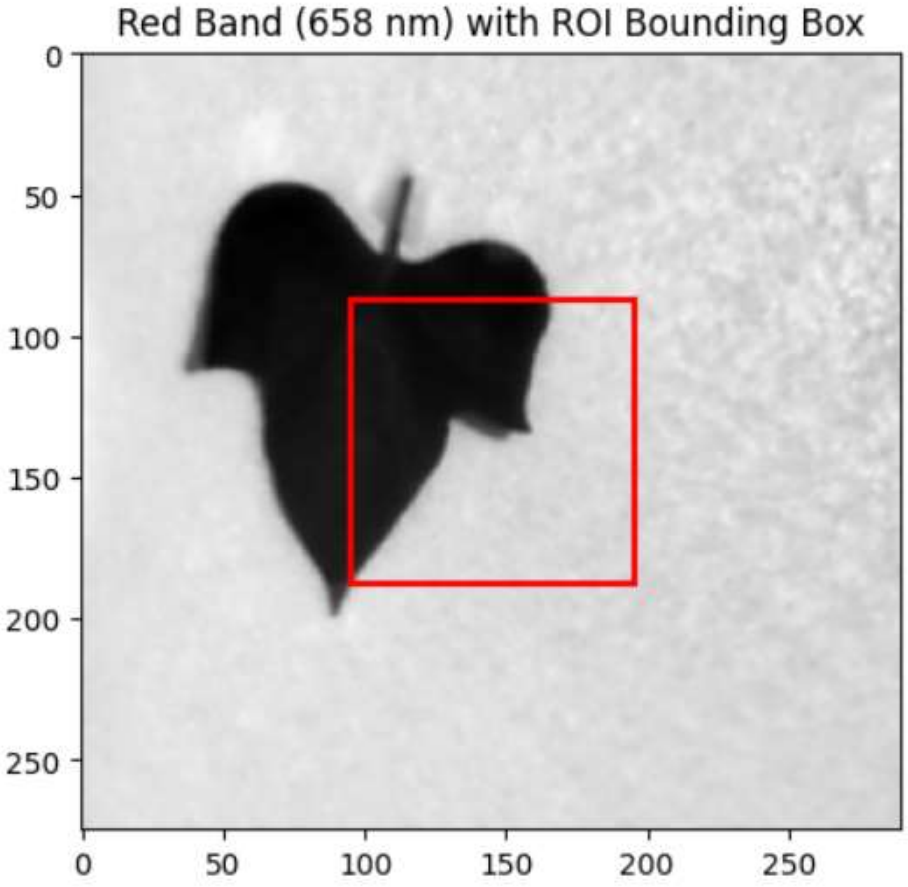

Figure 5.13: Bounding box

The figure above shows one of many experiments that I did with bounding boxes to create a region of interest. The pixels that are inside the red lines will be used in the NDVI calculation. However,

you can see visually how in Figure 5.13 for example, only a portion of the leaf pixels would contribute to the NDVI calculation and the rest of the pixels would be background pixels. This was a very common issue as this bounding box idea assumed that all images in the dataset would be right in the center of the box but this was not the case. Instead of using the bounding, I decided to create binary masked images instead. This proved to be far more effective at calculating accurate NDVI values. Figures 5.14 and 5.15 show a binary mask image and how my preprocessing functions from Chapter 4 operate behind the scenes.

Figure 5.14: Binary masked image

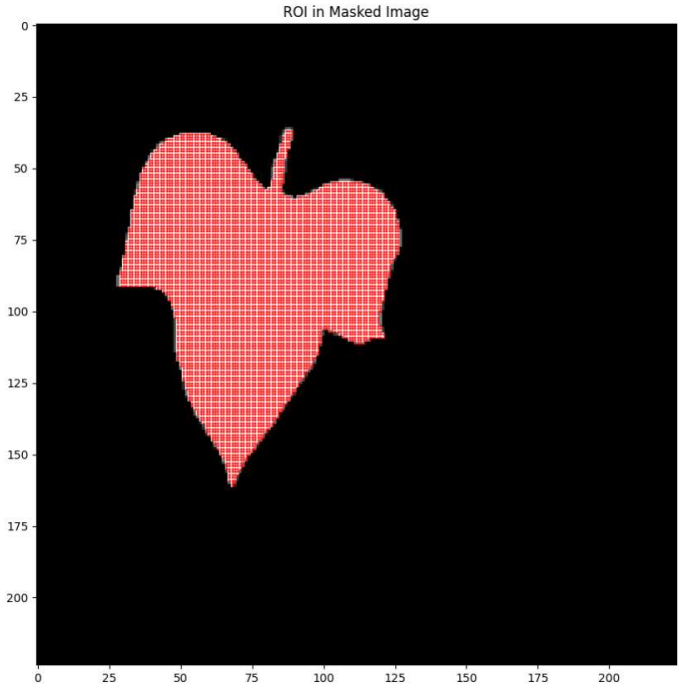

Figure 5.15: Binary masked image with highlighted pixels for NDVI calculations

Figure 5.14 shows what the binary masked images look like for all sweet potato leaf images. Figure 5.15 shows a visual representation of how the preprocessing functions from Chapter 4 target only the pixels that are within the leaf and not the background. Focusing on the pixels in that highlighted region, greatly improved the overall accuracy of the NDVI calculations. Figure 5.16 shows a comparison between NDVI calculations with a masked image and without one.

```
NDVI Values (Using Masked Images):
Image 1: NDVI = 0.5470
Image 2: NDVI = 0.5166
Image 3: NDVI = 0.5719
Image 4: NDVI = 0.5212
Image 5: NDVI = 0.5731
Image 6: NDVI = 0.5994
Image 7: NDVI = 0.5658
Image 8: NDVI = 0.4899
Image 9: NDVI = 0.4952
Image 10: NDVI = 0.6259

NDVI Values (Without Masked Images, Full Image Used):
Image 1: NDVI = 0.0837
Image 2: NDVI = 0.1245
Image 3: NDVI = 0.0926
Image 4: NDVI = 0.0908
Image 5: NDVI = 0.0953
Image 6: NDVI = 0.1046
Image 7: NDVI = 0.0942
Image 8: NDVI = 0.0861
Image 9: NDVI = 0.0918
Image 10: NDVI = 0.0939
```

Figure 5.16: NDVI calculations before and after an applied masked image

Figure 5.16 shows the output from a testing program that I used to verify the difference between using masked images and no masked images when calculating the NDVI values. Note that this particular test program took the entire hyperspectral dataset and displayed the first ten images. It is also worth noting that the dataset being loaded into this program was not shuffled, and is using the healthy plant images. Since healthy plant images are the beginning of the dataset, we should expect the NDVI values for these images to be above or near 0.5. We can see that when I used masked images with the highlighted region from figure 5.16, the NDVI values were above or near 0.5. However, when no masked images were used, the NDVI values were low. Some were not even at 0.1. If we are going based on the Figure 4.1 from Chapter 4, which showed the range for NDVI, these plants would be considered unhealthy. However, that was not the case and it turned out that it was because the initial process I was doing for the NDVI calculations done was erroneous.

After fixing the NDVI calculations, the next step was to work on formatting the images. My dataset folder was 13 GB and trying to load that much data was very cumbersome on the computer. Also, the images were 290 x 275 pixels and were using all 51 spectral bands. Since the NDVI formula only needs two bands to work properly, the other 49 bands are not needed. Reducing the number of bands significantly reduced the resource intensity of the dataset. I experimented with two sizes: 256 x 256 and 224 x 224 since they are standard image dimensions. I ultimately settled on 224 x 224 as the new image size did not it significantly remove portions of the leaf pixels. Only the background pixels were being lost and those pixel values doe not contribute to the NDVI calculations. Figure 5.17 shows the various image dimensions and how much storage each image takes up.

```
Original 51-band Shape: torch.Size([275, 290, 51]) - Storage: 16269000 bytes, 15.52 MB, 0.0152 GB
Original 2-band Shape: torch.Size([275, 290, 2]) - Storage: 638000 bytes, 0.61 MB, 0.0006 GB

Resized 256x256 51-band Shape: torch.Size([256, 256, 51]) - Storage: 13369344 bytes, 12.75 MB, 0.0125 GB
Resized 256x256 2-band Shape: torch.Size([256, 256, 2]) - Storage: 524288 bytes, 0.50 MB, 0.0005 GB

Resized 224x224 51-band Shape: torch.Size([224, 224, 51]) - Storage: 10235904 bytes, 9.76 MB, 0.0095 GB
Resized 224x224 2-band Shape: torch.Size([224, 224, 2]) - Storage: 401408 bytes, 0.38 MB, 0.0004 GB
```

Figure 5.17: Image dimensions and storage capacities

Figure 5.17 above was a Python test program that I wrote to compare the storage sizes of using all 51 bands and 2 bands as well as resizing the data files. As shown in the figure, each spectral data file at the original size of 290 x 275 pixels with all 51 bands approximately takes up 15.52 MB (megabytes) of data. While in an isolated environment, a single 15.52 MB image may not be a big deal but when you have over 1700 images that are that size, that creates a huge headache during the dataloading process. We can also see that reducing the number of spectral bands to 2 drastically reduces the image size to 0.6 MB. By further reducing the image size to 224 x 224 pixels, the data

size was down to 0.38 MB per image. By multiplying 0.38 by 1717, the total storage usage for the dataset is 652 MB.

After testing the resized images, I then began creating a basic 3D CNN model that had 1 hidden layer. At this point, I had already figured out the preprocessing steps and how the images were supposed to be formatted for the model. This first model would only have two output labels: "Healthy" and "Unhealthy". At this point, I had not augmented any images yet. The dataset only contained the 500 healthy sweet potato leaves. I just wanted to see what performance I was going to get with this configuration. Figure 5.18 below shows the results of these model.

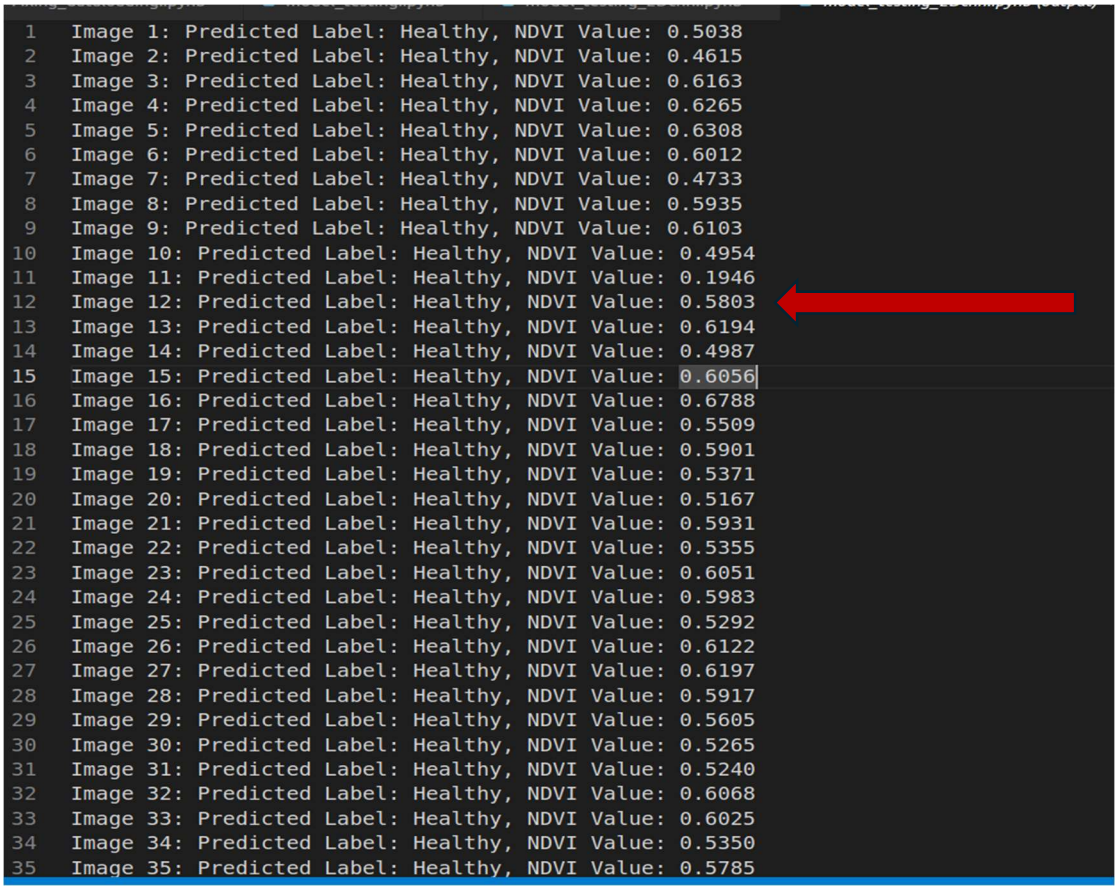

Figure 5.18: 3D CNN with 1 Hidden Layer

Figure 5.18 displays the output of the 3D CNN model with 1 hidden Layer. The hyperparameters for this version were the learning rate was set to 0.1, the batching was set to 16 and the initial weights were all set to 0.1. At this time, there were only two output labels, and the dataset predominately featured healthy leaves. I also added a threshold value for the NDVI calculation where any leaf's NDVI value was below 0.3, it should be labeled as unhealthy. In the figure above, the red arrow points to an image that was mislabeled as "Healthy" when its NDVI value was below the threshold. Adding to what is shown in Figure 5.19 above, I also observed in the output of this model version that there were hardly any plants that were getting labeled as "Unhealthy". This proved to me that I needed to get additional images, and augment my dataset, with unhealthy plants. Figures 5.19 to 5.22 below shows a revised version of the same 3D CNN model used in the previous figure.

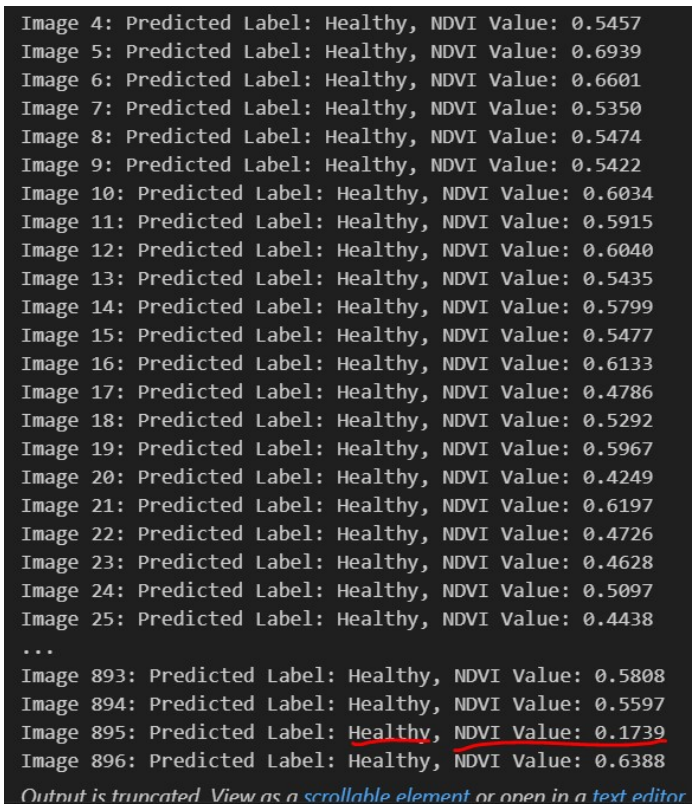

Figure 5.19: Truncated form of output labels

```
359    Image 359: Predicted Label: Healthy, NDVI Value: 0.6238
360    Image 360: Predicted Label: Healthy, NDVI Value: 0.6359
361    Image 361: Predicted Label: Healthy, NDVI Value: 0.3334
362    Image 362: Predicted Label: Healthy, NDVI Value: 0.5975
363    Image 363: Predicted Label: Healthy, NDVI Value: 0.5683
364    Image 364: Predicted Label: Healthy, NDVI Value: 0.5570
365    Image 365: Predicted Label: Healthy, NDVI Value: 0.1974
366    Image 366: Predicted Label: Healthy, NDVI Value: 0.5799
367    Image 367: Predicted Label: Healthy, NDVI Value: 0.5204
368    Image 368: Predicted Label: Healthy, NDVI Value: 0.4753
369    Image 369: Predicted Label: Healthy, NDVI Value: 0.5761
370    Image 370: Predicted Label: Healthy, NDVI Value: 0.4892
371    Image 371: Predicted Label: Healthy, NDVI Value: 0.2168
372    Image 372: Predicted Label: Healthy, NDVI Value: 0.5787
```

Figure 5.20: Output labels part 1

```
Image 746: Predicted Label: Unhealthy, NDVI Value: 0.1560
Image 747: Predicted Label: Healthy, NDVI Value: 0.5811
Image 748: Predicted Label: Healthy, NDVI Value: 0.4073
Image 749: Predicted Label: Healthy, NDVI Value: 0.4484
Image 750: Predicted Label: Healthy, NDVI Value: 0.6298
Image 751: Predicted Label: Healthy, NDVI Value: 0.6057
Image 752: Predicted Label: Healthy, NDVI Value: 0.5805
Image 753: Predicted Label: Healthy, NDVI Value: 0.5326
Image 754: Predicted Label: Healthy, NDVI Value: 0.3622
Image 755: Predicted Label: Healthy, NDVI Value: 0.5036
Image 756: Predicted Label: Healthy, NDVI Value: 0.4402
Image 757: Predicted Label: Healthy, NDVI Value: 0.5513
Image 758: Predicted Label: Healthy, NDVI Value: 0.6237
Image 759: Predicted Label: Healthy, NDVI Value: 0.4428
Image 760: Predicted Label: Healthy, NDVI Value: 0.6437
Image 761: Predicted Label: Healthy, NDVI Value: 0.5880
Image 762: Predicted Label: Healthy, NDVI Value: 0.5654
Image 763: Predicted Label: Healthy, NDVI Value: 0.5417
Image 764: Predicted Label: Healthy, NDVI Value: 0.6296
Image 765: Predicted Label: Healthy, NDVI Value: 0.5545
Image 766: Predicted Label: Healthy, NDVI Value: 0.5101
Image 767: Predicted Label: Healthy, NDVI Value: 0.6330
Image 768: Predicted Label: Unhealthy, NDVI Value: 0.1750
```

Figure 5.21: Output labels part 2

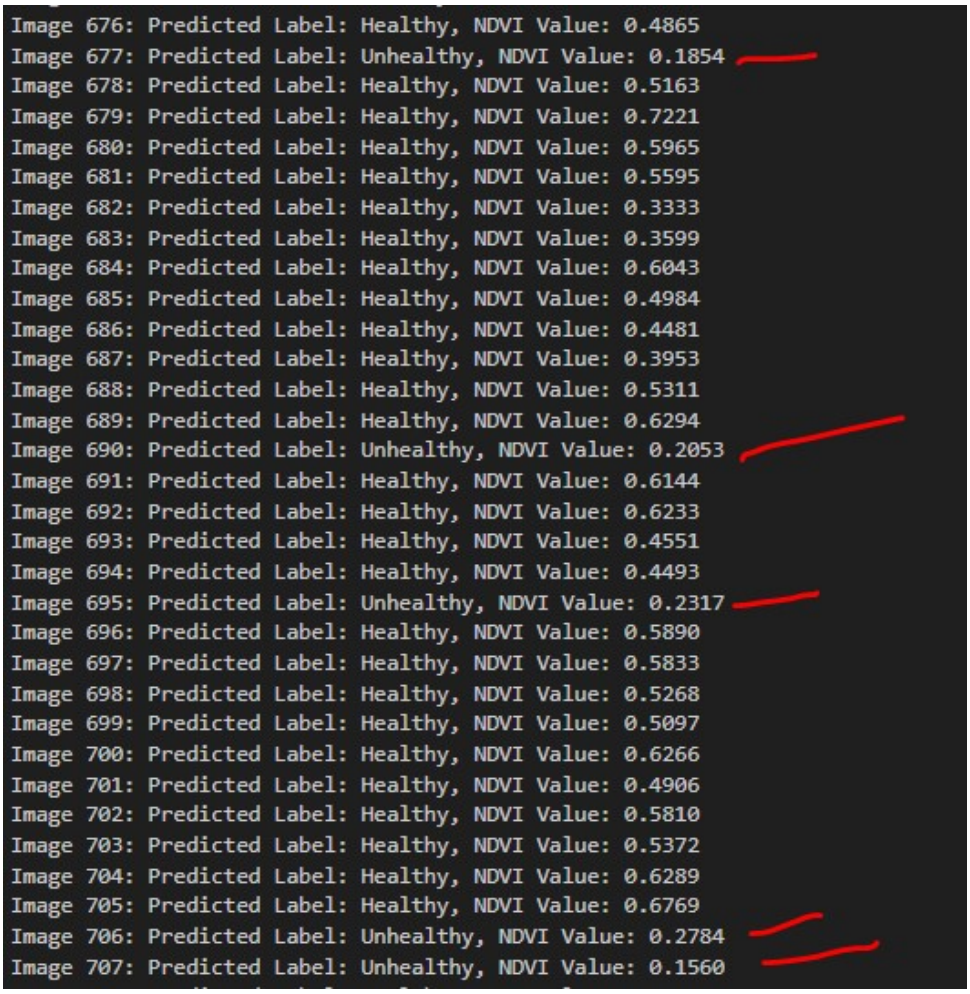

Figure 5.22: Output labels part 3

Figures 5.19 to 5.22 display a revised version of the 1 hidden layer 3D CNN model. The red pen marks in each figure highlight which leaves in the dataset were labeled as “Unhealthy” The revisements were a new learning rate of 0.01, batch sizing of 64, and two weight values of 0.7 and 0.3. An additional 400 images were added to the dataset. These images were of green-yellow, yellow, and brown sweet potato leaves shown in Chapter 4. This model still had the same two output labels as before. My observations with this version of the model were that even though additional images were added to the dataset, the model was still having a hard time labeling yellow and brown plants as “Unhealthy”. I then realized that I would need to have additional labels for

this project. I also decided to add a confusion matrix at the end when the model finished its training loop, as well as adding six counters that would count the number of NDVI values based on a predefined range. Figures 5.23 to 5.26 show the additional output metrics used to evaluate this version of the model.

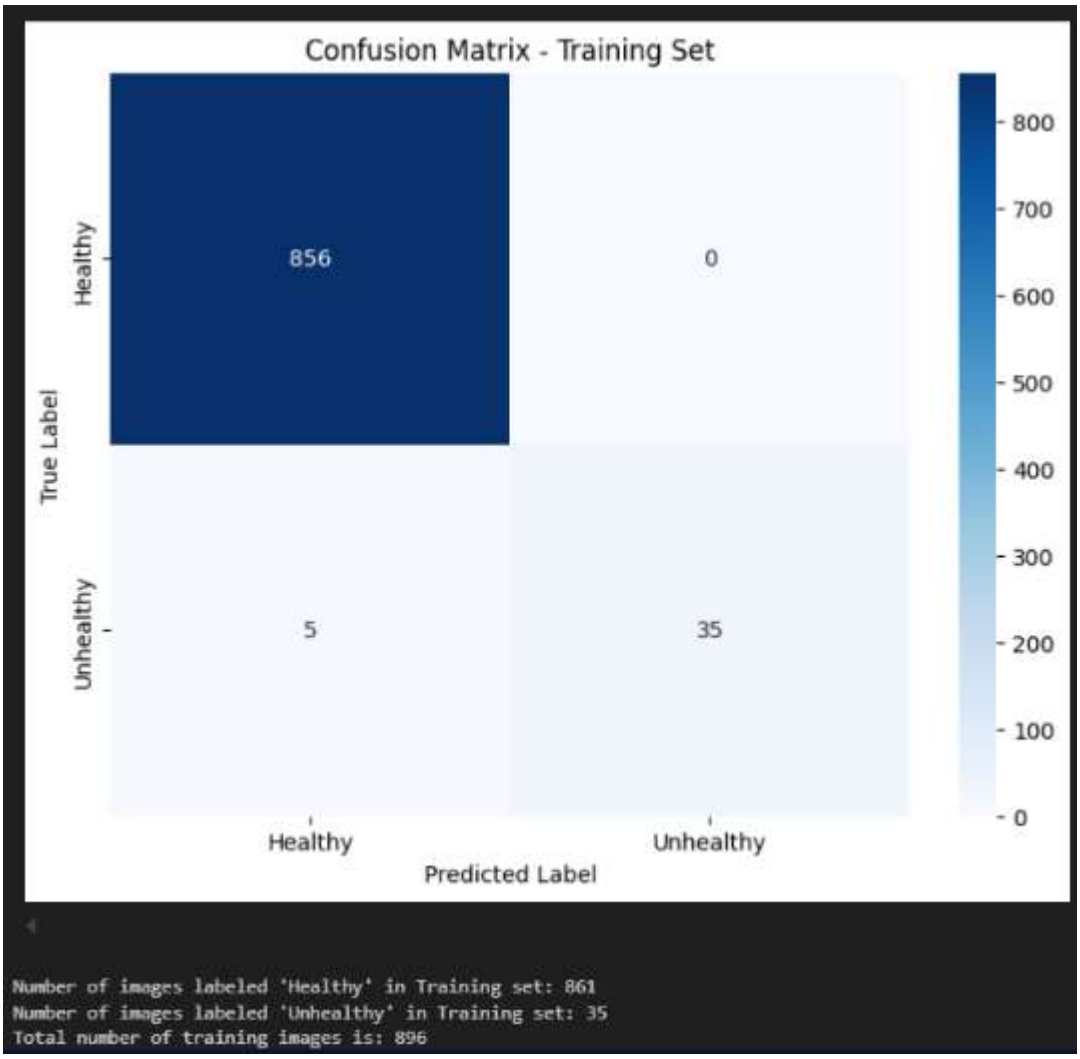

Figure 5.23: Confusion matrix for Training Set

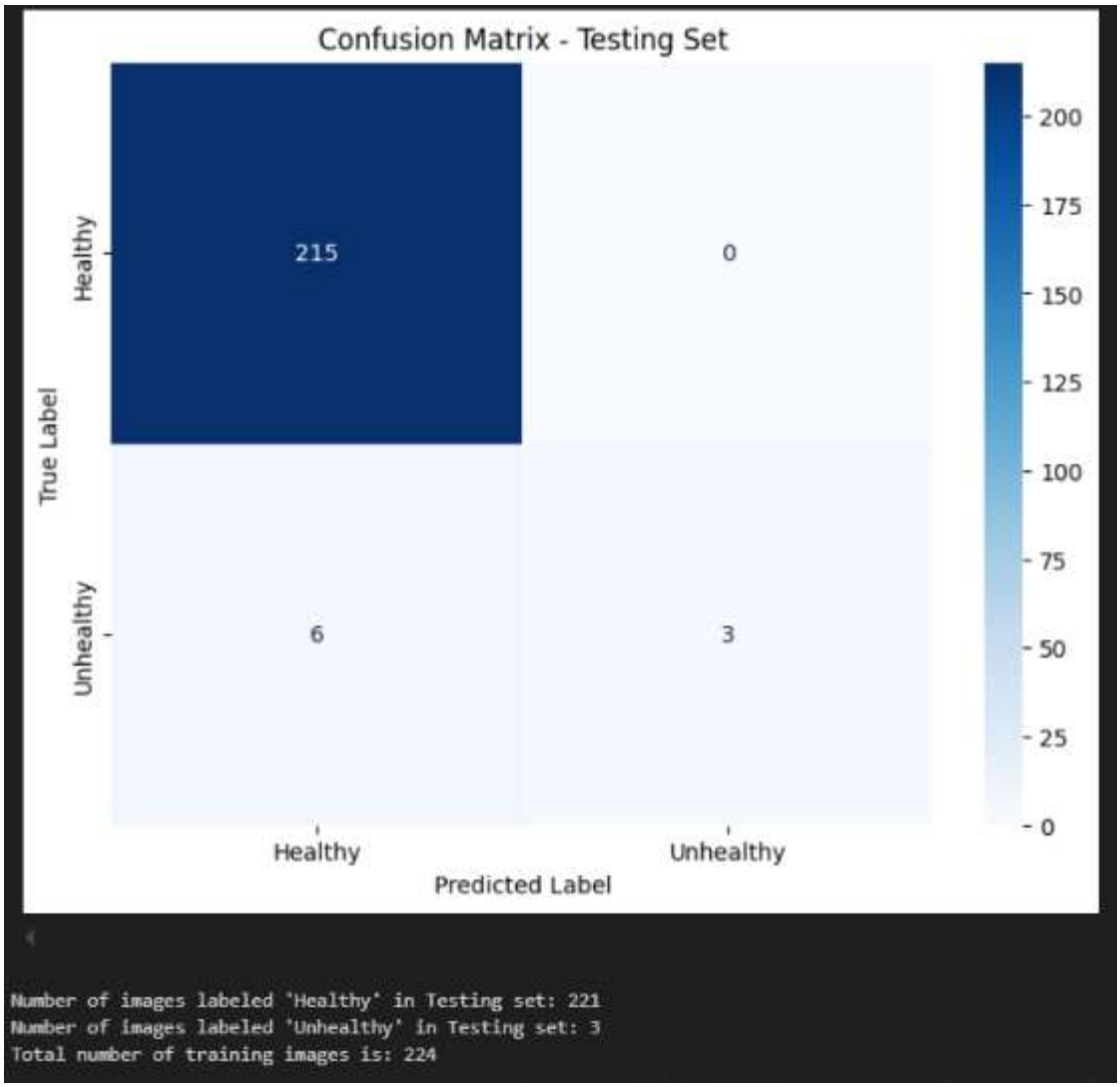

Figure 5.24: Confusion matrix for test set

```
NDVI Bucket Counts:
Leaves with NDVI <0.3: 40
Leaves with NDVI 0.3 to 0.39: 52
Leaves with NDVI 0.4 to 0.49: 151
Leaves with NDVI 0.5 to 0.59: 294
Leaves with NDVI >0.6: 267
```

Figure 5.25: Training set NDVI ranges

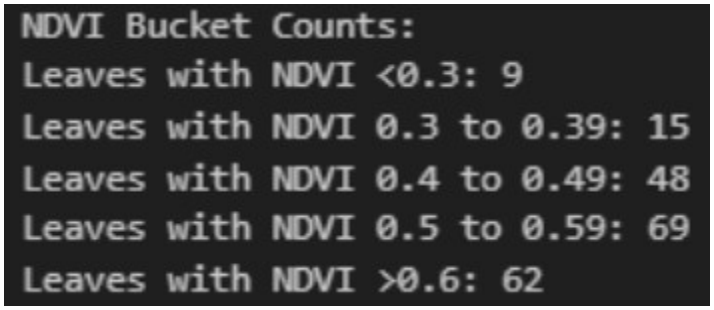

Figure 5.26: Testing set NDVI ranges

Figures 5.23 and 5.26 above show the confusion matrices for the two output labels and how they performed on the training and testing sets. We can see that the performance is not that great. We have too many images being labeled as healthy. At this point, I knew that the model's performance with this configuration was not good. I knew that I would need to an additional few hundred images of unhealthy leaves as well as expand the number of class labels from two to at least six. Figures 5.25 and 5.26 are part of the training loop and track the number of images that had a particular NDVI value. We can see that with the original threshold of 0.3 set for the "Unhealthy" class label, there is simply not enough images that are below 0.3. The majority of the plants are either healthy or somewhere in the range of moderately healthy. I decided to conduct more research into possible plant health labeling and ultimately settled on the following six class labels: "Healthy", "Moderately Healthy", "Early Necrosis", "Moderate Necrosis", "Severe Necrosis", and "Dead or Inanimate Object". These labels as mentioned earlier in this report were based on plant necrosis. I used the ranges defined in Figures 5.25 and 5.26 as the baseline for these six labels in the next revision of the model.

A new revision for the model was created and had the initial weights each set to 0.2. The learning rate was tested between 0.1 and 0.01 and the batching was reduced from 64 to 16 and later on down to 8. The figures below show the model's performance with the six plant necrosis labels. It

is also worth noting that at this point in time, the dataset was now being used inside the SDSU's TIDE service and juypter notebook environment.

Figure 5.27: Testing within the TIDE environment image labels and NDVI amounts

Figure 5.28: Confusion matrix of the training set

```
Number of images labeled 'Healthy' in Training set: 266
Number of images labeled 'Moderately Healthy' in Training set: 299
Number of images labeled 'Early Necrosis' in Training set: 154
Number of images labeled 'Moderate Necrosis' in Training set: 50
Number of images labeled 'Severe Necrosis' in Training set: 30
Number of images labeled 'Dead or Inanimate Object' in Training set: 97
```

Figure 5.29: Number of labels in the training set

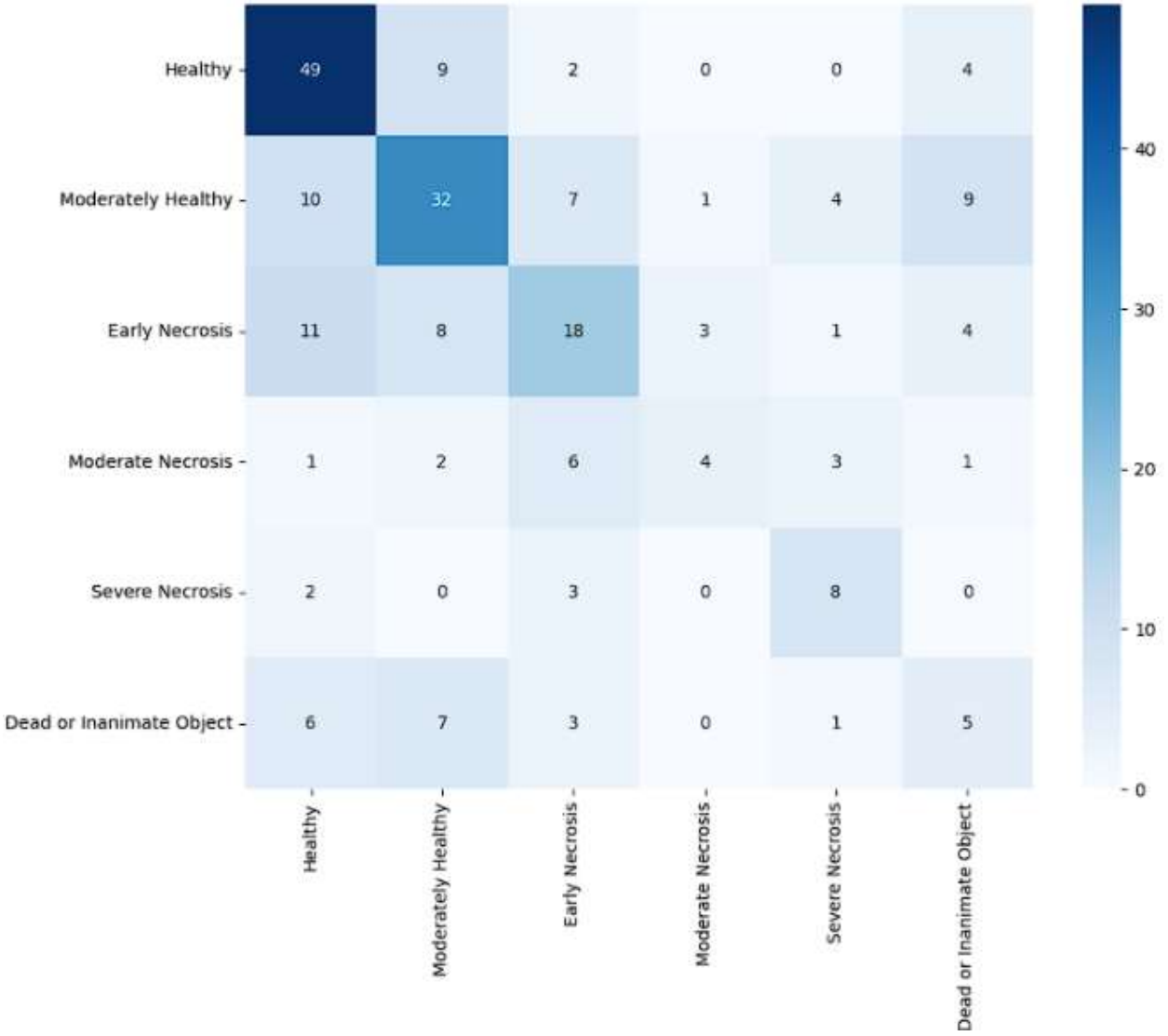

Figure 5.30: Confusion matrix of the testing set

```
Number of images labeled 'Healthy' in Testing set: 79
Number of images labeled 'Moderately Healthy' in Testing set: 58
Number of images labeled 'Early Necrosis' in Testing set: 39
Number of images labeled 'Moderate Necrosis' in Testing set: 8
Number of images labeled 'Severe Necrosis' in Testing set: 17
Number of images labeled 'Dead or Inanimate Object' in Testing set: 23
```

Figure 5.31: Number of labels in the testing set

Figures 5.27 to 5.31 above display the performance of the model with six class labels. I found that the model was very good at finding healthy plants but still not good at finding the other five class labels. Figure 5.27 shows in more detail how specific images are being mislabeled. Additionally, figures 5.28 to 5.31 show the distribution of labeled images within the training set and the testing set. Despite the confusion matrices Figures 5.28 and 5.30 show that there were few misclassifications, hence the near-perfect diagonal, this was a false positive. This result was wrong because Figure 5.27 shows the confusion matrices displaying incorrect data.

The next step in correcting the model was to expand the number of hidden layers as well as conduct exhaustive testing of all of the primary hyperparameters. The hyperparameters like the learning rate, batch size, number of epochs, neuron count, and initial weights. Overall accuracy was chosen as the primary evaluation metric. Machine learning performance techniques like cross-validation and early stopping were also added. Figures 5.32 and 5.33 show one of the first tests done on the number of epochs and the use of cross-validation.

```
Fold 1/5
Epoch [1/10], Fold [1/5], Loss: 1.4658
Epoch [2/10], Fold [1/5], Loss: 1.1020
Epoch [3/10], Fold [1/5], Loss: 0.9168
Epoch [4/10], Fold [1/5], Loss: 0.7935
Epoch [5/10], Fold [1/5], Loss: 0.7780
Epoch [6/10], Fold [1/5], Loss: 0.7049
Epoch [7/10], Fold [1/5], Loss: 0.6194
Epoch [8/10], Fold [1/5], Loss: 0.6271
Epoch [9/10], Fold [1/5], Loss: 0.5908
Epoch [10/10], Fold [1/5], Loss: 0.4846
Fold 1/5 Test Accuracy: 71.88%
```

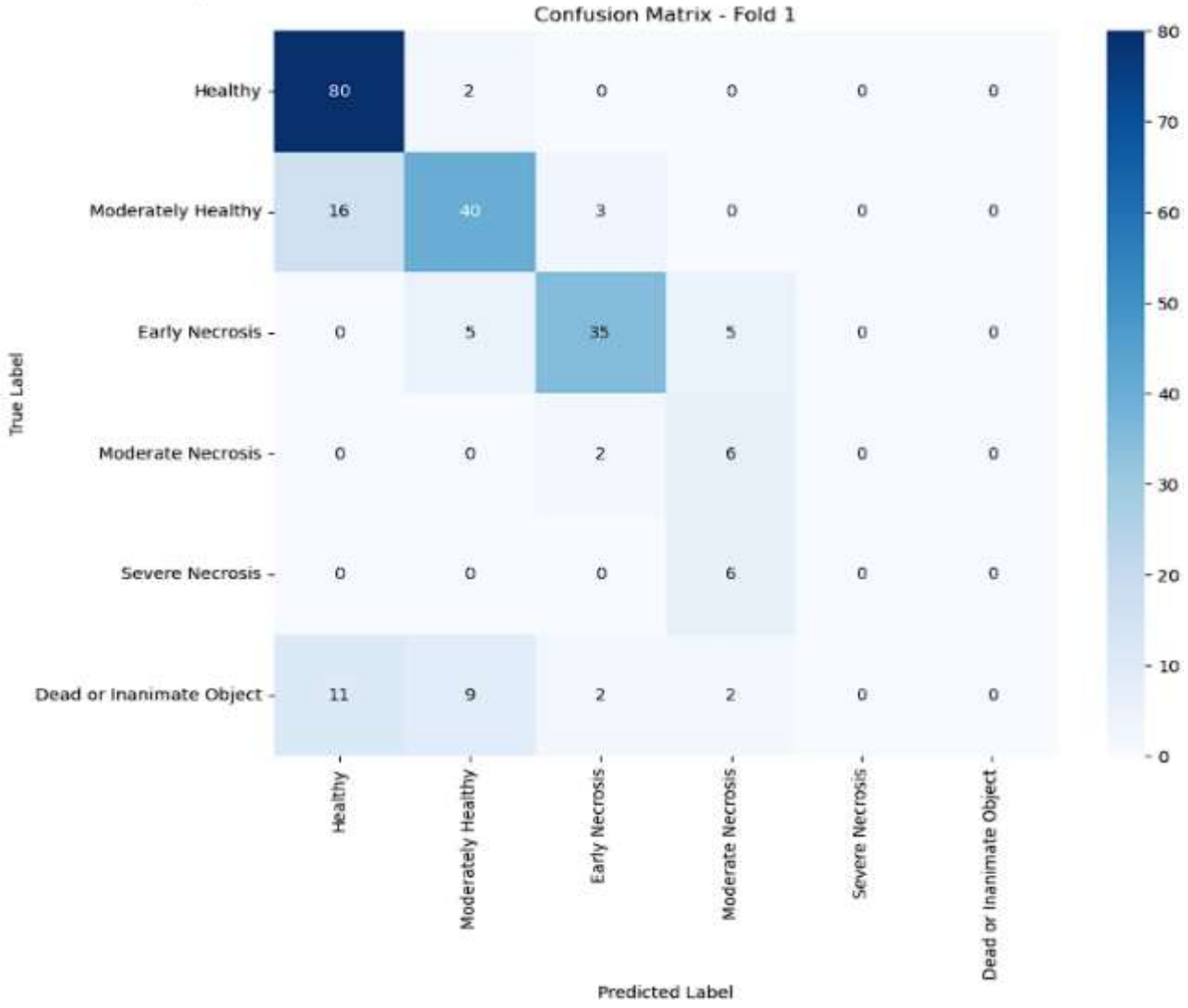

Figure 5.32: Epoch testing

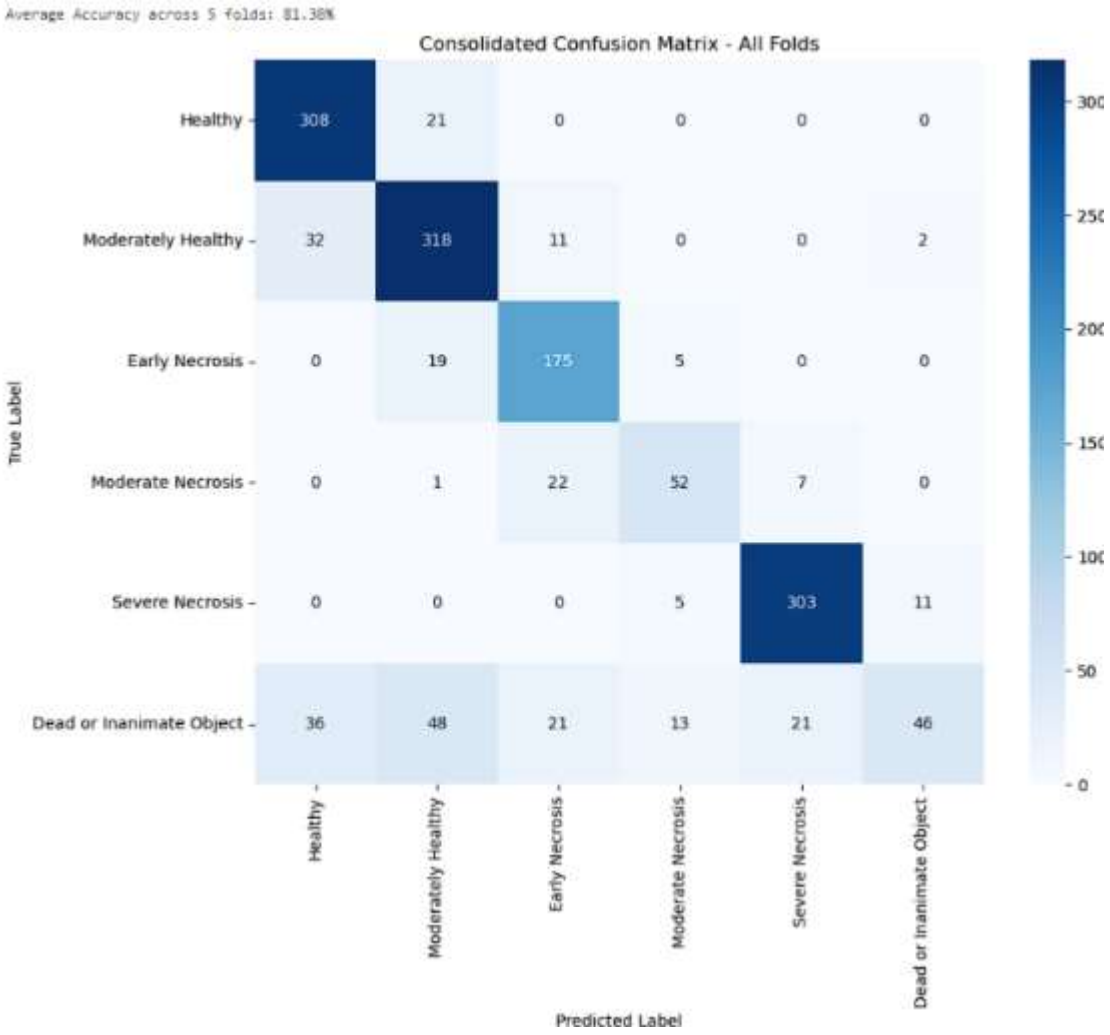

Figure 5.33: Early cross-validation testing

Figures 5.32 and 5.33 above show confusion matrices for the latest revisions of the 3D CNN model. Here I am using cross-validation with 5 folds as well as testing the number of epochs per fold. For the very first fold shown in Figure 5.32, the accuracy was 71.8%. Figure 5.33 on the other hand shows the 6th and final confusion matrix that was displayed at the very end of the training loop after the five other matrices were displayed. At the very top of that figure was the overall accuracy of all five cross-validation folds. The overall accuracy for the model at this point was 81.3 %, however, the performance was still not that great as shown in Figure 5.33, numerous images for “Dead or Inanimate Object” were mislabeled. After this iteration of the model was

tested, I decided to make some major changes to how I would be testing in the future. I extended the number of epochs that would run for each fold as well as changed the display in the confusion matrices. Instead of displaying the total number of images that were in each box, I would now display a percentage. Additionally, I added more print functions that would display what the hyperparameters are for that version of the model as a better way to keep track of the changes I would be making. Figures 5.34 and 5.35 below show these display changes.

```
=== Model Summary ===
Batch Size: 16
Learning Rate: 0.0001

Initial Class Weights for CrossEntropyLoss:
[0.2 0.2 0.2 0.2 0.1 0.1]
Patience: 5
Weight Decay: 1e-06
Number of Epochs: 40
Number of Fully Connected Layers (excluding fc1): 5
Dropout Probability: 0.2

Number of neurons in each fully connected layer:
fc2: (256, 256)
fc3: (256, 128)
fc4: (128, 128)
fc5: (128, 64)
fc6: (64, 6)
Average Accuracy across 5 folds: 81.72%
```

Figure 5.34: Model Summary version 1

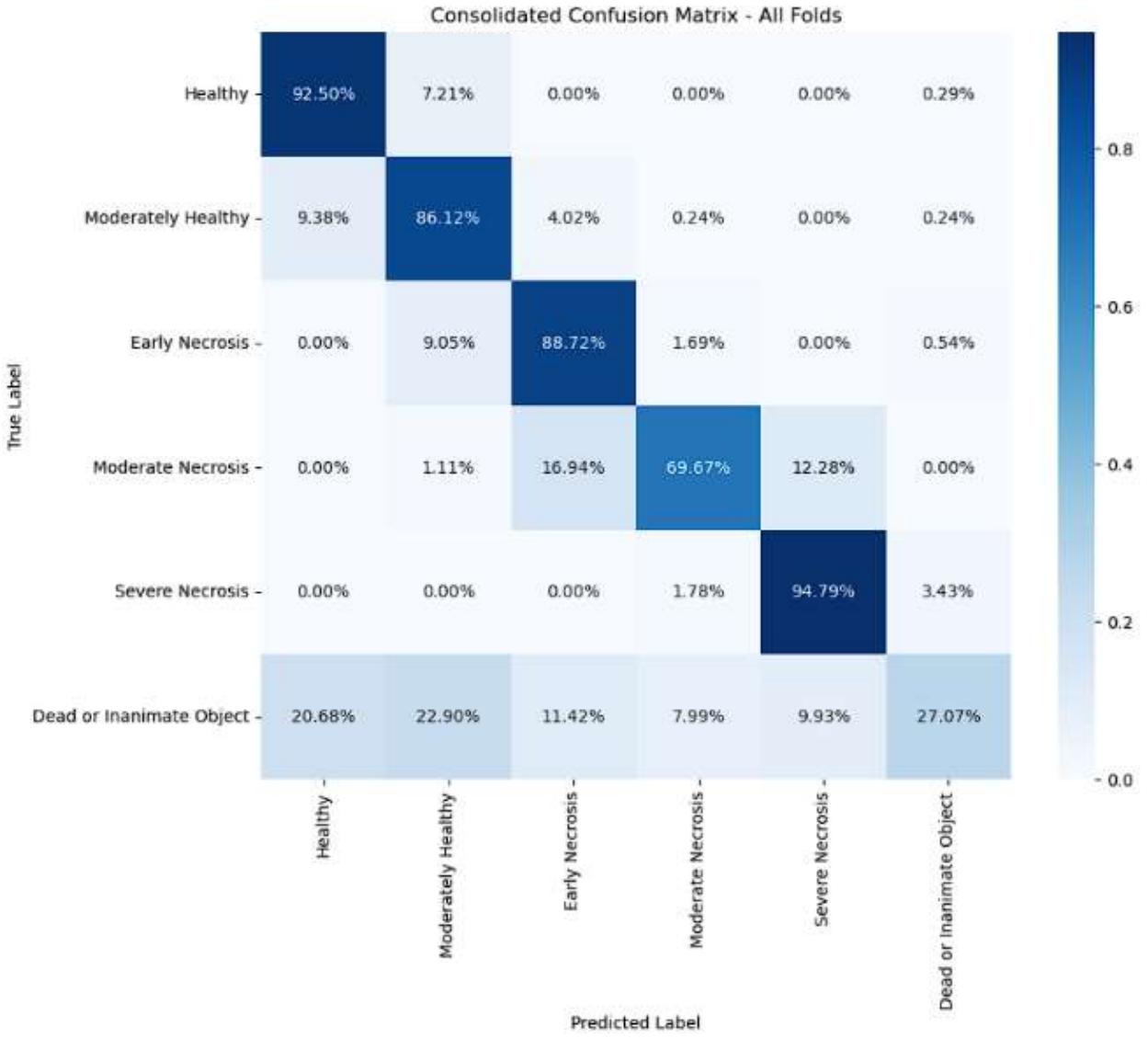

Figure 5.35: Confusion matrix revisions

Figures 5.34 and 5.35 showcase how the model improved by changing the hyperparameters and the exhaustive testing. Figure 5.34 shows the messages that are displayed at the end of the training loop. I have it display what the hyperparameters are for that particular version of the model. Figure 5.35 shows the associated final confusion matrix that displays the overall accuracy of the model. In this version of the model, the overall accuracy had improved up to 81%. However, as shown in Figure 5.34, the model was still struggling with two class labels: “Moderate Necrosis” and “Dead or Inanimate Object”. The preliminary goal was to get each label above 80%. The next revision

for this model was to further implement the early stopping algorithm as well as add the black and white images that were shown in Chapter 4 to help the “Dead or Inanimate Object” label. Figures 5.36 and 5.37 results of these additional changes.

```
=== Model Summary ===
Batch Size: 16
Learning Rate: 0.0001

Initial Class Weights for CrossEntropyLoss (showing up to 6 values per fold):
Fold 1: [0.150, 0.200, 0.200, 0.250, 0.050, 0.150]
Fold 2: [0.150, 0.200, 0.200, 0.250, 0.050, 0.150]
Fold 3: [0.150, 0.200, 0.200, 0.250, 0.050, 0.150]
Fold 4: [0.150, 0.200, 0.200, 0.250, 0.050, 0.150]
Fold 5: [0.150, 0.200, 0.200, 0.250, 0.050, 0.150]

Final Class Weights at End of Training (showing up to 6 values per fold):
Fold 1: [0.006, -0.031, 0.015, -0.024, -0.056, -0.039]
Fold 2: [0.004, -0.034, -0.038, -0.007, -0.033, -0.037]
Fold 3: [-0.039, 0.032, 0.016, -0.001, -0.026, -0.025]
Fold 4: [0.043, 0.065, -0.030, -0.010, 0.020, 0.056]
Fold 5: [-0.021, -0.000, -0.057, -0.065, 0.017, 0.033]
Patience (Early Stopping): 5
Weight Decay: 0.0005
Total Number of Epochs: 50
Number of Epochs completed with Early Stopping per fold: [20, 28, 29, 28, 23]
Number of Fully Connected Layers (excluding fc1): 7
Dropout Probability: 0.2
Average Accuracy across 5 folds: 84.37%
```

Figure 5.36: Early stopping feature model summary

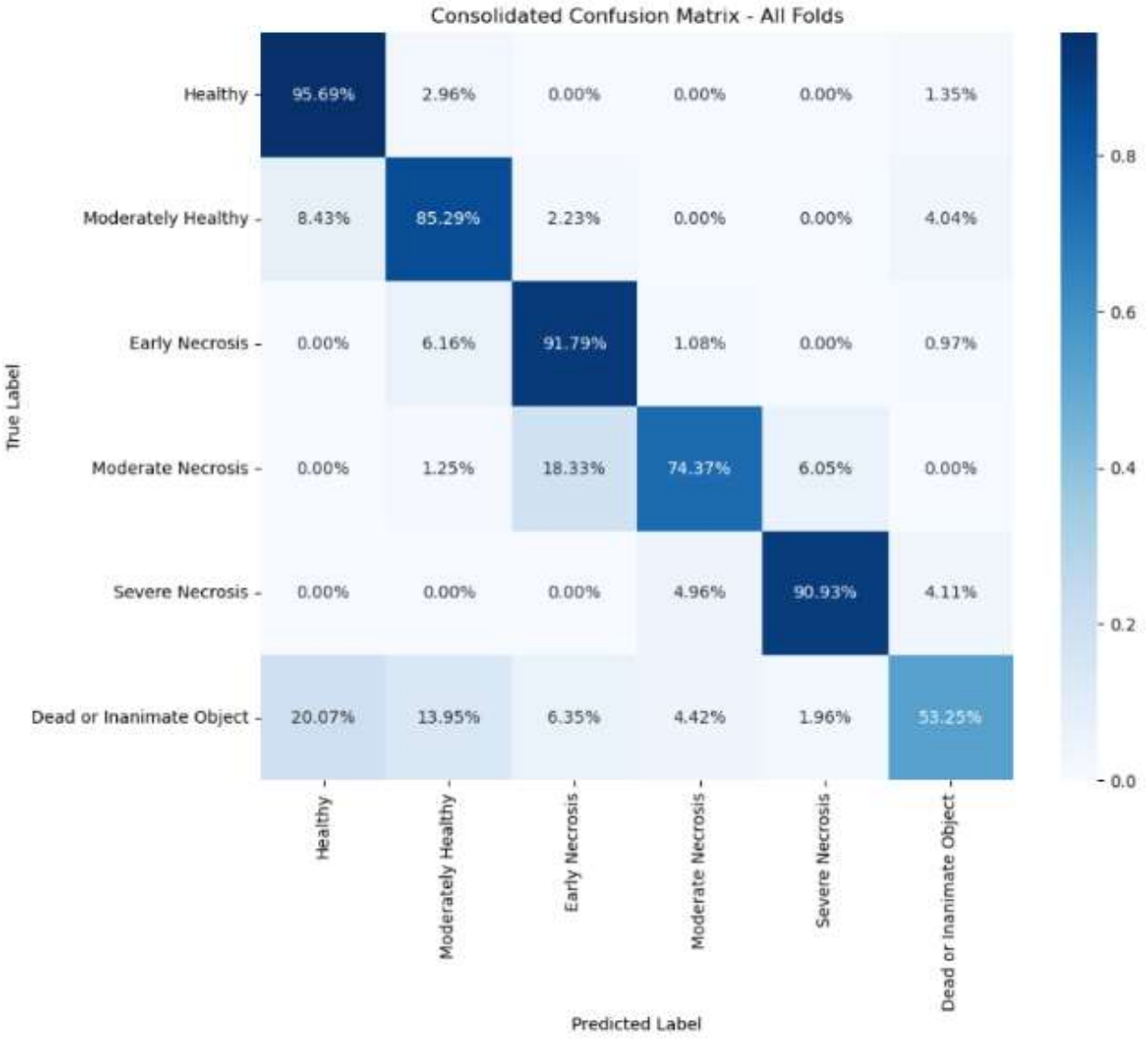

Figure 5.37: Early stopping feature confusion matrix

Figures 5.36 and 5.37 both show the addition of the early stopping algorithm for the training loop. The number of epochs for this training loop was set to 50 and with the early stopping algorithm, it will stop a particular fold from reaching 50 epochs if the validation loss does not improve after 5 epochs. Figure 5.38 below shows how a specific fold is run during the training loop.

```
Fold 1/5
Epoch [1/80], Fold [1/5], Train Loss: 1.7102, Val Loss: 1.7282, Time: 5.36s
Epoch [2/80], Fold [1/5], Train Loss: 1.5467, Val Loss: 1.6365, Time: 3.39s
Epoch [3/80], Fold [1/5], Train Loss: 1.4905, Val Loss: 1.6250, Time: 3.36s
Epoch [4/80], Fold [1/5], Train Loss: 1.4730, Val Loss: 1.5947, Time: 3.40s
Epoch [5/80], Fold [1/5], Train Loss: 1.4338, Val Loss: 1.5633, Time: 3.36s
Epoch [6/80], Fold [1/5], Train Loss: 1.3930, Val Loss: 1.4342, Time: 3.88s
Epoch [7/80], Fold [1/5], Train Loss: 1.2800, Val Loss: 1.2269, Time: 3.43s
Epoch [8/80], Fold [1/5], Train Loss: 1.1969, Val Loss: 1.1906, Time: 3.46s
Epoch [9/80], Fold [1/5], Train Loss: 1.1759, Val Loss: 1.1603, Time: 3.40s
Epoch [10/80], Fold [1/5], Train Loss: 1.1369, Val Loss: 1.1287, Time: 3.43s
Epoch [11/80], Fold [1/5], Train Loss: 1.1175, Val Loss: 1.1416, Time: 3.40s
Epoch [12/80], Fold [1/5], Train Loss: 1.1088, Val Loss: 1.1121, Time: 3.39s
Epoch [13/80], Fold [1/5], Train Loss: 1.0707, Val Loss: 1.1113, Time: 3.43s
Epoch [14/80], Fold [1/5], Train Loss: 1.0741, Val Loss: 1.0776, Time: 3.37s
Epoch [15/80], Fold [1/5], Train Loss: 1.0440, Val Loss: 1.0780, Time: 3.43s
Epoch [16/80], Fold [1/5], Train Loss: 1.0176, Val Loss: 1.0286, Time: 3.36s
Epoch [17/80], Fold [1/5], Train Loss: 0.9826, Val Loss: 0.9723, Time: 3.38s
Epoch [18/80], Fold [1/5], Train Loss: 0.9420, Val Loss: 0.9278, Time: 3.41s
Epoch [19/80], Fold [1/5], Train Loss: 0.9022, Val Loss: 0.8814, Time: 3.40s
Epoch [20/80], Fold [1/5], Train Loss: 0.8806, Val Loss: 0.8383, Time: 3.37s
Epoch [21/80], Fold [1/5], Train Loss: 0.8438, Val Loss: 0.7911, Time: 3.46s
Epoch [22/80], Fold [1/5], Train Loss: 0.8151, Val Loss: 0.7668, Time: 3.45s
Epoch [23/80], Fold [1/5], Train Loss: 0.7744, Val Loss: 0.7481, Time: 3.43s
Epoch [24/80], Fold [1/5], Train Loss: 0.7660, Val Loss: 0.6457, Time: 3.43s
Epoch [25/80], Fold [1/5], Train Loss: 0.7341, Val Loss: 0.6191, Time: 3.43s
Epoch [26/80], Fold [1/5], Train Loss: 0.6688, Val Loss: 0.6496, Time: 3.41s
Epoch [27/80], Fold [1/5], Train Loss: 0.6325, Val Loss: 0.5701, Time: 3.40s
Epoch [28/80], Fold [1/5], Train Loss: 0.6069, Val Loss: 0.5090, Time: 3.38s
Epoch [29/80], Fold [1/5], Train Loss: 0.5739, Val Loss: 0.5157, Time: 3.36s
Epoch [30/80], Fold [1/5], Train Loss: 0.5262, Val Loss: 0.4664, Time: 3.42s
Epoch [31/80], Fold [1/5], Train Loss: 0.5237, Val Loss: 0.4627, Time: 3.35s
Epoch [32/80], Fold [1/5], Train Loss: 0.4873, Val Loss: 0.5484, Time: 3.38s
Epoch [33/80], Fold [1/5], Train Loss: 0.4789, Val Loss: 0.4201, Time: 3.38s
Epoch [34/80], Fold [1/5], Train Loss: 0.4594, Val Loss: 0.4932, Time: 3.35s
Epoch [35/80], Fold [1/5], Train Loss: 0.4395, Val Loss: 0.4296, Time: 3.41s
Epoch [36/80], Fold [1/5], Train Loss: 0.4656, Val Loss: 0.4457, Time: 3.35s
Epoch [37/80], Fold [1/5], Train Loss: 0.4478, Val Loss: 0.4254, Time: 3.39s
Epoch [38/80], Fold [1/5], Train Loss: 0.3891, Val Loss: 0.4945, Time: 3.48s
Early stopping at epoch 38 for Fold 1/5
Fold 1/5 Test Accuracy: 83.09%
```

Figure 5.38: Fold 1 of 5 of the model results

Figure 5.38 displays the first of five folds used in the cross-validation process. Each fold is evaluated for accuracy once training is halted by the early stopping mechanism, which in this instance occurs at epoch 38. Looking specifically at the column labeled “Val Loss,” it reflects the difference between the model’s predictions and the actual class labels. This value is calculated at each epoch during training as the model updates its internal parameters through backpropagation. Validation loss evaluates how well the model performs on a separate dataset that was not used during training, providing insight into the model’s ability to generalize to unseen inputs. A lower

validation loss typically indicates better generalization capability. In this example, early stopping was applied with a patience setting of 5 epochs, meaning training is terminated if no improvement is seen in the validation loss for five epochs in a row. The goal of this technique is to preserve the model state that achieved the best validation performance. As shown in Figure 5.38, epoch 35 produced the lowest validation loss value of 0.4296. Since the loss values in epochs 36 through 38 did not improve upon that, the training loop was stopped at epoch 38, and the model reverted to the weights and performance metrics from epoch 35.

5.3 Final Results

This section presents the final results of this project, summarizing the impact of hyperparameter tuning on the model's performance. Through extensive testing of various parameters-including the number of hidden layers, epoch count, batch size, neuron count, initial weights, learning rate, and weighted decay the model achieved an overall accuracy ranging from 83% to 85% in diagnosing plant health based on necrosis. The variation in accuracy is due to the randomness caused by cross-validation. Every time the model is trained, the cross-validation will slightly modify the distribution of leaf types inside the training and testing sets. The dataset consists of 1,717 sweet potato leaf images, which include the healthy green leaves, the moderately healthy green-yellow leaves, and the unhealthy yellow, yellow-brown, and brown leaves. Because of these variations, each cross-validation fold may have a slightly different ratio of leaf categories, leading to minor fluctuations in the final accuracy score. Figures 5.39 and 5.40 illustrate the model's accuracy with the most recent training run, with Figure 5.39 showing the hyperparameters and overall accuracy of 85.13%.

```
=== Model Summary ===
Batch Size: 16
Learning Rate: 0.0001

Initial Class Weights for CrossEntropyLoss (showing up to 6 values per fold):
Fold 1: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 2: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 3: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 4: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 5: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]

Final Class Weights at End of Training and Accuracy per Fold (showing up to 6 values per fold):
Fold 1: Weights: [0.046, 0.022, -0.043, 0.018, -0.009, -0.049], Accuracy: 86.59%
Fold 2: Weights: [0.032, -0.030, 0.032, -0.029, 0.000, 0.029], Accuracy: 84.84%
Fold 3: Weights: [-0.013, 0.033, 0.011, 0.004, 0.040, 0.002], Accuracy: 86.30%
Fold 4: Weights: [0.016, 0.036, 0.058, -0.007, -0.030, -0.031], Accuracy: 83.97%
Fold 5: Weights: [-0.026, 0.016, 0.015, -0.032, 0.004, 0.050], Accuracy: 83.97%
Patience (Early Stopping): 5
Weight Decay: 0.0005
Total Number of Epochs: 80
Number of Epochs completed with Early Stopping per fold: [46, 51, 41, 38, 38]
Number of Fully Connected Layers (excluding fc1): 7
Dropout Probability: 0.4
Average Accuracy across 5 folds: 85.13%
```

Figure 5.39: Model summary for the current model setup

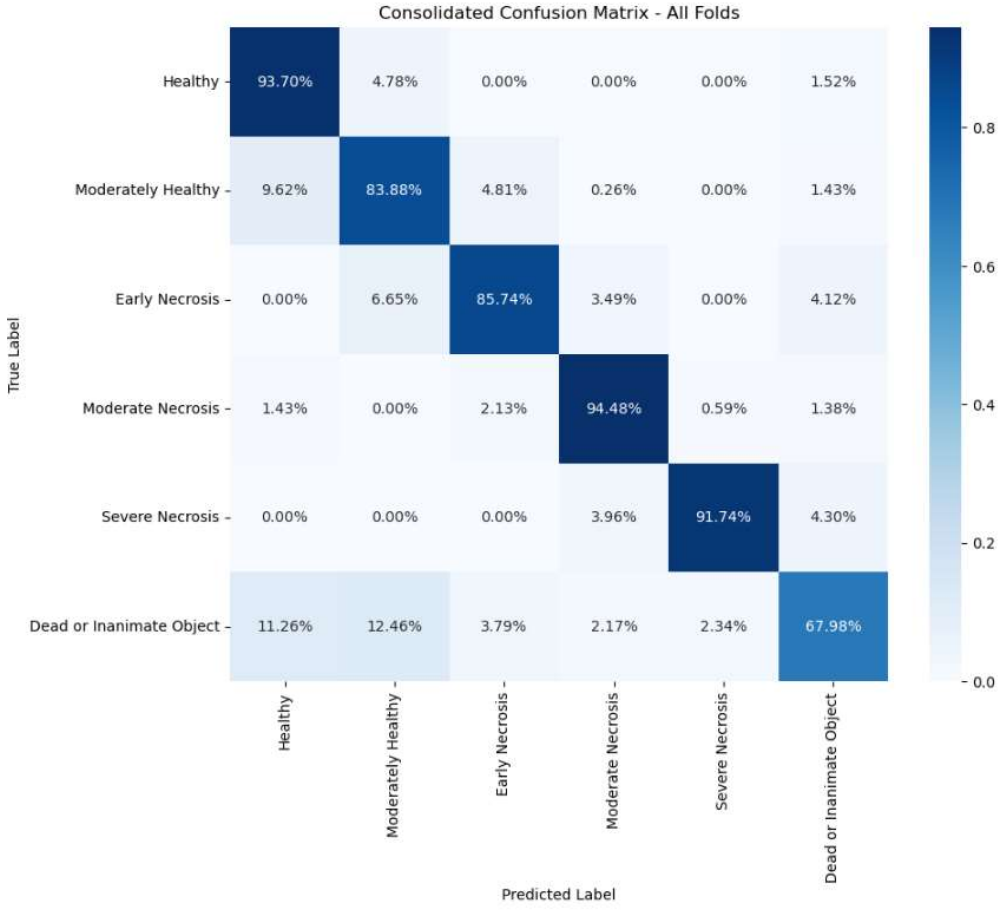

Figure 5.40: Consolidated confusion fusion matrix for the current model setup

Both Figures 5.39 and 5.40 provide insights into the model's performance based on the current hyperparameter configuration shown in Figure 5.39. Through extensive testing, I determined that the 3D CNN's architecture with six fully connected hidden layers, each containing 256 neurons, produced the most consistent results. The batch size was set to 80 to allow a significant time buffer for the early stopping algorithm. A learning rate of 0.0001 and a weight decay of 0.0005 were selected to balance convergence speed and generalization and prevent drastic weight updates that could derail the training process. Additionally, the early stopping algorithm had a patience of 5 as I have found that to be the most optimal threshold allowing the training to halt at the right time. It was found to be not too early and not too late. The dropout was set to 0.4, which means that around 40% of the neurons in each layer will get randomly set to 0. The initial weights are currently set to 0.2, 0.2, 0.2, 0.2, 0.10, 0.4 where the first 0.2 corresponds to the first label (Healthy) and the 0.4 corresponds to the sixth label (Dead or Inanimate Object). With these parameters, the model achieved an average accuracy ranging between 83% and 85%.

Figure 5.40 shows the consolidated confusion matrix related to Figure 5.39. It shows the classification performance across the six class labels. Five of the six labels consistently achieved an accuracy above 85%. In some cases, one or two of the labels would be slightly below 85% as shown in Figure 5.41 and in some instances, most of the labels would exceed 90%. However, due to randomness in the training and testing sets, minor fluctuations can occur, causing slight variations in classification accuracy for different folds and for the consolidated confusion matrix at the end. The "Dead or Inanimate Object" class exhibits the lower accuracy which can be attributed to the limited number of training samples for that category. This particular class label accuracy fluctuated between 55% and 75%. As discussed in Chapter 4, the dataset contains approximately 100 black and white images that represent this class. Even after adjusting the initial

class weights, the model struggled to effectively learn this class at a higher accuracy. This suggests that increasing the number of training samples for this particular label and continuing to adjust its initial weights could significantly improve classification accuracy for both the label and the model.

```
=== Model Summary ===
Batch Size: 16
Learning Rate: 0.0001

Initial Class Weights for CrossEntropyLoss:
Fold 1: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 2: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 3: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 4: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 5: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]

Final Class Weights at End of Training and Accuracy per Fold:
Fold 1: Weights: [0.010, 0.051, -0.003, 0.003, -0.009, -0.038], Accuracy: 85.71%
Fold 2: Weights: [0.054, -0.029, 0.072, 0.028, -0.029, 0.044], Accuracy: 84.55%
Fold 3: Weights: [0.024, 0.020, -0.019, -0.038, 0.005, 0.024], Accuracy: 84.26%
Fold 4: Weights: [0.013, 0.025, -0.040, -0.016, 0.017, 0.027], Accuracy: 83.09%
Fold 5: Weights: [-0.013, -0.003, 0.054, -0.005, -0.041, 0.050], Accuracy: 79.01%
Patience (Early Stopping): 5
Weight Decay: 0.0005
Total Number of Epochs: 80
Number of Epochs completed with Early Stopping per fold: [37, 42, 53, 39, 46]
Number of Fully Connected Layers (excluding fc1): 7
Dropout Probability: 0.4
Average Accuracy across 5 folds: 83.32%
```

Figure 5.41: Extra testing model summary 1

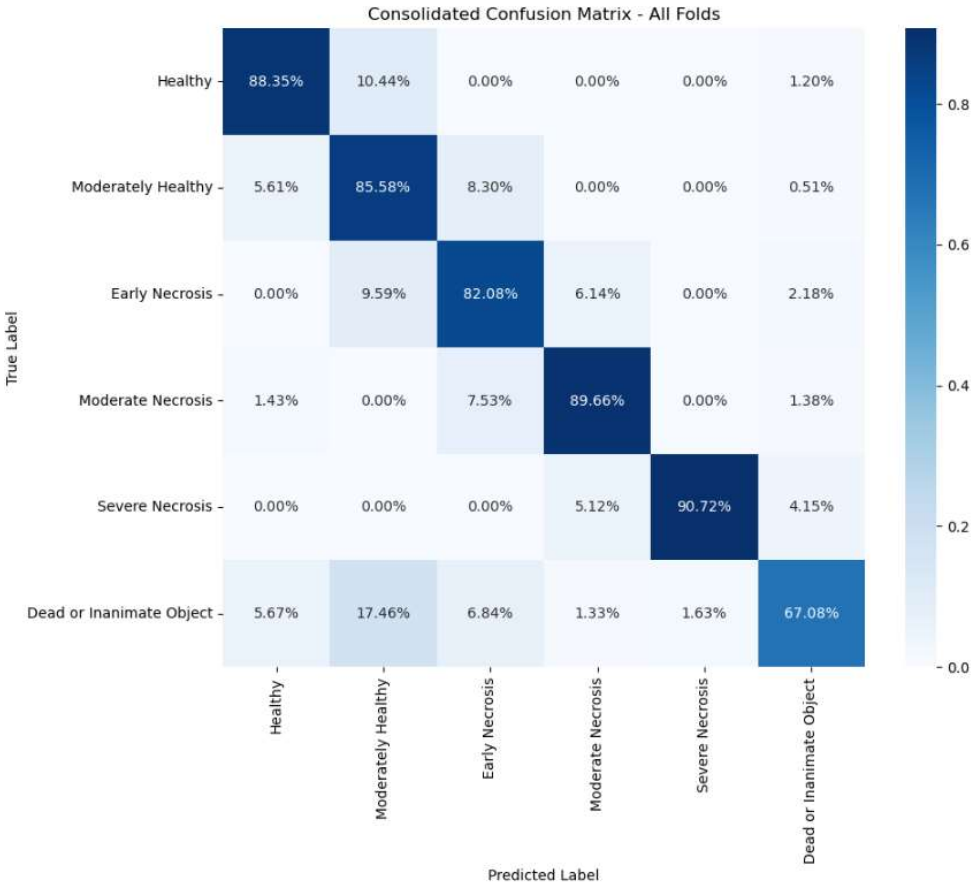

Figure 5.42: Extra testing confusion matrix 1

```
=== Model Summary ===
Batch Size: 16
Learning Rate: 0.0001

Initial Class Weights for CrossEntropyLoss:
Fold 1: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 2: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 3: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 4: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 5: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]

Final Class Weights at End of Training and Accuracy per Fold:
Fold 1: Weights: [-0.022, 0.024, -0.058, 0.026, -0.024, 0.030], Accuracy: 85.71%
Fold 2: Weights: [0.020, 0.058, -0.003, 0.019, -0.026, -0.048], Accuracy: 84.26%
Fold 3: Weights: [-0.028, 0.004, 0.033, 0.010, 0.049, 0.054], Accuracy: 83.38%
Fold 4: Weights: [-0.018, -0.049, 0.049, -0.026, 0.038, -0.033], Accuracy: 85.71%
Fold 5: Weights: [-0.005, -0.055, 0.031, -0.033, -0.028, -0.030], Accuracy: 83.67%
Patience (Early Stopping): 5
Weight Decay: 0.0005
Total Number of Epochs: 80
Number of Epochs completed with Early Stopping per fold: [43, 36, 40, 47, 46]
Number of Fully Connected Layers (excluding fc1): 7
Dropout Probability: 0.4
Average Accuracy across 5 folds: 84.55%
```

Figure 5.43: Extra testing model summary 2

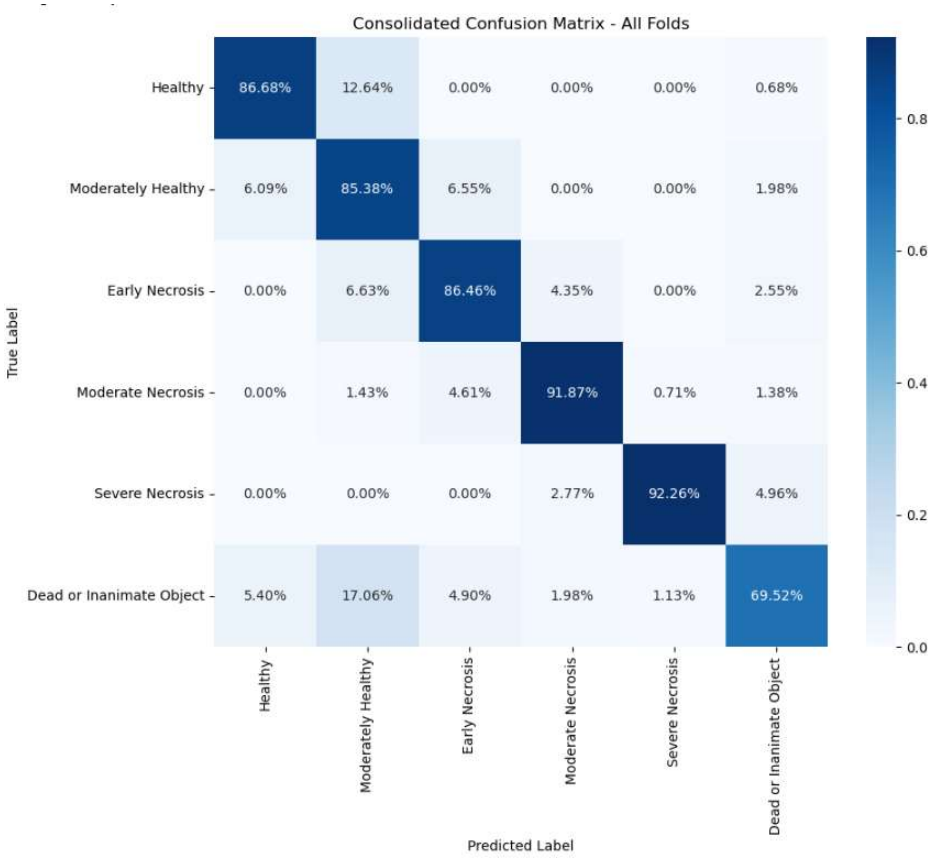

Figure 5.44: Extra testing confusion matrix 2

```
=== Model Summary ===
Batch Size: 16
Learning Rate: 0.0001

Initial Class Weights for CrossEntropyLoss:
Fold 1: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 2: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 3: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 4: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]
Fold 5: [0.20, 0.20, 0.20, 0.20, 0.10, 0.40]

Final Class Weights at End of Training and Accuracy per Fold:
Fold 1: Weights: [0.025, -0.003, -0.029, 0.013, -0.026, 0.063], Accuracy: 86.30%
Fold 2: Weights: [-0.028, 0.038, -0.021, -0.044, 0.028, 0.013], Accuracy: 84.26%
Fold 3: Weights: [0.039, -0.011, -0.027, 0.061, 0.060, -0.025], Accuracy: 81.05%
Fold 4: Weights: [0.037, 0.070, 0.030, -0.043, 0.005, 0.054], Accuracy: 79.88%
Fold 5: Weights: [0.031, -0.047, 0.053, 0.034, -0.039, 0.017], Accuracy: 80.17%
Patience (Early Stopping): 5
Weight Decay: 0.0005
Total Number of Epochs: 80
Number of Epochs completed with Early Stopping per fold: [42, 39, 44, 43, 34]
Number of Fully Connected Layers (excluding fc1): 7
Dropout Probability: 0.4
Average Accuracy across 5 folds: 82.33%
```

Figure 5.45: Extra testing model summary 3

Figure 5.46: Extra testing confusion matrix 3

Figures 5.41 to 5.46 illustrate the model's performance across multiple training loops. All training loops above used the same hyperparameters as shown in Figure 5.39. These results validate the model's consistency in achieving an overall accuracy range between 83% and 85%. The fluctuations in accuracy across different training loops highlight that, even after exhaustive hyperparameter tuning and modifications, some level of variability is inevitable due to differences in the training and testing set distributions. Additionally, the figures above confirm that the sixth label, "Dead or Inanimate Object," remains the poorest-performing category, reinforcing the earlier observation that this label lacks sufficient data , leading to lower classification accuracy.

Chapter 6

Conclusion

In conclusion, this project successfully implemented a 3D Convolutional Neural Network for plant health diagnostics using hyperspectral imaging. The model effectively classified sweet potato leaves into six distinct label categories, demonstrating the capabilities of both three-dimensional convolutional neural networks and the hyperspectral data for image classification. This project showcased the potential applications of hyperspectral imaging and deep learning in smart farming, particularly in automated plant health monitoring. The model achieved an overall accuracy ranging from 83% to 85. Five out of the six class labels consistently exceeded 85% accuracy, reinforcing the reliability of this approach for plant health diagnostics.

Chapter 7 Despite my success in completing this project, I did encounter some limitations, particularly in classifying the "Dead or Inanimate Object" label. The issue was a slight data imbalance, as this label had significantly fewer training samples than the other five labels. As a result, the model struggled to accurately classify these types of images, which led to a lower accuracy for this specific label and a lower overall accuracy for the model. Adding 200 to 300 more black-and-white images to the dataset could likely improve this label and increase the model's overall accuracy.

Chapter 8 Another limitation of this project was the experimental setup. Ideally, the imaging environment should have included a fully enclosed setup for more consistent lighting conditions. Additionally, the distance between the light source and the sample could have been optimized to reduce the over-illumination. As mentioned earlier, not tilting the backdrop would cause unwanted light bleeding that would impact the spectral images quality. Having a better setup would mitigate this. Moreover, a more stable mounting system for the camera would have improved image

consistency. The current camera placement was designed to mimic vertical farming conditions, as it is intended to be integrated into the Astro Cultivators' robotic arm system for automated plant health monitoring.

Chapter 9 Further improvements to this project could include expanding the dataset by collecting additional sweet potato leaf images or incorporating images from other plant species. Additionally, refining the labeling process and experimenting with alternative data augmentation techniques could further improve classification accuracy. Further adjustments to hyperparameters, like modifying the number of hidden layers or applying other initial weight values, could improve the model's performance more. This model focuses solely on the NDVI calculation and can be expanded to include additional plant health metrics, such as chlorophyll content, chlorophyll absorption, photochemical reflectance, plant senescence, pigment index, and carotenoid reflectance. However, integrating more health metrics would require processing different spectral band indices, which could introduce greater storage and computational complexity during the preprocessing and data-loading stages.

References

[1] *"What is image classification?"* GeeksforGeeks, 20 June 2024. [Online]. Available: www.geeksforgeeks.org/what-is-image-classification/.

[2] Prashanth, *"RGB,"* SPS Prashanth Blog, Aug. 2016. [Online]. Available: spsprashanth.blogspot.com/2016/08/rgb.html.

[3] *"The farmer's drone camera: RGB, multispectral, and LIDAR explained,"* DroneFly, 4 Mar. 2024. [Online]. Available: https://www.dronefly.com/blogs/tech-breakdown/rgb-multispectral-and-lidar-for-farming-drones

[4] T. S. Le, R. Harper, and B. Dell, "Application of remote sensing in detecting and monitoring water stress in forests," *Remote Sensing*, vol. 15, article 3360, 2023. [Online]. Available: https://doi.org/10.3390/rs15133360.

[5] Morteza, *"Hyperspectral image classification using CNN in TensorFlow,"* YouTube, 30 May 2024. [Online]. Available: www.youtube.com/watch?v=xfq-8EIF1D4.

[6] *"Hyperspectral imaging in agriculture and vegetation,"* Specim. [Online]. Available: www.specim.com/hyperspectral-imaging-applications/agriculture-and-vegetation.

[7] X. Jai, C. Dongye, Z. Zhang, and Z. Zhao, *"All in one: RGB, RGB-D, and RGB-T salient object detection,"* arXiv, arXiv:2311.14746v1 [cs.CV], 23 Nov. 2023. [Online]. Available: https://arxiv.org/abs/2311.14746.

[8] S. Savary, L. Willocquet, S. J. Pethybridge, *et al.*, "The global burden of pathogens and pests on major food crops," *Nat. Ecol. Evol.*, vol. 3, pp. 430–439, 2019. [Online]. Available: https://doi.org/10.1038/s41559-018-0793-y.

[9] A. Yee-Rendon, I. Torres-Pacheco, A. S. Trujillo-Lopez, K. P. Romero-Bringas, and J. R. Millan-Almaraz, "Analysis of new RGB vegetation indices for PHYVV and TMV identification in jalapeño pepper (*Capsicum annuum*) leaves using CNNs-based model," *Plants*, vol. 10, article 1977, 2021. [Online]. Available: https://doi.org/10.3390/plants10101977.

[10] I. Sa, *et al.*, "weedNet: Dense semantic weed classification using multispectral images and MAV for smart farming," *arXiv*, 11 Sept. 2017. [Online]. Available: arXiv:1709.03329v1.

[11] A. Khan, *et al.*, "Real-time plant health assessment via implementing cloud-based scalable transfer learning on AWS DeepLens," *arXiv*, arXiv:2009.04110v2 [cs.CV], 10 Sept. 2020. [Online]. Available: https://arxiv.org/abs/2009.04110.

[12] A. Yee-Rendon, I. Torres-Pacheco, A. S. Trujillo-Lopez, K. P. Romero-Bringas, and J. R. Millan-Almaraz, "Analysis of new RGB vegetation indices for PHYVV and TMV identification in jalapeño pepper (*Capsicum annuum*) leaves using CNNs-based model," *Plants*, vol. 10, article 1977, 2021. [Online]. Available: https://doi.org/10.3390/plants10101977.

[13] Y. Luo, *et al.*, "HSI-CNN: A novel convolution neural network for hyperspectral image," *Int. Conf. Pattern Recognit.*, 2018.

[14] S. Eshkabilov, *et al.*, "Hyperspectral image data and waveband indexing methods to estimate nutrient concentration on lettuce (*Lactuca sativa L.*) cultivars," *Sensors*, vol. 22, article 8158, 2022. [Online]. Available: https://doi.org/10.3390/s22218158.

[15] D. Hong, *et al.*, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, arXiv:2008.02457v2 [cs.CV], 15 Jan. 2021. [Online]. Available: https://arxiv.org/abs/2008.02457.

[16] M. F. Baumgardner, L. L. Biehl, and D. A. Landgrebe, *Indian Pines*, 12 June 1992. [Online]. Available: https://purr.purdue.edu/publications/1947/citations?v=1.

[17] M. A. Hossen, *et al.*, "Total nitrogen estimation in agricultural soils via aerial multispectral imaging and LIBS," *Sci. Rep.*, vol. 11, no. 1, 2021. [Online]. Available: https://doi.org/10.1038/s41598-021-90624-6.

[18] T. Wei, *et al.*, *PlantSeg: A large-scale in-the-wild dataset for plant disease segmentation*, 2024. [Online]. Available: https://doi.org/10.5281/zenodo.1329389127.

[19] L. Zhang, *et al.*, "Elimination of leaf angle impacts on plant reflectance spectra using fusion of hyperspectral images and 3D point clouds," *Sensors*, vol. 23, article 44, 2023. [Online]. Available: https://doi.org/10.3390/s23010044.

[20] A. Morales-Badajoz, *et al.*, "Astro Cultivators: Autonomous growth system for space farming based on machine vision and multi-sensor fusion," in *Proc. ACM Cyber-Phys. Syst. Internet Things Week Workshops*, San Antonio, TX, USA, 9–12 May 2023. [Online]. Available: https://dl.acm.org/doi/10.1145/3576914.3588338.

[21] V. Lodhi, D. Chakravarty, and P. Mitra, "Hyperspectral imaging system: Development aspects and recent trends," *Sens. Imaging*, vol. 20, article 35, 2019. [Online]. Available: https://doi.org/10.1007/s11220-019-0257-8.

[22] *"Laptop screen pictures, images and stock photos,"* iStock. [Online]. Available: https://www.istockphoto.com/photos/laptop-screen.

[23] Morteza, *"Best way to visualize hyperspectral data in Python,"* YouTube, 7 Mar. 2024. [Online]. Available: https://www.youtube.com/watch?v=2EqA56XgggQ&t=92s.

[24] *"What is hyperspectral imaging: A comprehensive guide,"* Specim. [Online]. Available: https://www.specim.com/technology/what-is-hyperspectral-imaging/.

[25] R. Njambi, *"Full spectrum: Multispectral imagery and hyperspectral imagery,"* Up42, 16 Dec. 2022. [Online]. Available: https://up42.com/blog/full-spectrum-multispectral-imagery-and-hyperspectral-imagery.

[26] *"What is hyperspectral imaging,"* Nireos. [Online]. Available: https://nireos.com/application/what-is-hyperspectral-imaging/.

[27] SpecimSpectral**,** *"What is hyperspectral imaging - tutorial,"* YouTube, 15 Nov. 2019. [Online]. Available: https://www.youtube.com/watch?v=ayp7hP0Xr8Q.

[28] G. Stark, *"Light,"* Encyclopedia Britannica, 3 Feb. 2025. [Online]. Available: https://www.britannica.com/science/light.

[29] K. Spring and M. Davidson, *"Sources of visible light,"* Evident Scientific. [Online]. Available: https://evidentscientific.com/en/microscope-resource/knowledge-hub/lightandcolor/lightsourcesintro.

[30] N. M. Kraetzig, *"5 things to know about NDVI (Normalized Difference Vegetation Index),"* UP42, 1 Sept. 2020. [Online]. Available: https://up42.com/blog/5-things-to-know-about-ndvi.

[31] *"NDVI: Normalized difference vegetation index,"* EOS Data Analytics, 12 Jan. 2023. [Online]. Available: https://eos.com/make-an-analysis/ndvi/.

[32] *"What is NDVI (Normalized Difference Vegetation Index)?,"* GIS Geography, 19 Feb. 2021. [Online]. Available: https://gisgeography.com/ndvi-normalized-difference-vegetation-index/.

[33] *"NDVI (Normalized Difference Vegetation Index) and PRI (Photochemical Reflectance Index) — The researcher's complete guide,"* METER Group. [Online]. Available: https://metergroup.com/education-guides/ndvi-and-pri-the-researchers-complete-guide/.

[34] *"What is NDVI index?,"* PhysicsOpenLab, 30 Jan. 2017. [Online]. Available: https://physicsopenlab.org/2017/01/30/ndvi-index/.

[35] *"Difference between a hyperspectral camera and a regular camera?,"* CHNSpec, 2024. [Online]. Available: https://www.chnspec.net/Difference-hyperspectral-camera-regular-camera.html.

[36] Great Learning Editorial Team, *"Types of neural networks and definition of neural network,"* Great Learning, 4 Aug. 2022. [Online]. Available: https://www.mygreatlearning.com/blog/types-of-neural-networks/.

[37] *"Introduction to convolution neural network,"* GeeksforGeeks, 2024. [Online]. Available: https://www.geeksforgeeks.org/introduction-convolution-neural-network/.

[38] *"Activation functions in neural networks,"* GeeksforGeeks, 2025. [Online]. Available: https://www.geeksforgeeks.org/activation-functions-neural-networks/.

[39] N. Bhargav, *"Neurons in neural networks,"* Baeldung, 2024. [Online]. Available: https://www.baeldung.com/cs/neural-networks-neurons.

[40] S. Ronaghan, *"Deep learning: Overview of neurons and activation functions,"* Medium, 26 July 2018. [Online]. Available: https://srnghn.medium.com/deep-learning-overview-of-neurons-and-activation-functions-1d98286cf1e4.

[41] *"Weights and bias in neural networks,"* GeeksforGeeks, 4 Oct. 2024. [Online]. Available: https://www.geeksforgeeks.org/the-role-of-weights-and-bias-in-neural-networks/.

[42] Manav, *"Introduction to convolutional neural networks (CNN),"* Analytics Vidhya, last updated 21 Nov. 2024. [Online]. Available: https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/.

[43] A. Gillis, *"What is a convolutional neural network (CNN)?,"* TechTarget. [Online]. Available: https://www.techtarget.com/searchenterpriseai/definition/convolutional-neural-network.

[44] *"What is fully connected layer in deep learning?,"* GeeksforGeeks, last updated 27 May 2024. [Online]. Available: https://www.geeksforgeeks.org/what-is-fully-connected-layer-in-deep-learning/.

[45] A. Kumar, *"Different types of CNN architectures explained: Examples,"* VitalFlux, 4 Dec. 2023. [Online]. Available: https://vitalflux.com/different-types-of-cnn-architectures-explained-examples/.

[46] X. Fan and P. Truong, *"An introduction to convolutional neural network (CNN),"* Medium, 10 Feb. 2022. [Online]. Available: https://medium.com/sfu-cspmp/an-introduction-to-convolutional-neural-network-cnn-207cdb53db97.

[47] J. Liu, T. Wang, A. Skidmore, Y. Sun, P. Jia, and K. Zhang, *"Integrated 1D, 2D, and 3D CNNs enable robust and efficient land cover classification from hyperspectral imagery," Remote Sensing*, vol. 15, 2023, article 4797. [Online]. Available: https://doi.org/10.3390/rs15194797.

[48] N. Makkar, L. Yang and S. Prasad, "Adversarial Learning Based Discriminative Domain Adaptation for Geospatial Image Analysis," in *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 150-162, 2022, doi: 10.1109/JSTARS.2021.3132259

[49] P. Mahajan, *"Max pooling,"* Medium, 5 July 2020. [Online]. Available: https://poojamahajan5131.medium.com/max-pooling-210fc94c4f11.

[50] *"Cross validation in machine learning,"* GeeksforGeeks, last updated 28 Feb. 2025. [Online]. Available: https://www.geeksforgeeks.org/cross-validation-machine-learning/.

[51] J. Brownlee, *"A gentle introduction to early stopping to avoid overtraining neural networks,"* Machine Learning Mastery, 6 Aug. 2019. [Online]. Available:

https://machinelearningmastery.com/early-stopping-to-avoid-overtraining-neural-network-models/.